Managing state

07

1. Secrets & configs

2. Mounting volumes

3. Popular volumes

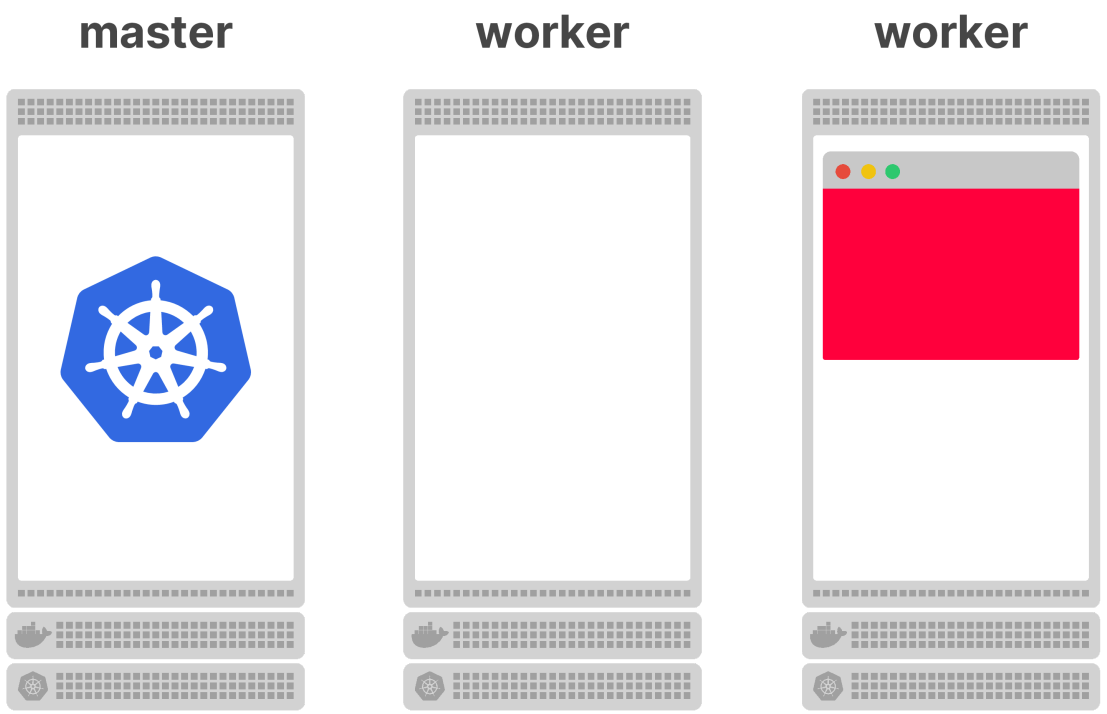
1. Secrets & configs
2. **Mounting volumes**
3. Popular volumes

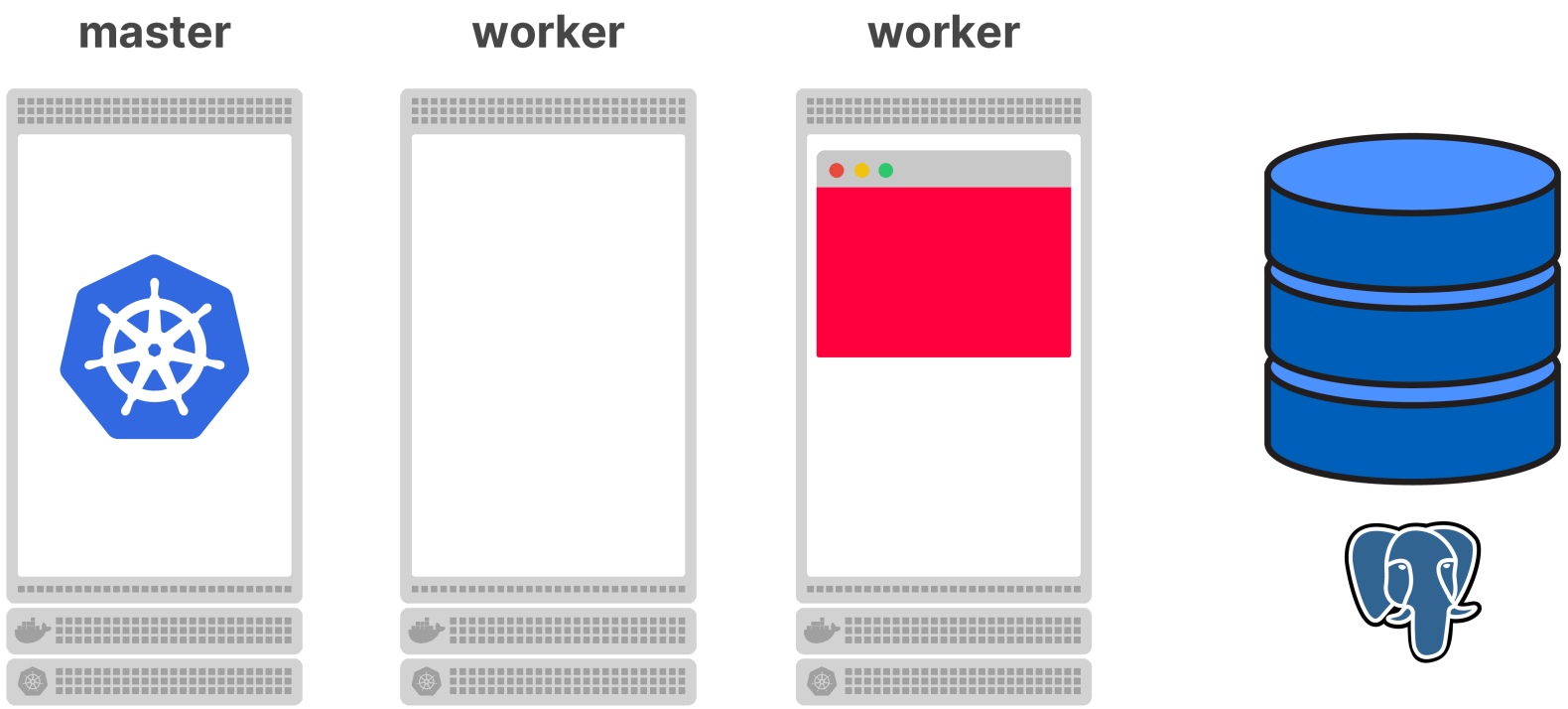
1. Secrets & configs
2. Mounting volumes
3. **Popular volumes**

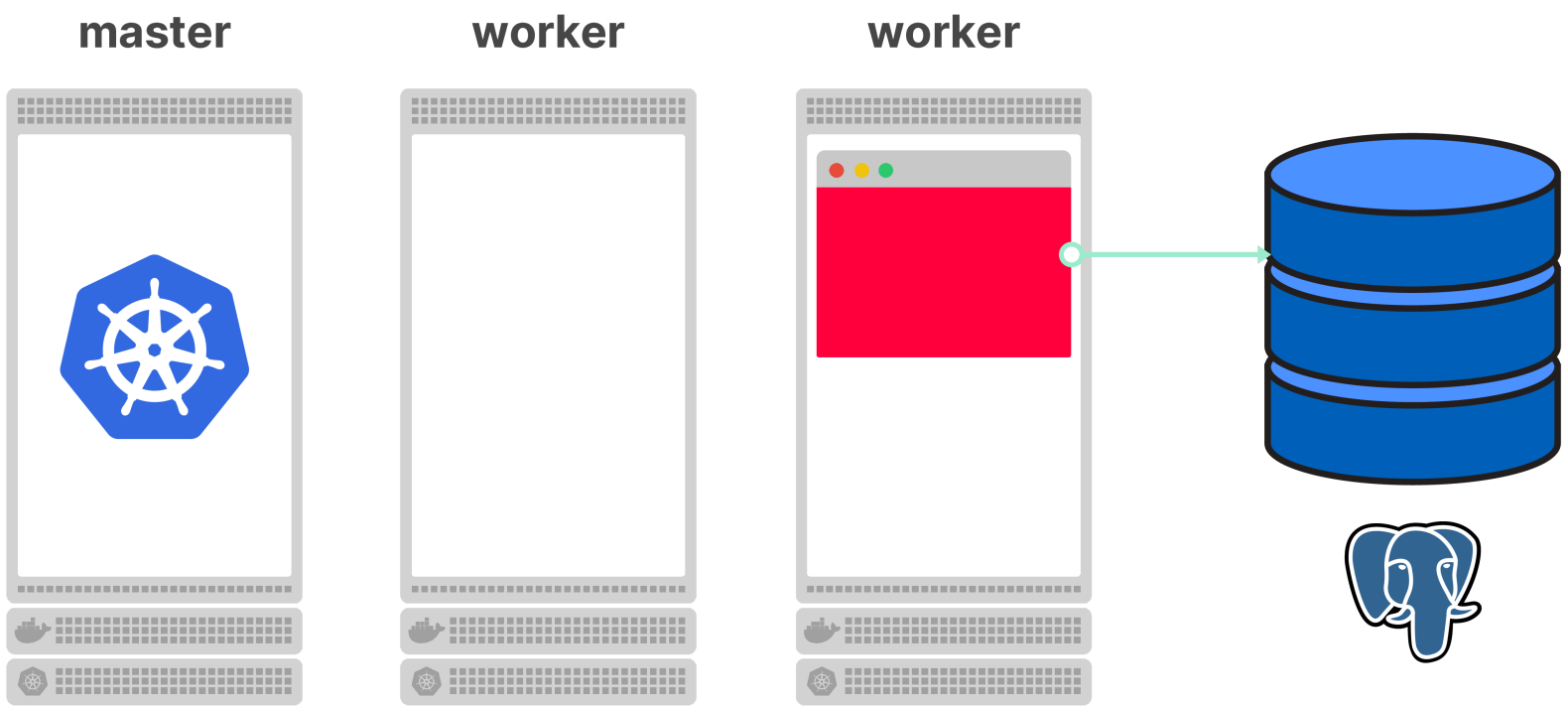
Secrets and configs

01

Connecting to a database


master
worker
worker


master
worker
worker


master
worker
worker

Connecting to a database

Database URL

Username

Password

Connecting to a database

Database URL

Username

Password

Connecting to a database

Database URL

Username

Password

Connecting to a database

Database URL

Username 🔒

Password 🔒

ConfigMaps

```
~$ cat configmap.yaml

apiVersion: v1
kind: ConfigMap
metadata:
  name: my-config
data:
  database_url: https://amazon_rds


~$ _
```

~$ cat configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
name: my-config
data:
database_url: https://amazon_rds
~$ _

```
~$ cat configmap.yaml

apiVersion: v1
kind: ConfigMap
metadata:
  name: my-config
data:
  database_url: https://amazon_rds


~$ _
```

Using a ConfigMap in a Pod

ConfigMaps

```
~$ cat pod.yaml
apiVersion: v1
kind: Pod
spec:
  containers:
    - name: test-container
      image: k8s.gcr.io/busybox
      env:
        - name: DATABASE_URL
          valueFrom:
            configMapKeyRef:
              name: my-config
              key: database_url

~$ _
```

```
~$ cat pod.yaml
apiVersion: v1
kind: Pod
spec:
  containers:
    - name: test-container
      image: k8s.gcr.io/busybox
      env:
        - name: DATABASE_URL
          valueFrom:
            configMapKeyRef:
              name: my-config
              key: database_url

~$ _
```

```
~$ cat pod.yaml
apiVersion: v1
kind: Pod
spec:
  containers:
    - name: test-container
      image: k8s.gcr.io/busybox
      env:
        - name: DATABASE_URL
          valueFrom:
            configMapKeyRef:
              name: my-config
              key: database_url

~$ _
```

```
~$ cat pod.yaml
apiVersion: v1
kind: Pod
spec:
  containers:
    - name: test-container
      image: k8s.gcr.io/busybox
      env:
        - name: DATABASE_URL
          valueFrom:
            configMapKeyRef:
              name: my-config
              key: database_url

~$ _
```

```
~$ cat pod.yaml
apiVersion: v1
kind: Pod
spec:
  containers:
    - name: test-container
      image: k8s.gcr.io/busybox
      env:
        - name: DATABASE_URL
          valueFrom:
            configMapKeyRef:
              name: my-config
              key: database_url

~$ _
```

```
~$ cat pod.yaml
apiVersion: v1
kind: Pod
spec:
  containers:
    - name: test-container
      image: k8s.gcr.io/busybox
      env:
        - name: DATABASE_URL
          valueFrom:
            configMapKeyRef:
              name: my-config
              key: database_url

~$ _
```

Secrets

```
~$ cat secret.yaml

apiVersion: v1
kind: Secret
metadata:
  name: my-secret
type: Opaque
data:
  username: bXlzcWw=
  password: bXlzcWw=


~$ _
```

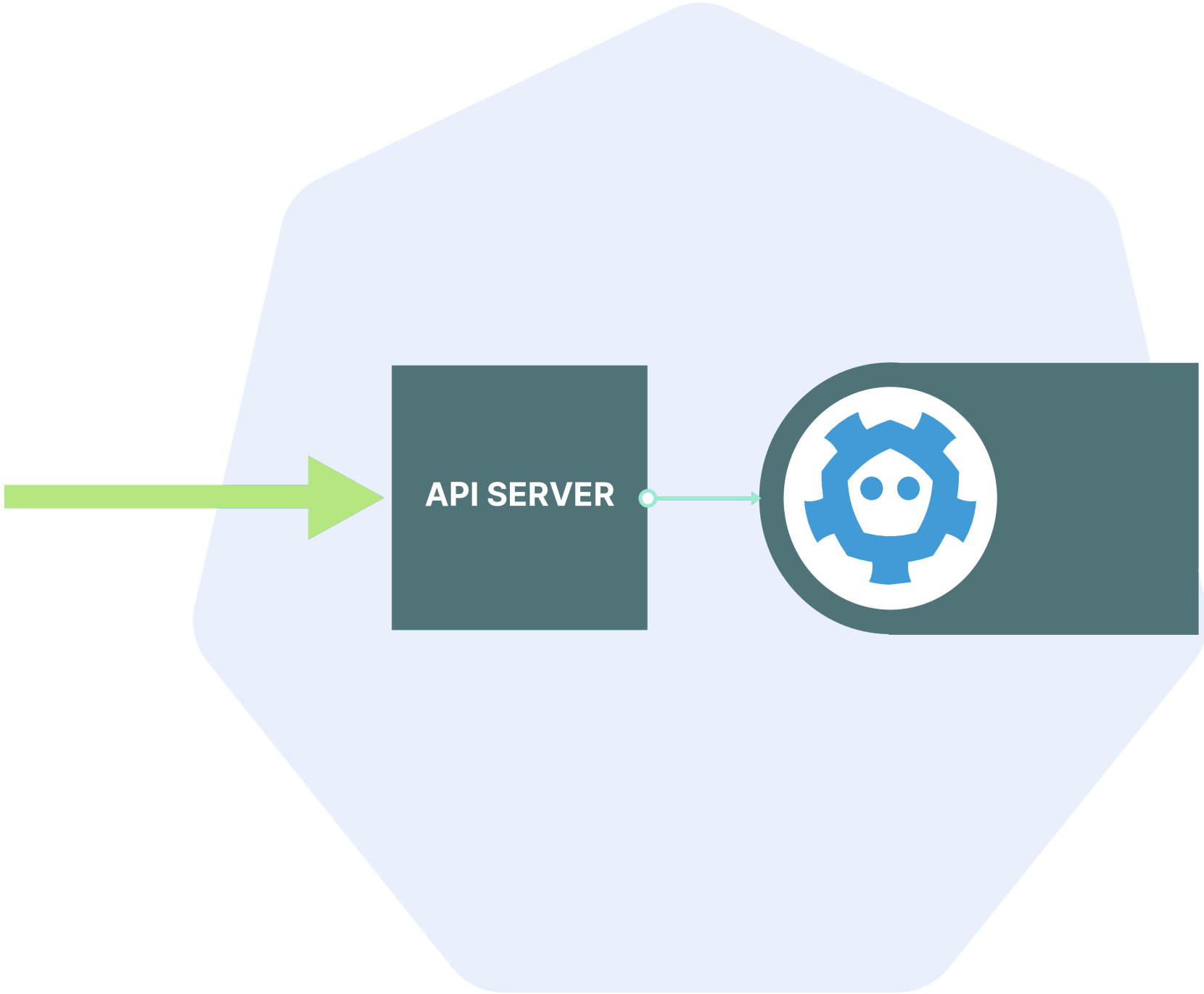
```
~$ cat secret.yaml

apiVersion: v1
kind: Secret
metadata:
  name: my-secret
type: Opaque
data:
  username: bXlzcWw=
  password: bXlzcWw=


~$ _
```

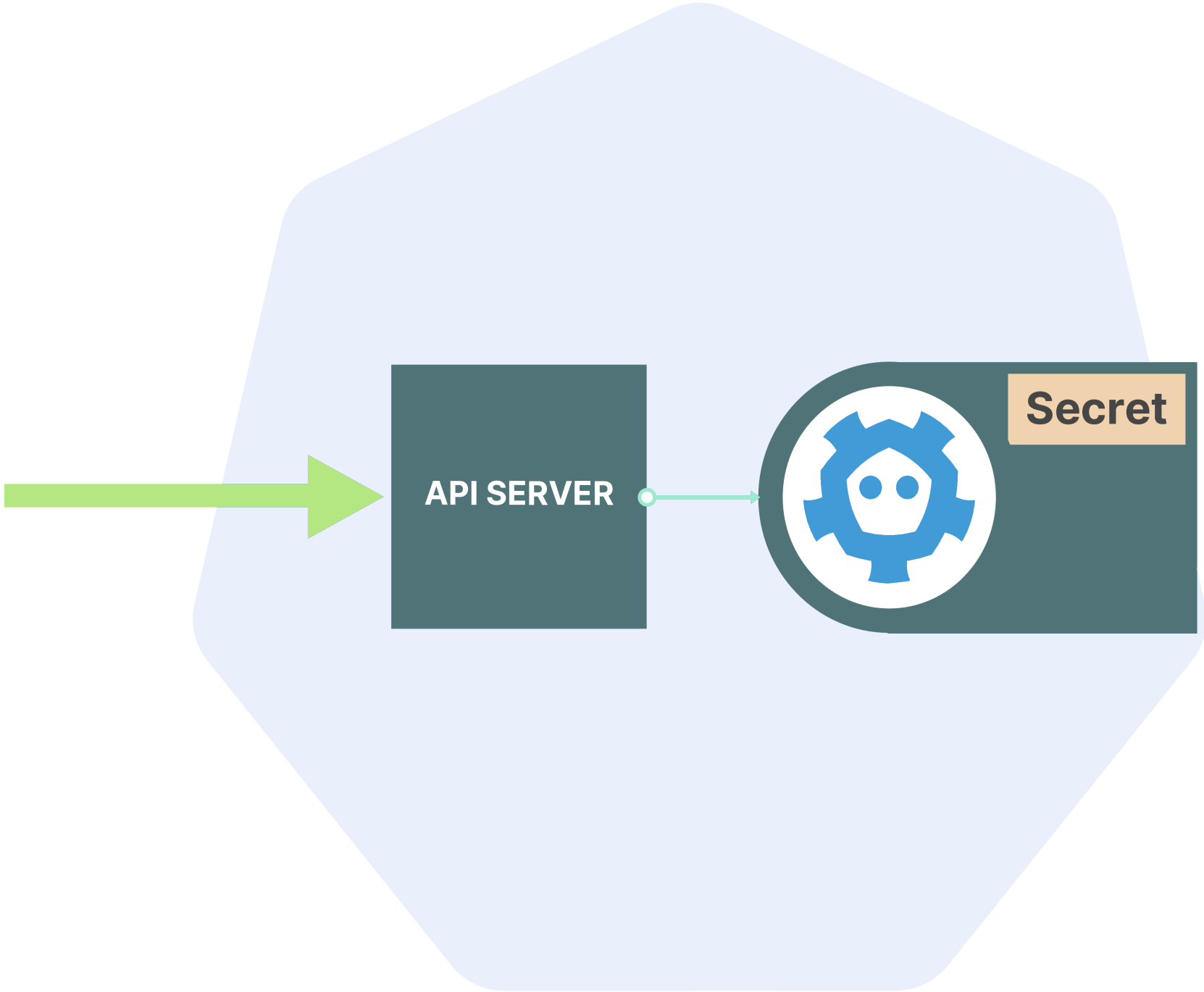
base64 encrypted

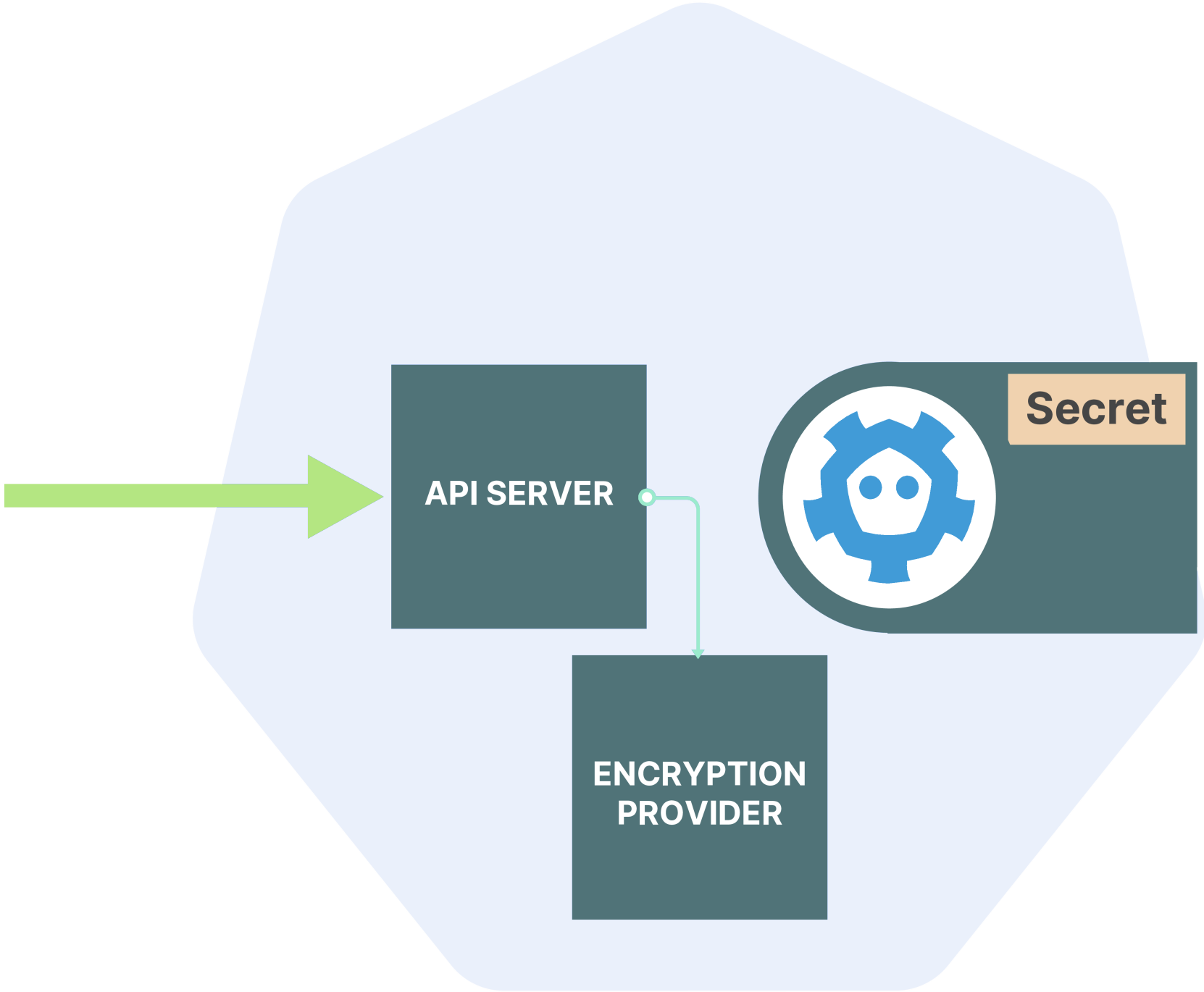
Secrets

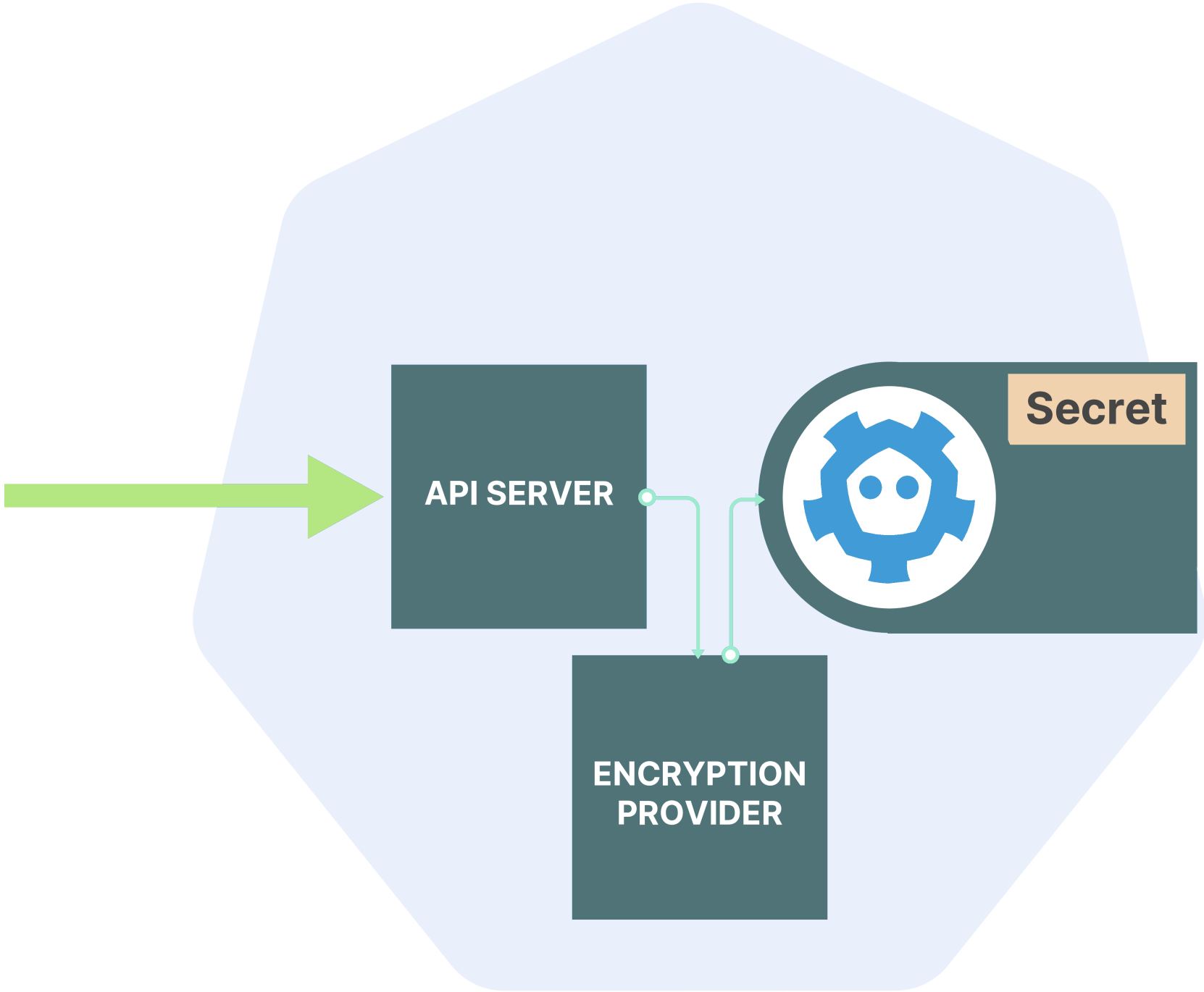
base64 ~~**enerypted**~~
encoded

Secrets


API SERVER


API SERVER
Secret


API SERVER
Secret
ENCRYPTION
PROVIDER


API SERVER
Secret
ENCRYPTION
PROVIDER

Using a Secret in a Pod

Secrets

```
~$ cat pod.yaml
apiVersion: v1
kind: Pod
spec:
  containers:
    - name: test-container
      image: k8s.gcr.io/busybox
      env:
        - name: DATABASE_USERNAME
          valueFrom:
            secretKeyRef:
              name: my-SECRET
              key: username

~$ _
```

```
~$ cat pod.yaml
apiVersion: v1
kind: Pod
spec:
  containers:
    - name: test-container
      image: k8s.gcr.io/busybox
      env:
        - name: DATABASE_USERNAME
          valueFrom:
            secretKeyRef:
              name: my-secret
              key: username

~$ _
```

```
~$ cat pod.yaml
apiVersion: v1
kind: Pod
spec:
  containers:
    - name: test-container
      image: k8s.gcr.io/busybox
      env:
        - name: DATABASE_USERNAME
          valueFrom:
            secretKeyRef:
              name: my-secret
              key: username

~$ _
```

```
~$ cat pod.yaml
apiVersion: v1
kind: Pod
spec:
  containers:
    - name: test-container
      image: k8s.gcr.io/busybox
      env:
        - name: DATABASE_USERNAME
          valueFrom:
            secretKeyRef:
              name: my-secret
              key: username

~$ _
```

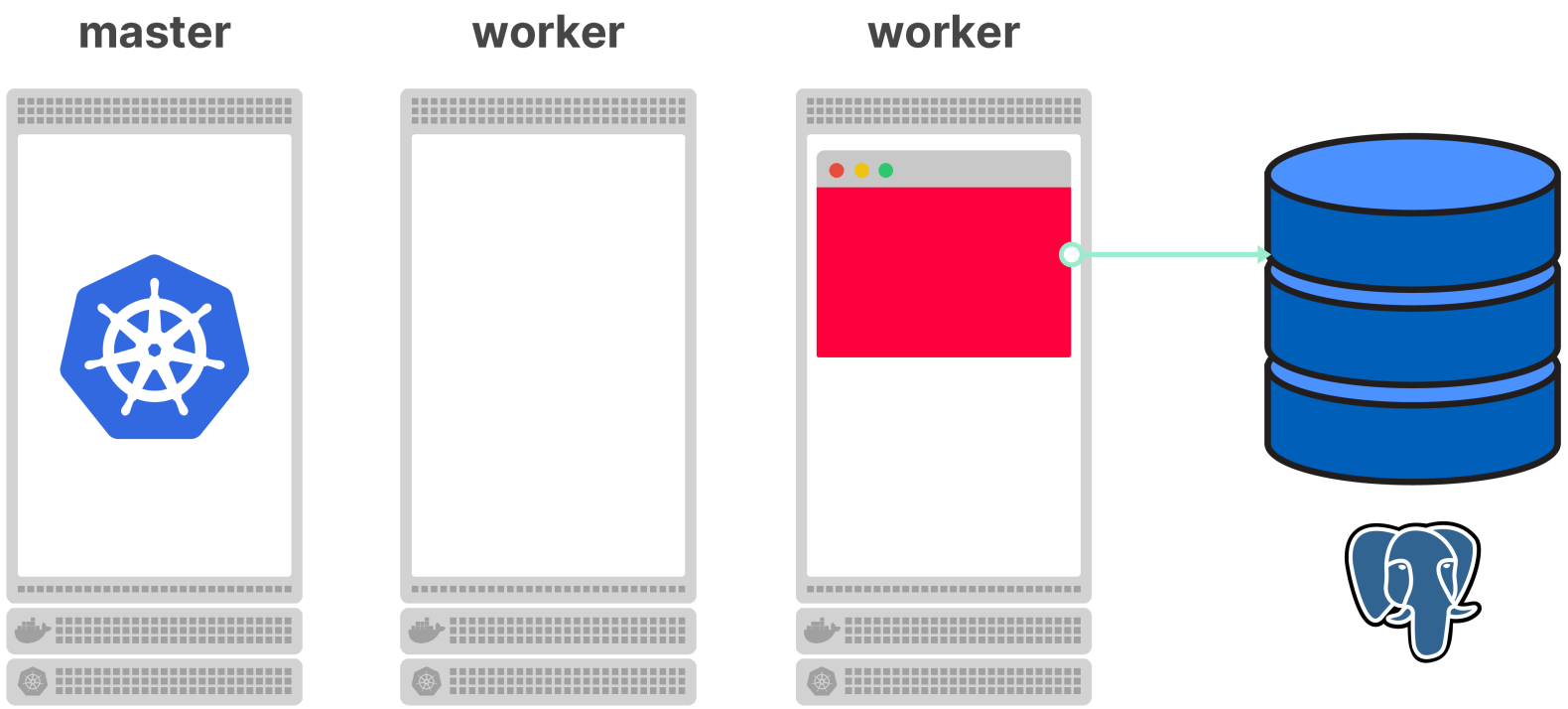
```
~$ cat pod.yaml
apiVersion: v1
kind: Pod
spec:
  containers:
    - name: test-container
      image: k8s.gcr.io/busybox
      env:
        - name: DATABASE_USERNAME
          valueFrom:
            secretKeyRef:
              name: my-secret
              key: username

~$ _
```

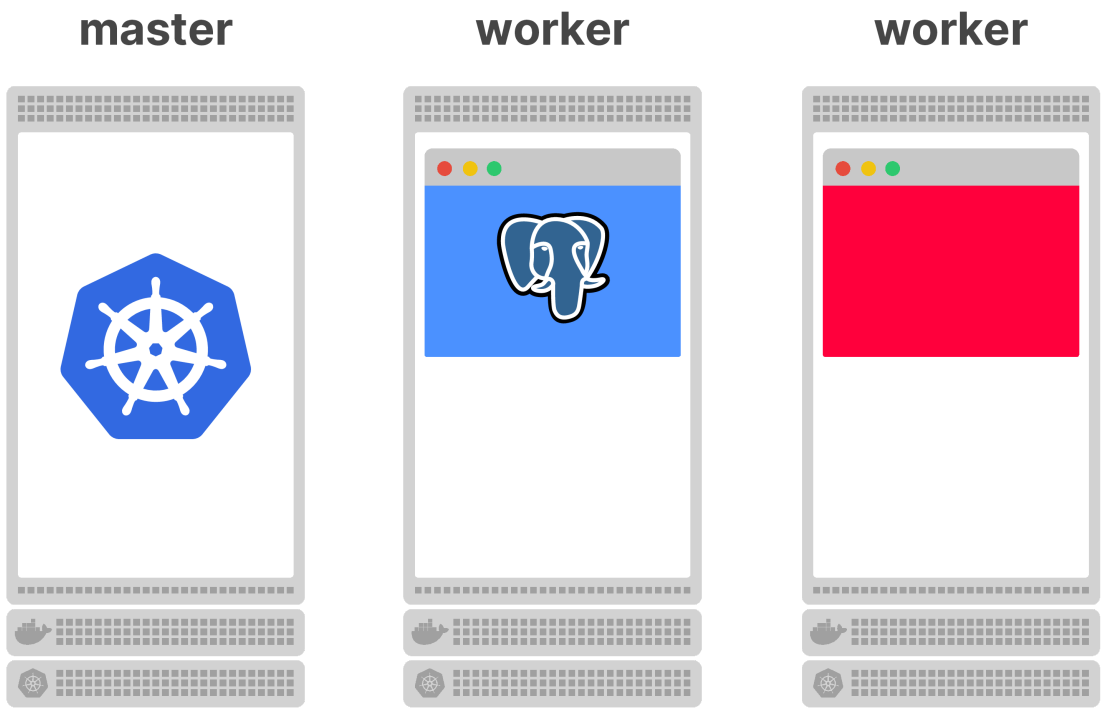
```
~$ cat pod.yaml
apiVersion: v1
kind: Pod
spec:
  containers:
    - name: test-container
      image: k8s.gcr.io/busybox
      env:
        - name: DATABASE_USERNAME
          valueFrom:
            secretKeyRef:
              name: my-secret
              key: username

~$ _
```

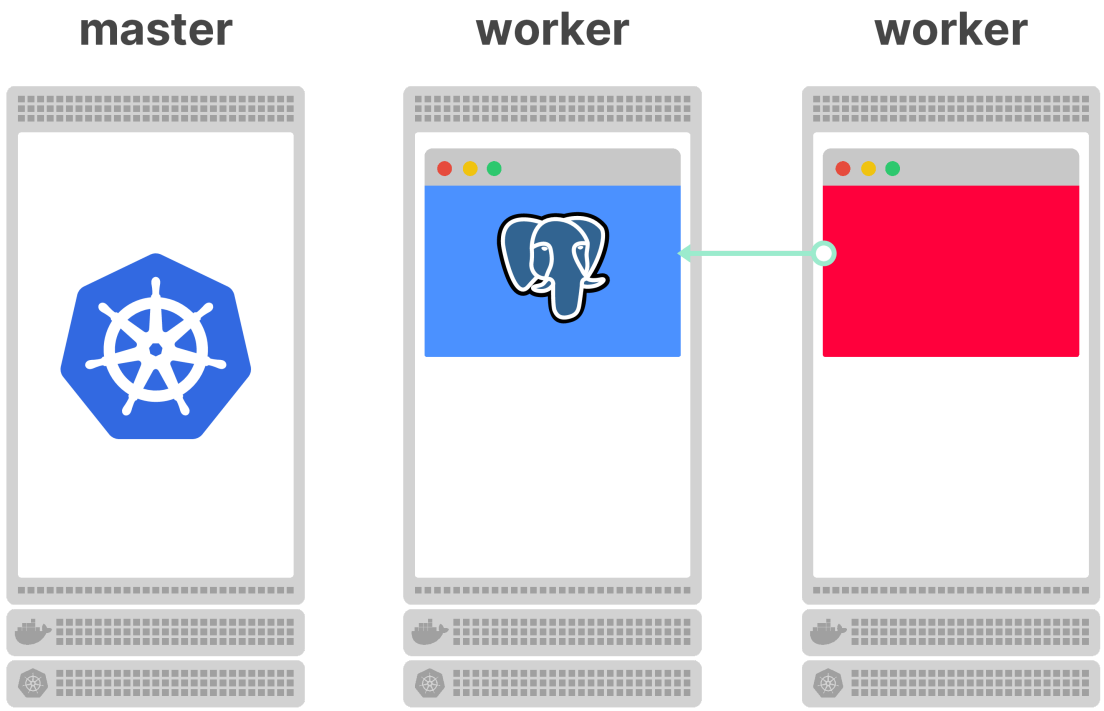
Mounting volumes

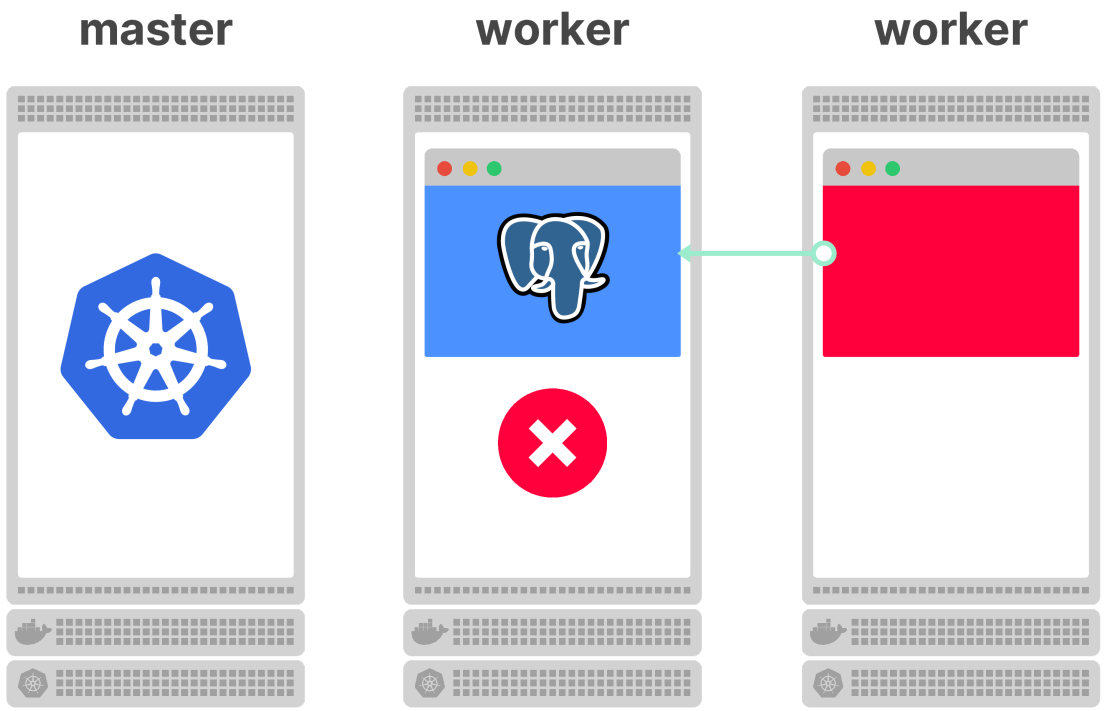
02

Managing state in Pods


master
worker
worker


master
worker
worker


master
worker
worker


master
worker
worker

Kubernetes Volumes

Same lifecycle as Pod

Unmounted when Pod dies

Survives containers restart

Kubernetes Volumes

Same lifecycle as Pod

Unmounted when Pod dies

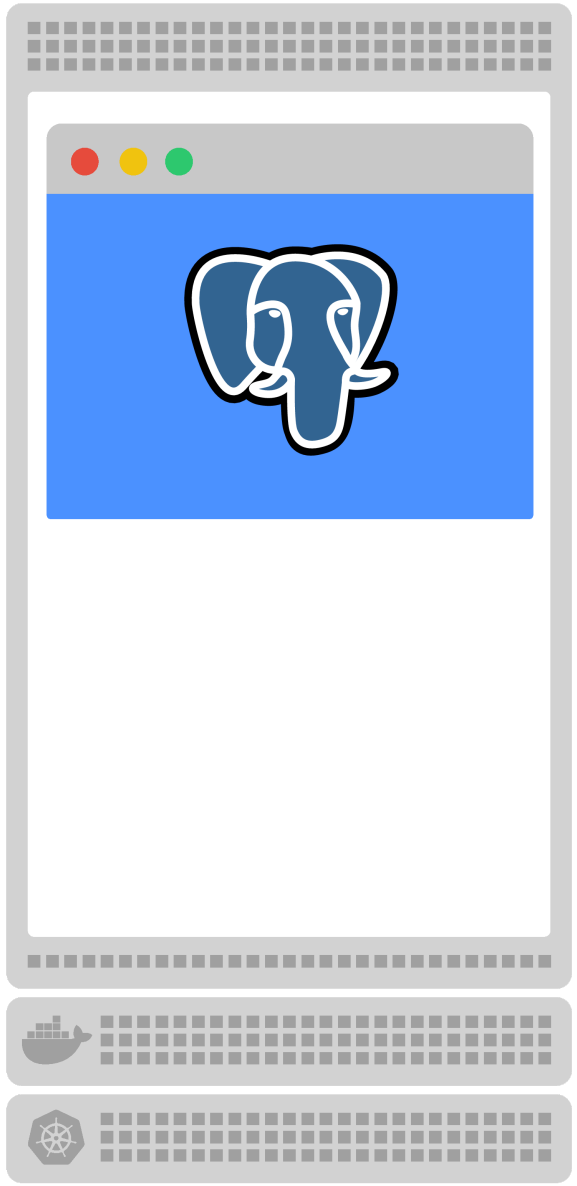
Survives containers restart

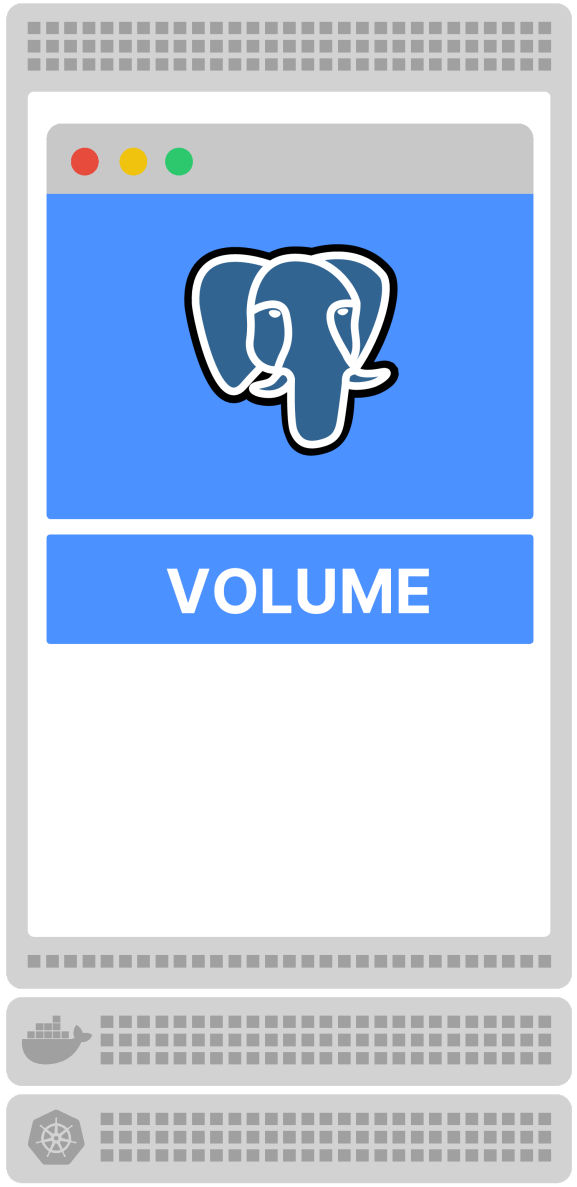
Kubernetes Volumes

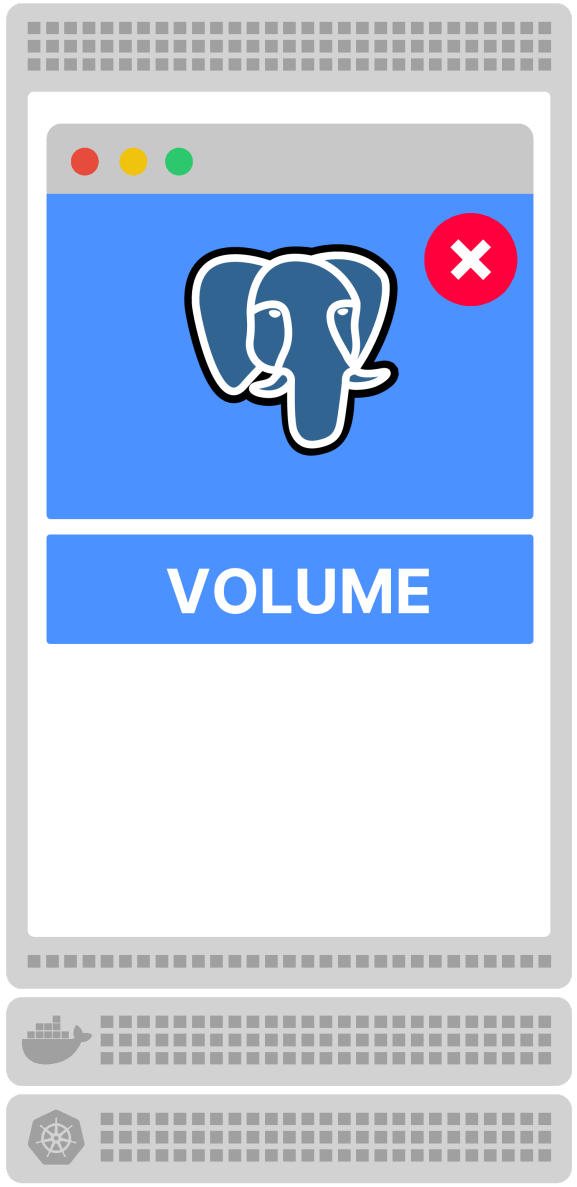
Same lifecycle as Pod

Unmounted when Pod dies

Survives containers restart



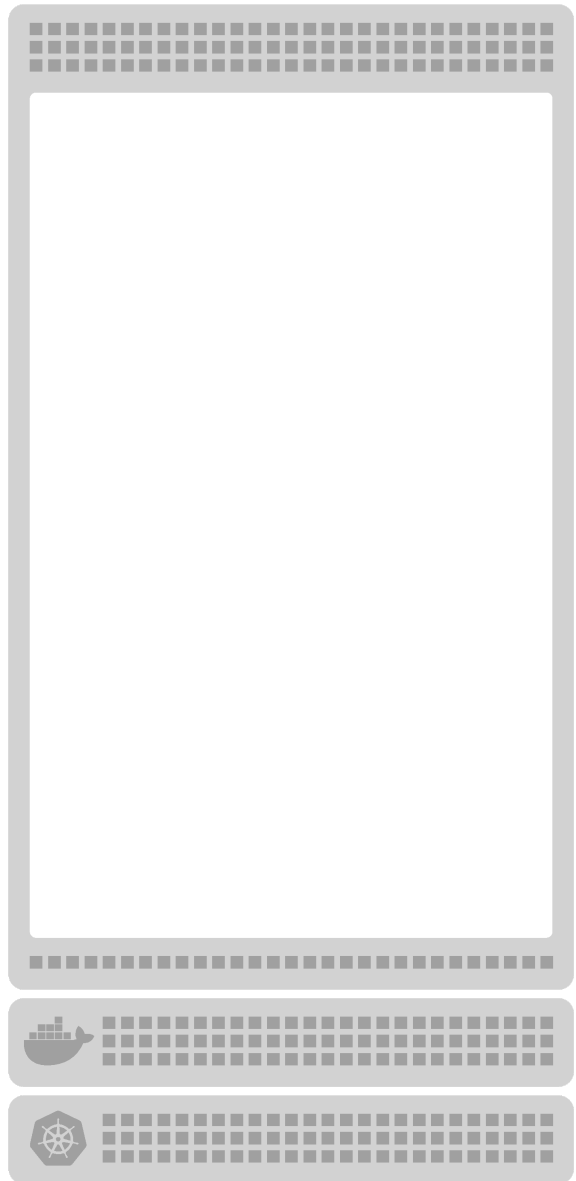

VOLUME


VOLUME

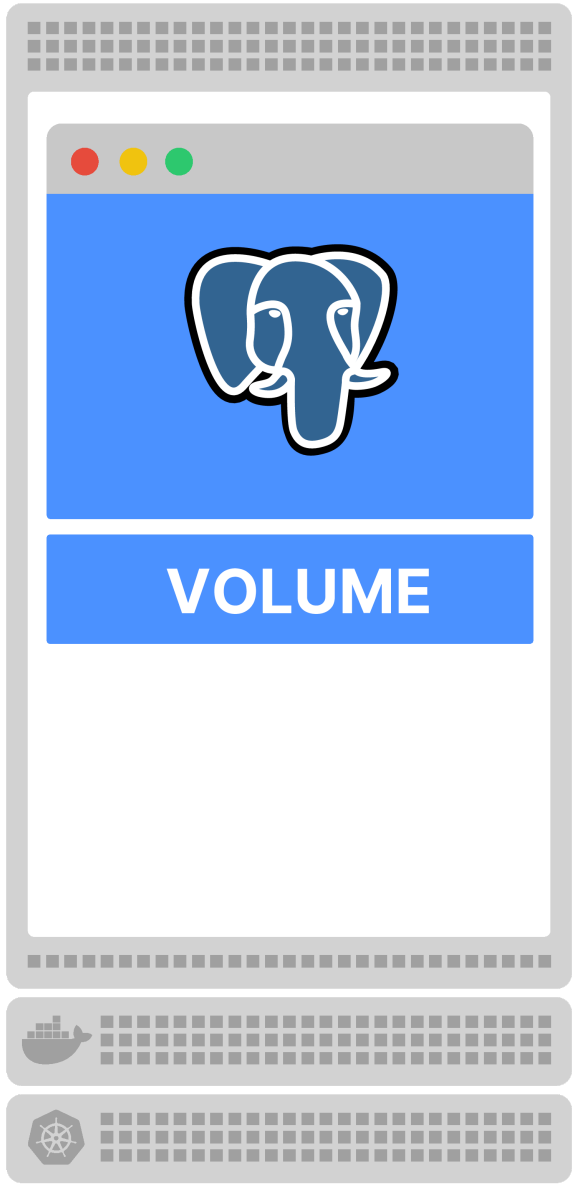
```
~$ cat pod.yaml
apiVersion: v1
kind: Pod
spec:
  containers:
  - image: k8s.gcr.io/test-webserver
    name: test-container
    volumeMounts:
    - mountPath: /data
      name: ebs-volume
  volumes:
  - name: ebs-volume
    awsElasticBlockStore:
      volumeID: vol-867g5kii
      fsType: ext4
~$ _
```

```
~$ cat pod.yaml
apiVersion: v1
kind: Pod
spec:
  containers:
  - image: k8s.gcr.io/test-webserver
    name: test-container
    volumeMounts:
    - mountPath: /data
      name: ebs-volume
  volumes:
  - name: ebs-volume
    awsElasticBlockStore:
      volumeID: vol-867g5kii
      fsType: ext4
~$ _
```

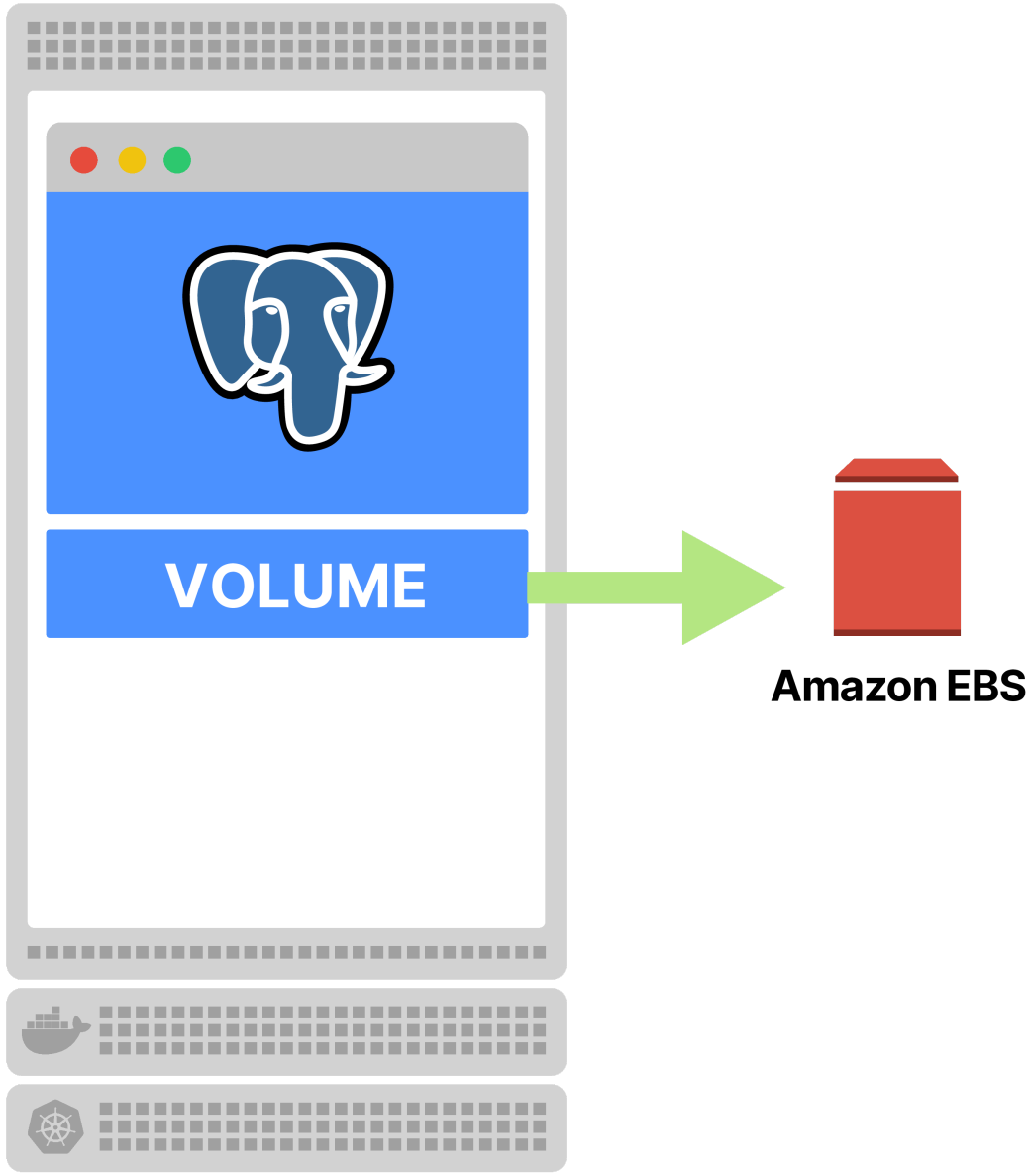
```
~$ cat pod.yaml
apiVersion: v1
kind: Pod
spec:
  containers:
  - image: k8s.gcr.io/test-webserver
    name: test-container
    volumeMounts:
    - mountPath: /data
      name: ebs-volume
  volumes:
  - name: ebs-volume
    awsElasticBlockStore:
      volumeID: vol-867g5kii
      fsType: ext4
~$ _
```

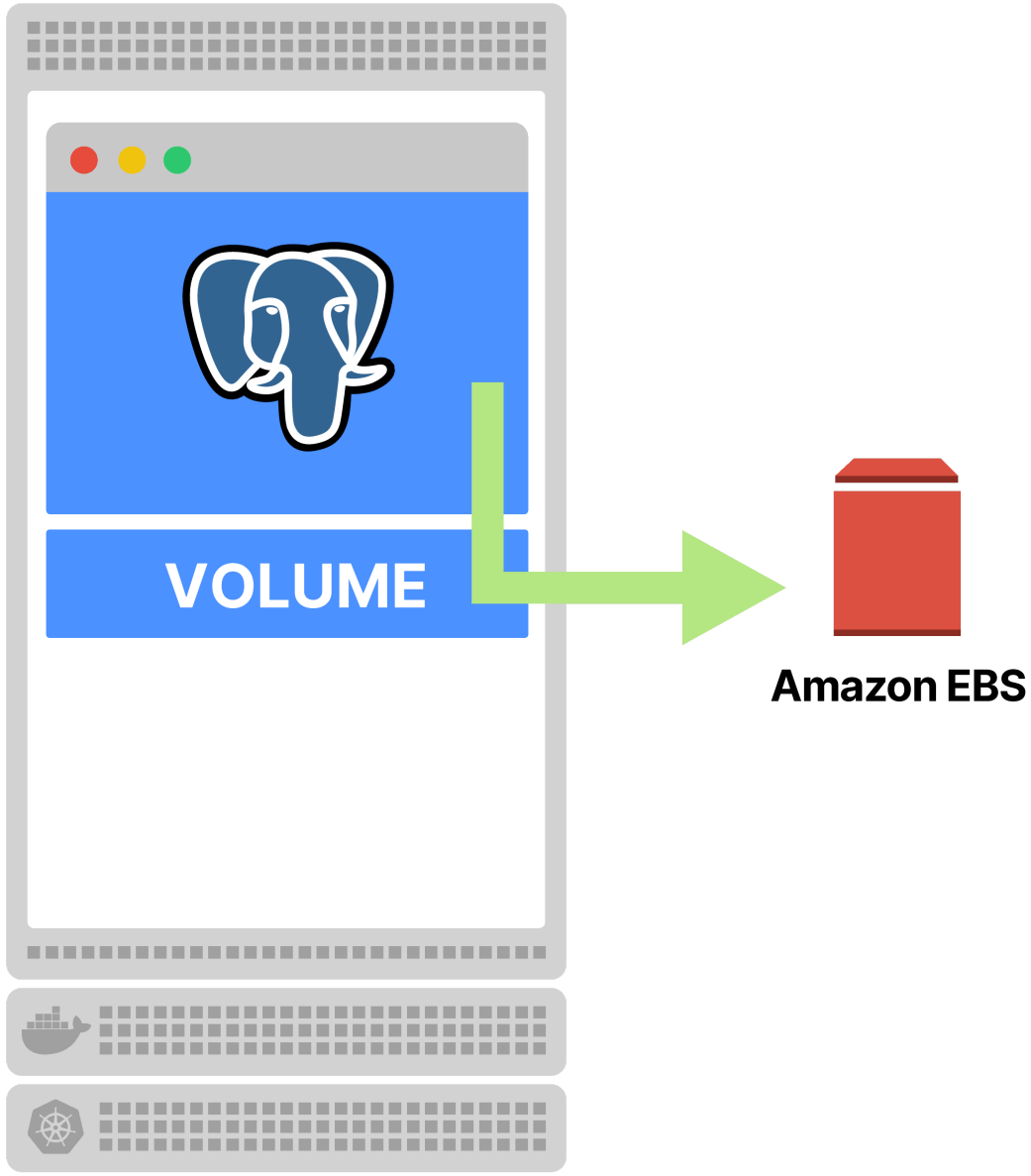


Amazon EBS



Amazon EBS


VOLUME
Amazon EBS


VOLUME
Amazon EBS

awsElasticBlockStore

awsElasticBlockStore

azureDisk

Kubernetes Volumes

awsElasticBlockStore

azureDisk

azureFile

cephfs

Kubernetes Volumes

awsElasticBlockStore
azureDisk
azureFile
cephfs
configMap
csi
downwardAPI

Kubernetes Volumes

awsElasticBlockStore emptyDir
azureDisk fc
azureFile flexVolume
cephfs flocker
configMap gcePersistentDisk
csi glusterfs
downwardAPI

Kubernetes Volumes

awsElasticBlockStore emptyDir
azureDisk fc iscsi
azureFile flexVolume local
cephfs flocker projected
configMap gcePersistentDisk secret
csi glusterfs persistentVolumeClaim
downwardAPI hostPath

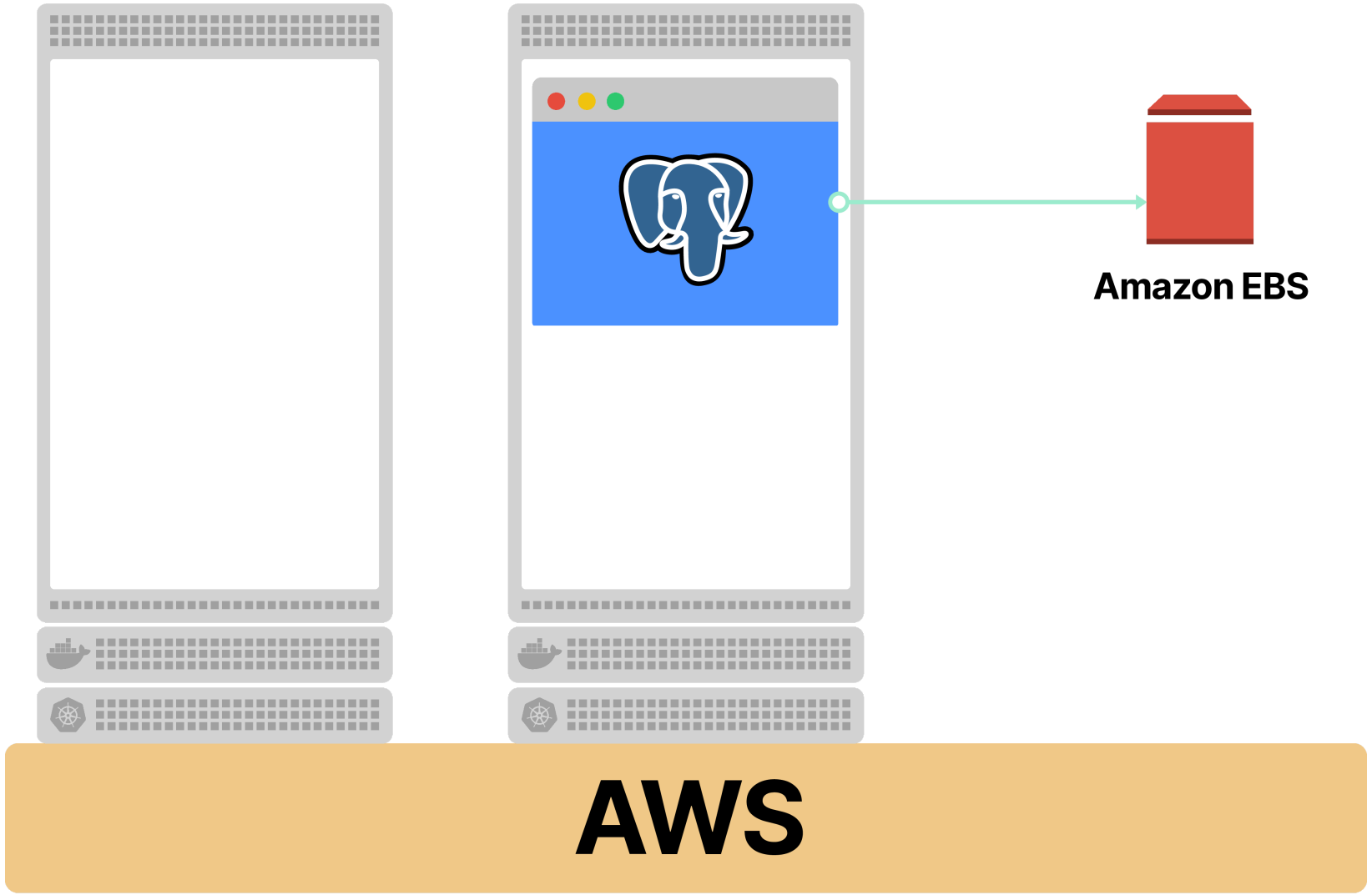
Kubernetes Volumes

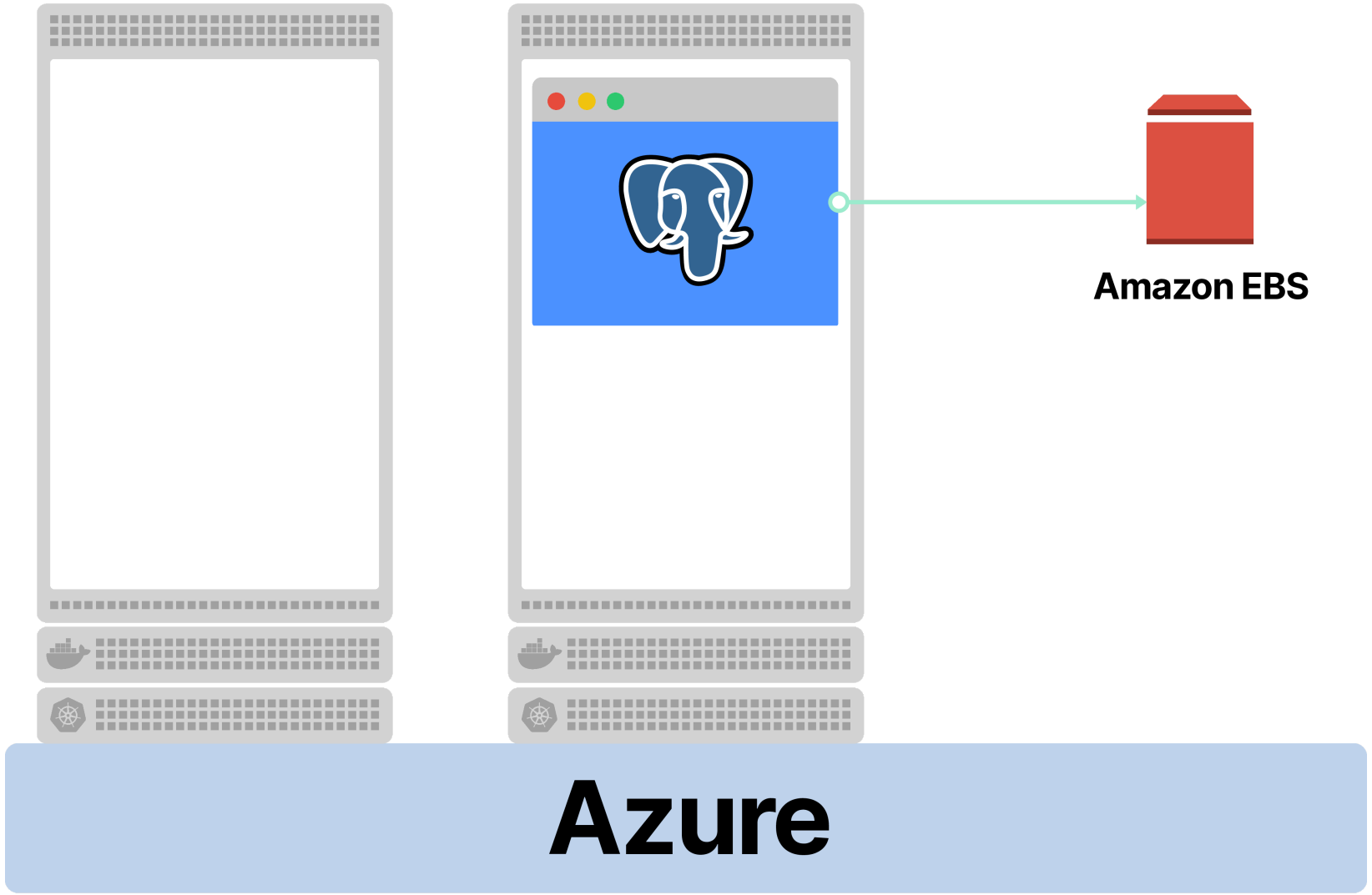
awsElasticBlockStore emptyDir
azureDisk fc iscsi portworxVolume
azureFile flexVolume local nfs rbd
cephfs flocker projected scaleIO
configMap gcePersistentDisk secret
csi glusterfs persistentVolumeClaim
downwardAPI hostPath quobyte

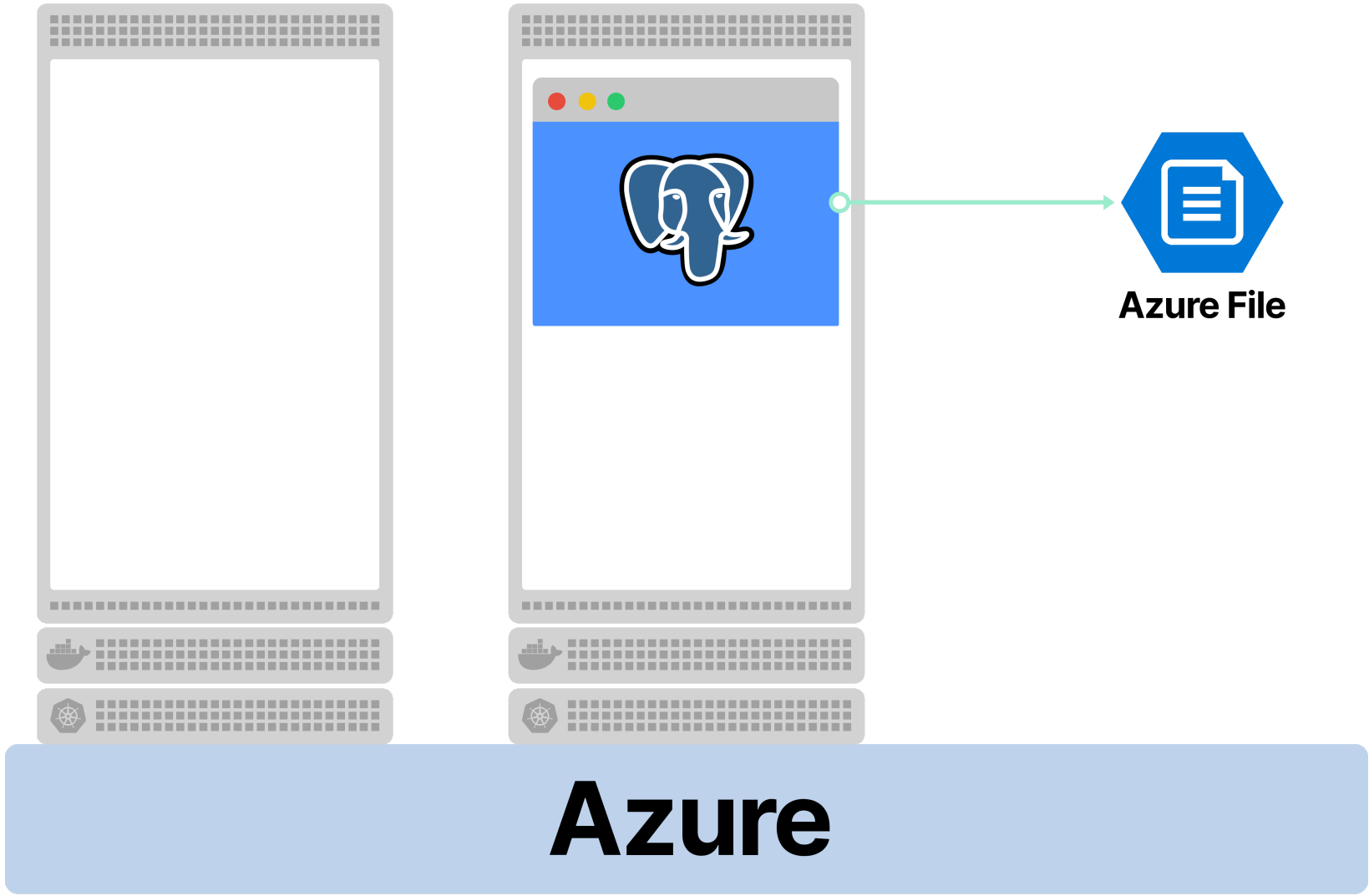
Kubernetes Volumes

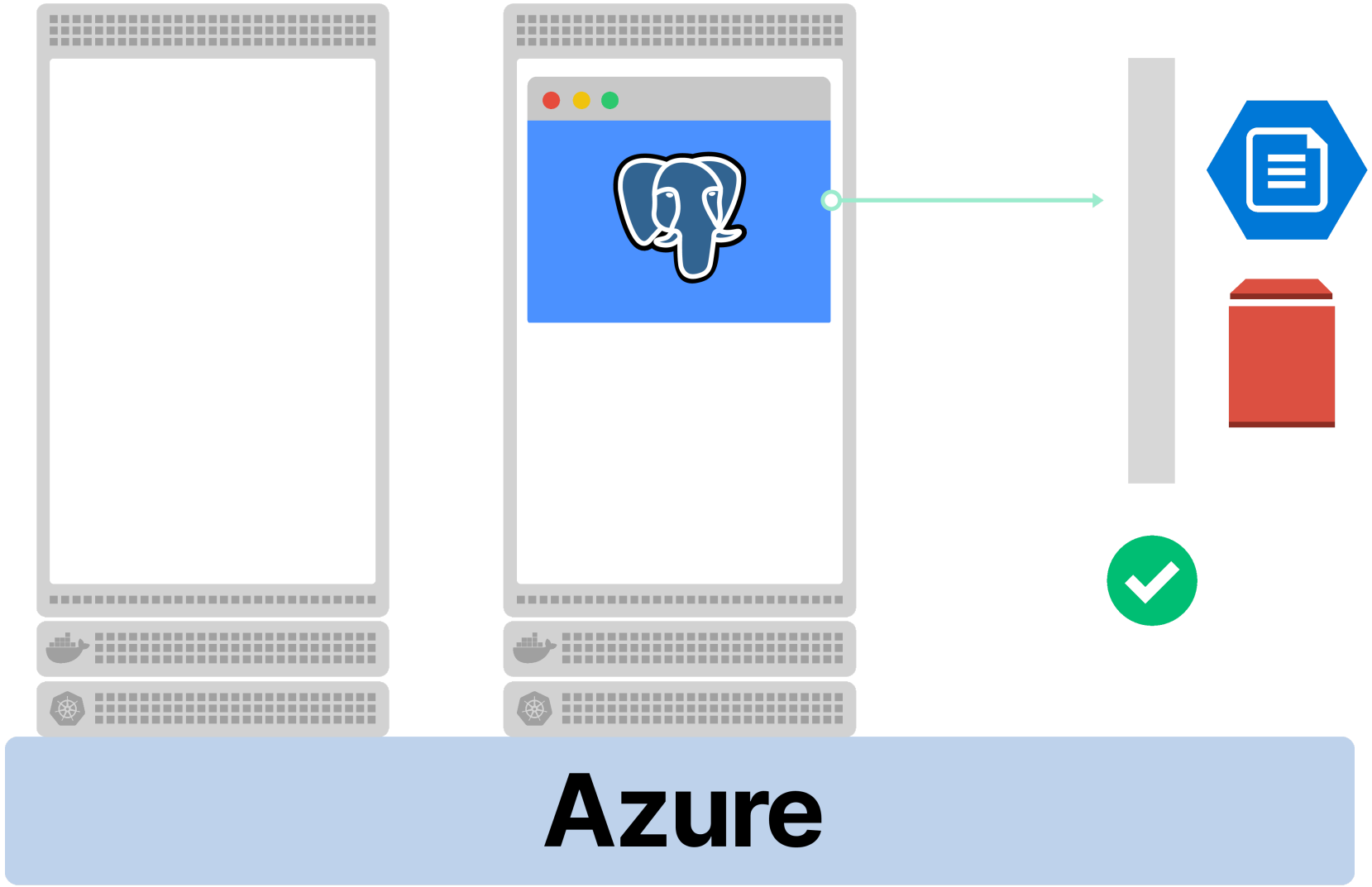
awsElasticBlockStore **emptyDir**
azureDisk fc iscsi portworxVolume
azureFile **flexVolume** **local** nfs rbd
cephfs flocker projected scaleIO
configMap gcePersistentDisk **secret**
csi glusterfs **persistentVolumeClaim**
downwardAPI **hostPath** quobyte

Volumes portability


Amazon EBS
AWS


Amazon EBS
Azure


Azure File
Azure


Azure

Persistent Volumes

PERSISTENT VOLUME

PERSISTENT VOLUME
VOLUME

Persistent Volumes

Independent lifecycle

Uses volumes

Created by admins

Global object

Persistent Volumes

Independent lifecycle

Uses volumes

Created by admins

Global object

Persistent Volumes

Independent lifecycle

Uses volumes

Created by admins

Global object

Persistent Volumes

Independent lifecycle

Uses volumes

Created by admins

Global object

```
~$ cat pv.yaml
kind: PersistentVolume
apiVersion: v1
metadata:
  name: task-pv
spec:
  capacity:
    storage: 100Gi
  accessModes:
    - ReadWriteOnce
  awsElasticBlockStore:
    volumeID: vol-867g5kii
    fsType: ext4
  persistentVolumeReclaimPolicy: retain

~$ _
```

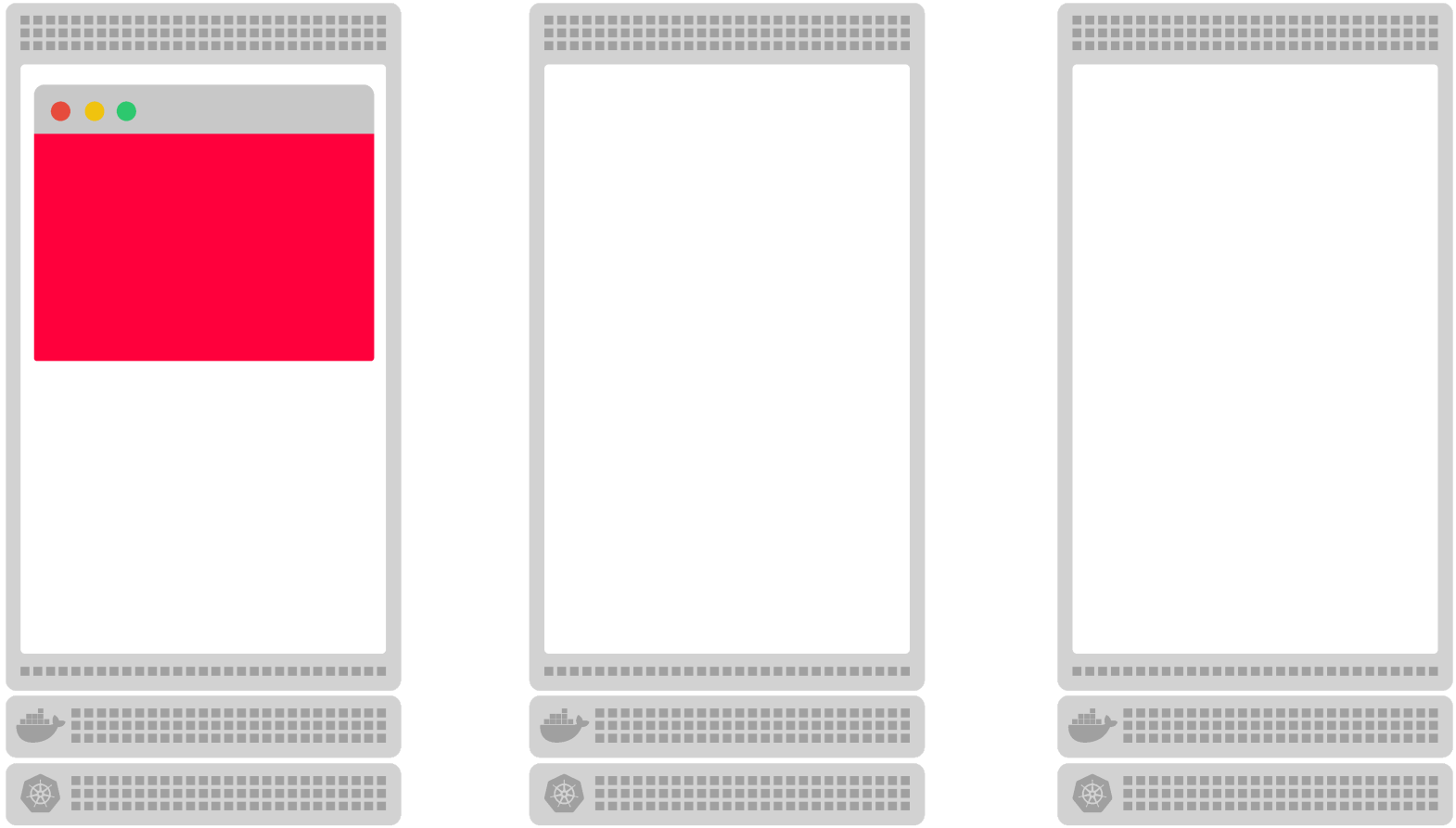
```
~$ cat pv.yaml
kind: PersistentVolume
apiVersion: v1
metadata:
  name: task-pv
spec:
  capacity:
    storage: 100Gi
  accessModes:
    - ReadWriteOnce
  awsElasticBlockStore:
    volumeID: vol-867g5kii
    fsType: ext4
  persistentVolumeReclaimPolicy: retain

~$ _
```

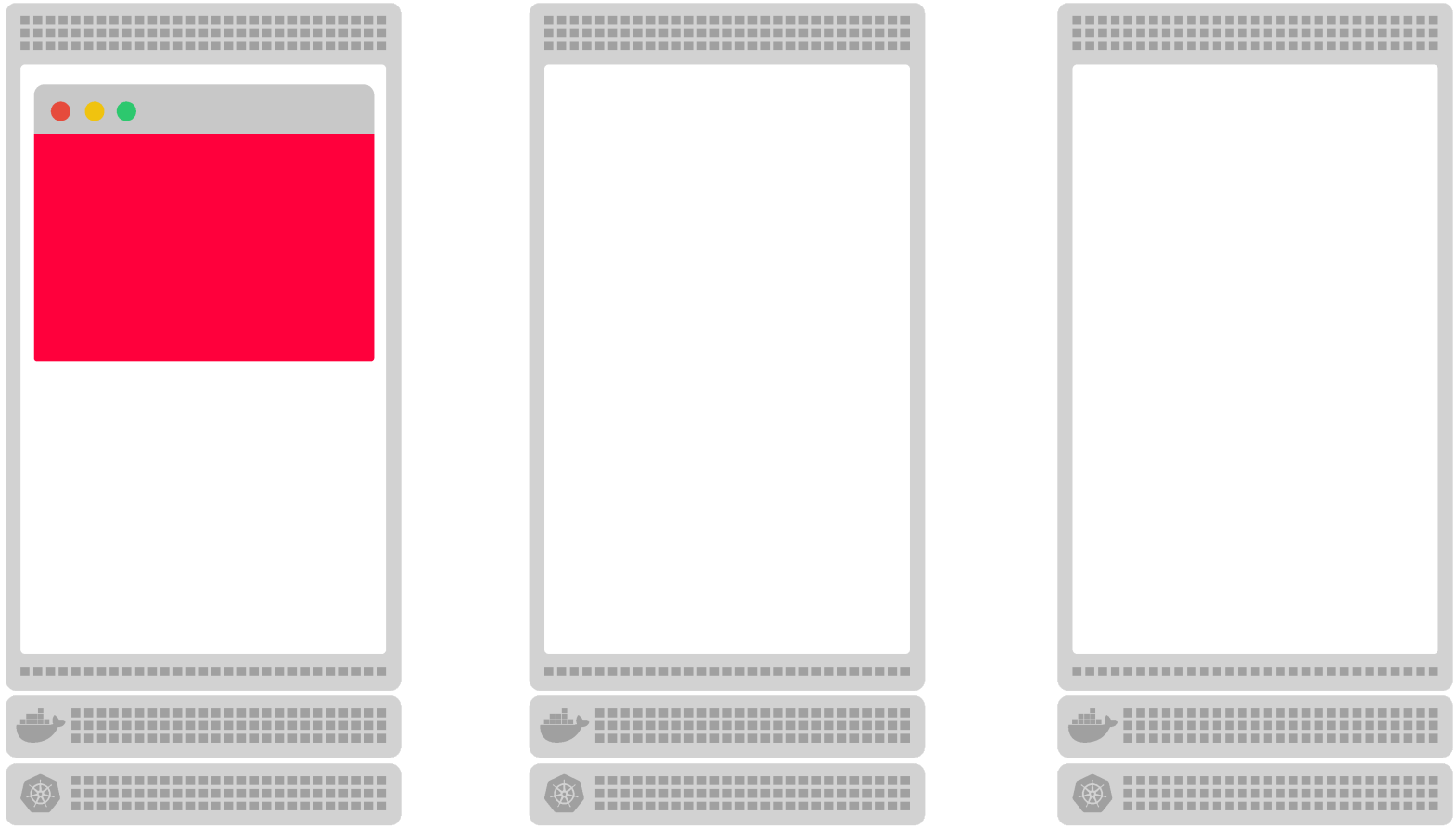
```
~$ cat pv.yaml
kind: PersistentVolume
apiVersion: v1
metadata:
  name: task-pv
spec:
  capacity:
    storage: 100Gi
  accessModes:
    - ReadWriteOnce
  awsElasticBlockStore:
    volumeID: vol-867g5kii
    fsType: ext4
  persistentVolumeReclaimPolicy: retain

~$ _
```

```
~$ cat pv.yaml
kind: PersistentVolume
apiVersion: v1
metadata:
  name: task-pv
spec:
  capacity:
    storage: 100Gi
  accessModes:
    - ReadWriteOnce
  awsElasticBlockStore:
    volumeID: vol-867g5kii
    fsType: ext4
  persistentVolumeReclaimPolicy: retain

~$ _
```

```
~$ cat pv.yaml
kind: PersistentVolume
apiVersion: v1
metadata:
  name: task-pv
spec:
  capacity:
    storage: 100Gi
  accessModes:
    - ReadWriteOnce
  awsElasticBlockStore:
    volumeID: vol-867g5kii
    fsType: ext4
  persistentVolumeReclaimPolicy: retain

~$ _
```

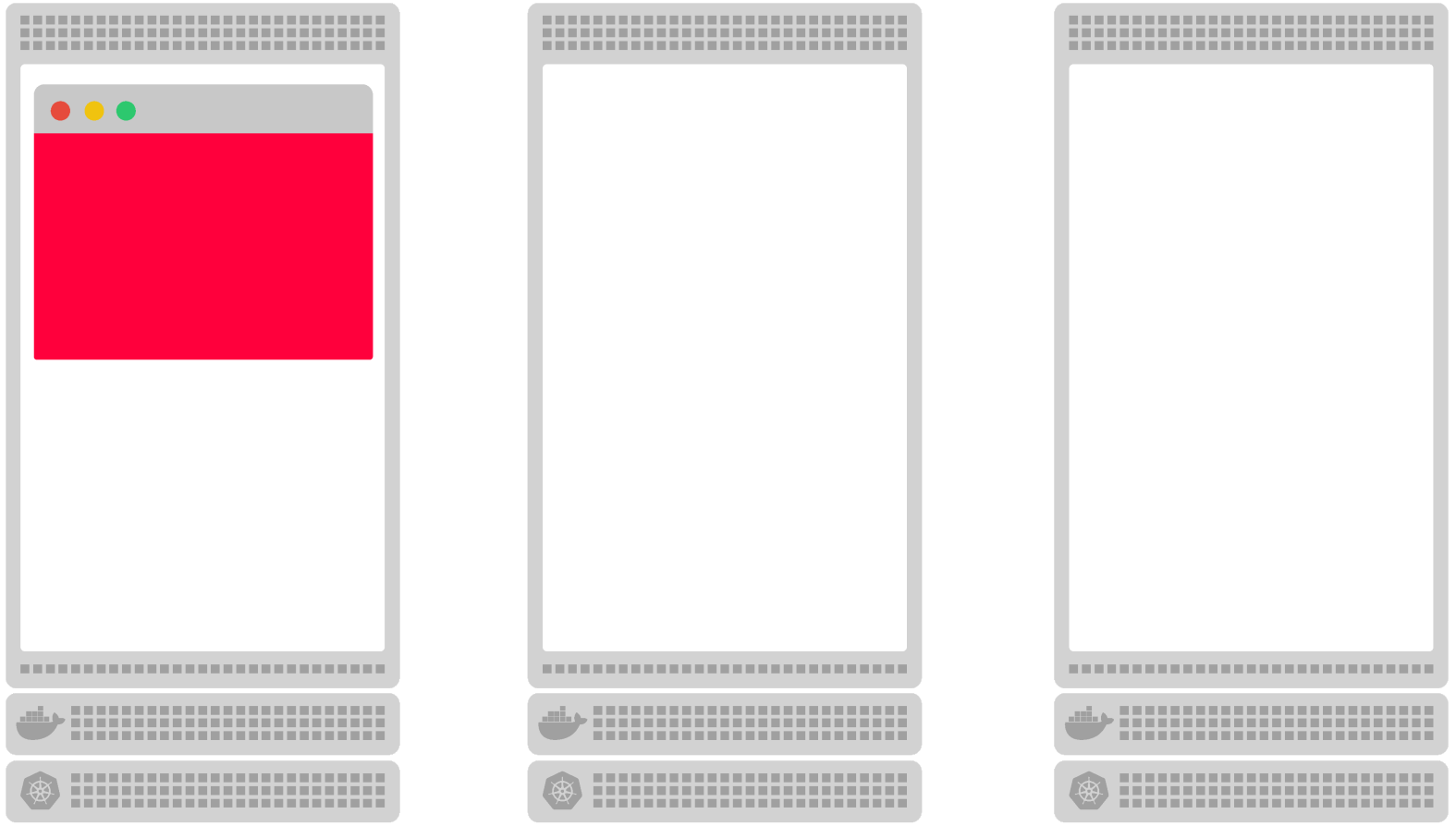
Persistent Volumes in practice





Amazon EBS

Azure File



Persistent Volume

Persistent Volume

Using Persistent Volumes

PERSISTENT VOL
VOLUME

PERSISTENT VOLUME CLAIM
PERSISTENT VOLUME
VOLUME

Persistent volume claims

Request for storage (PV)

Scheduler

1-to-1 relationship

Persistent volume claims

Request for storage (PV)

Scheduler

1-to-1 relationship

Persistent volume claims

Request for storage (PV)

Scheduler

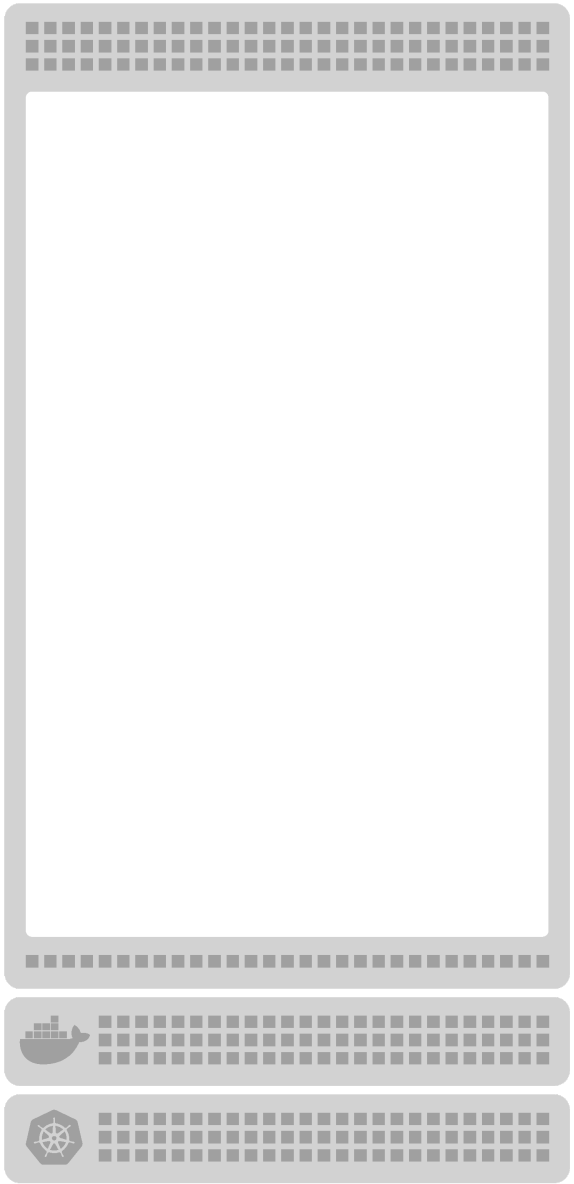
1-to-1 relationship

```
~$ cat pvc.yaml
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: task-pv
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 20Gi

~$ _
```

```
~$ cat pvc.yaml
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: task-pv
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 20Gi

~$ _
```

```
~$ cat pvc.yaml
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: task-pv
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 20Gi

~$ _
```

Persistent Volumes Claims in practice



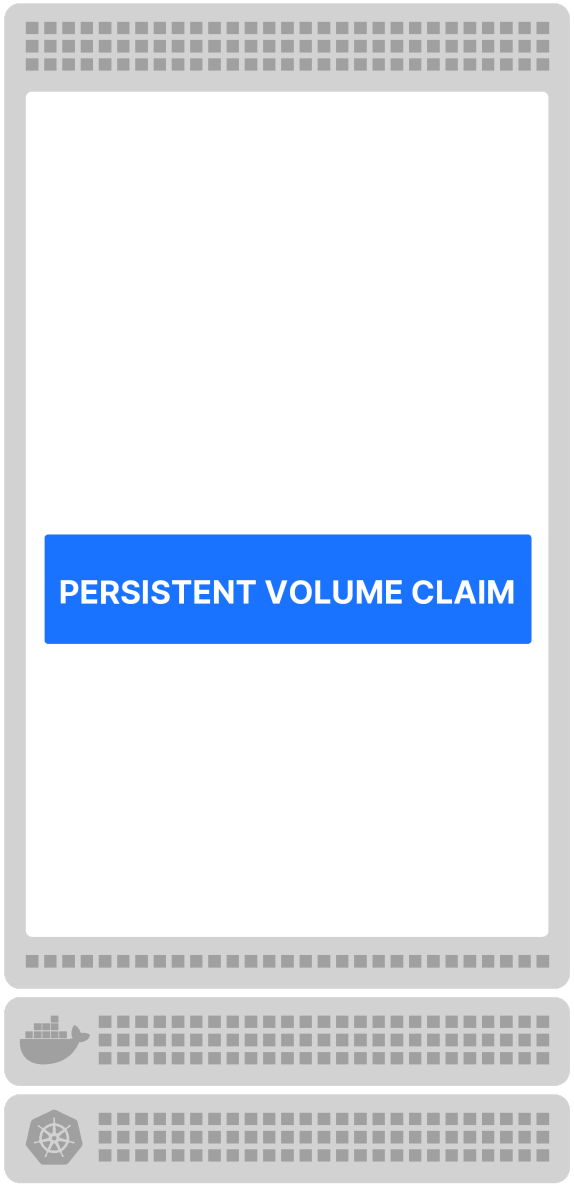
20 GiB
RWM

10 GiB
ROX

90 GiB
RWO

Amazon EBS

```
~$ cat pvc.yaml
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: task-pv
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 90Gi

~$ _
```

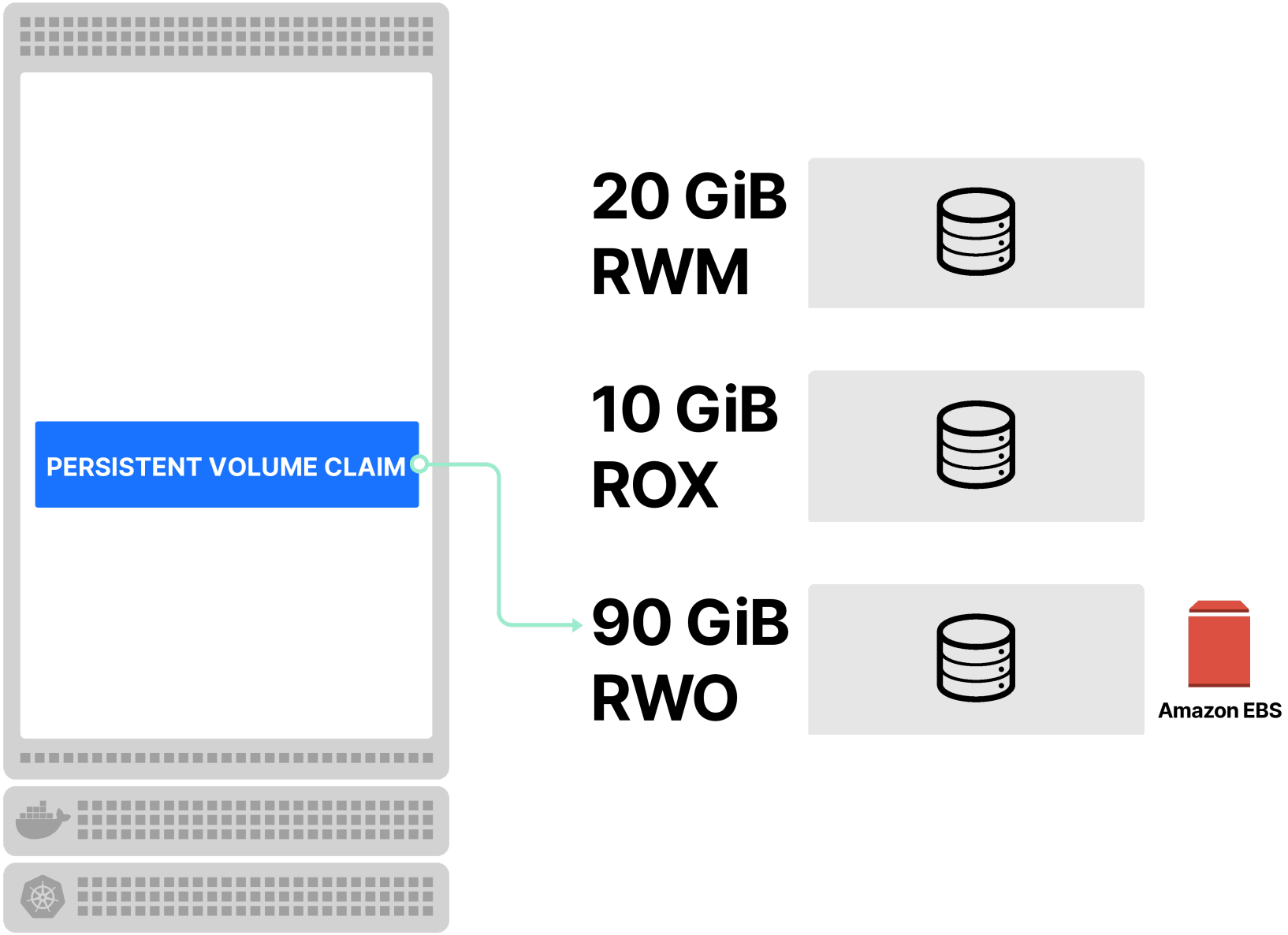


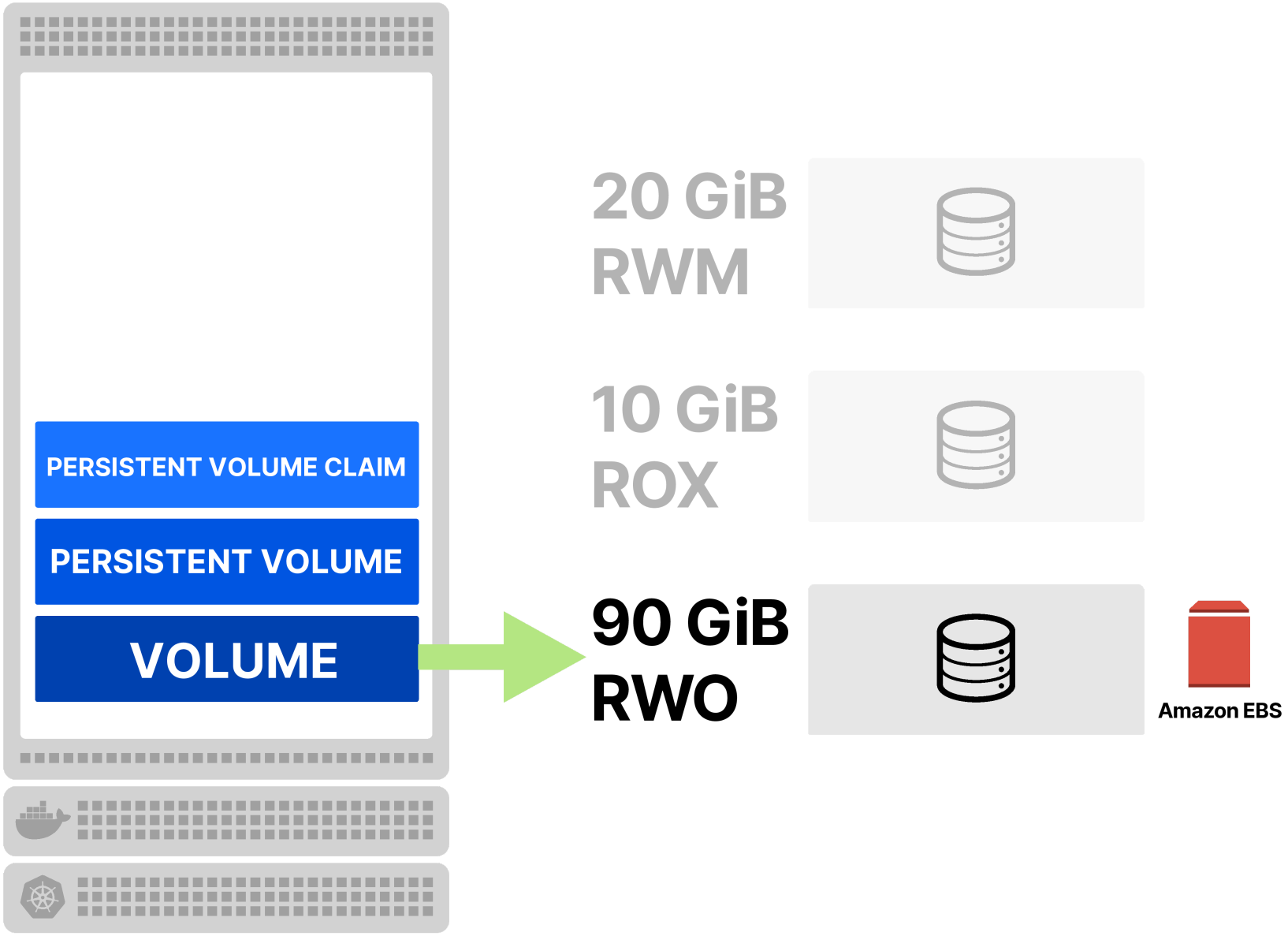
20 GiB
RWM

10 GiB
ROX

90 GiB
RWO

Amazon EBS

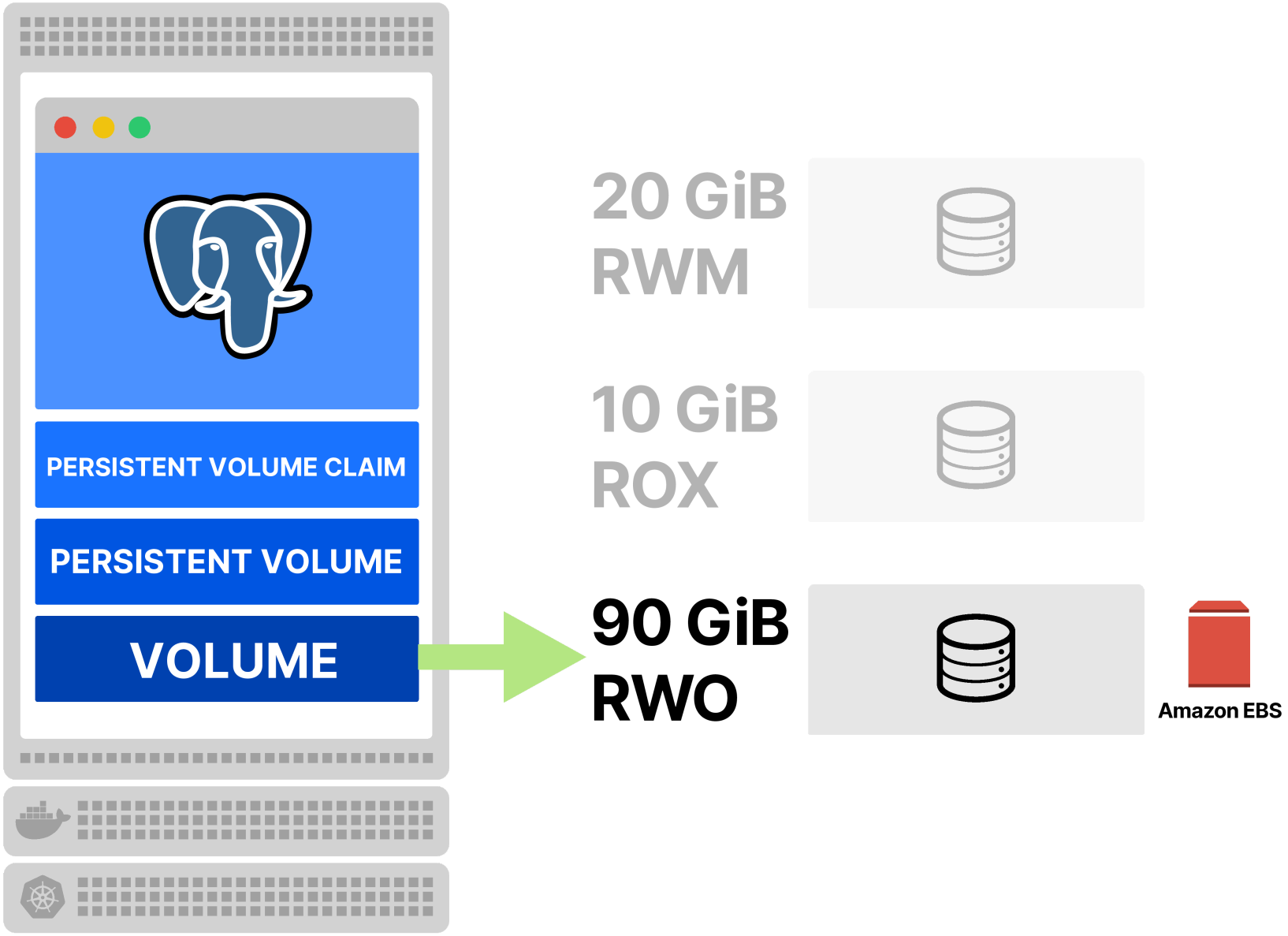

PERSISTENT VOLUME CLAIM
20 GiB
RWM
10 GiB
ROX
90 GiB
RWO
Amazon EBS

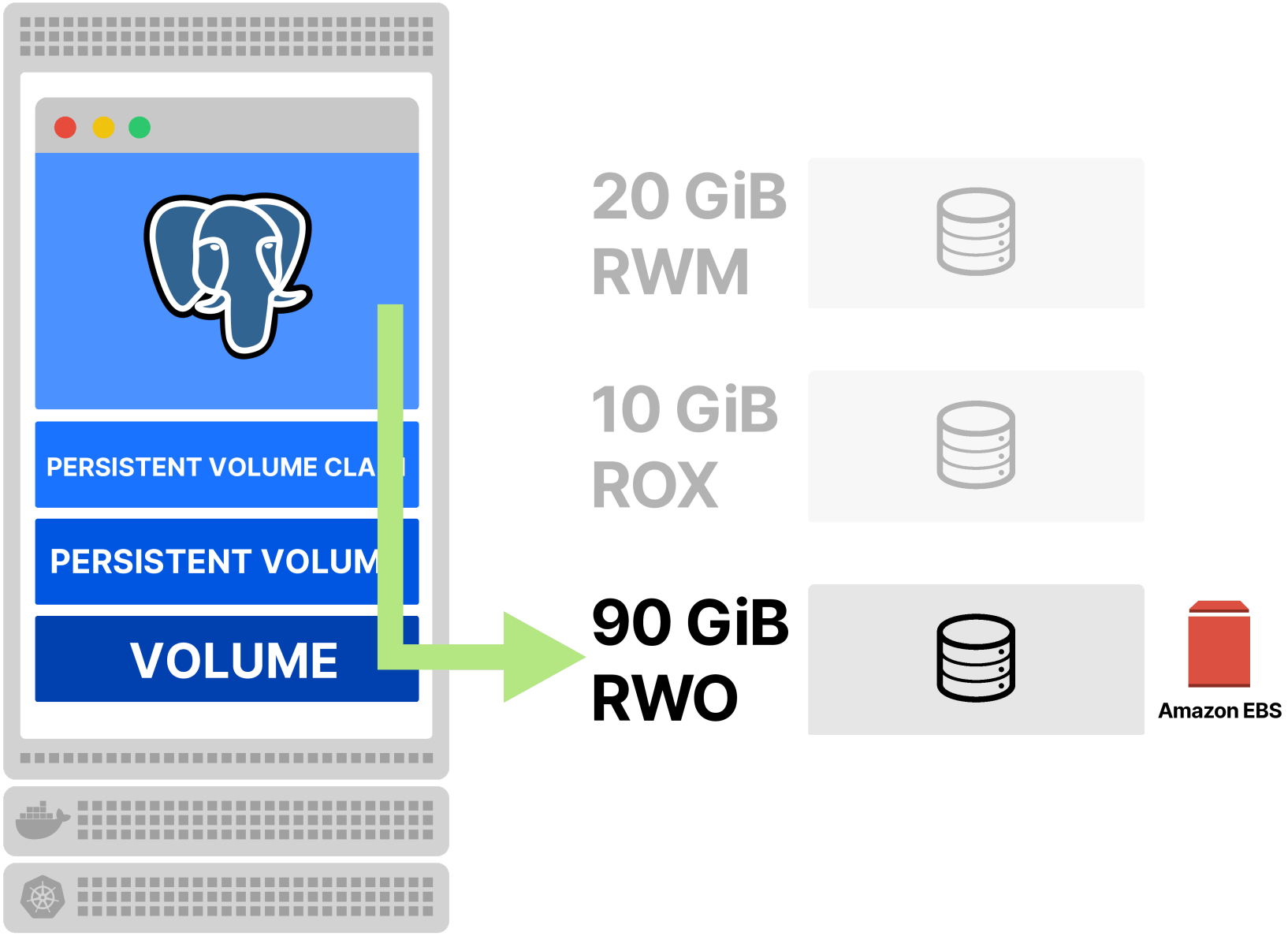

PERSISTENT VOLUME CLAIM
PERSISTENT VOLUME
VOLUME
20 GiB
RWM
10 GiB
ROX
90 GiB
RWO
Amazon EBS

```
~$ cat pod.yaml
kind: Pod
apiVersion: v1
spec:
  containers:
    - name: myfrontend
      image: nginx
      volumeMounts:
      - mountPath: "/var/www/html"
        name: mypd
  volumes:
    - name: mypd
      persistentVolumeClaim:
        claimName: myclaim
~$ _
```

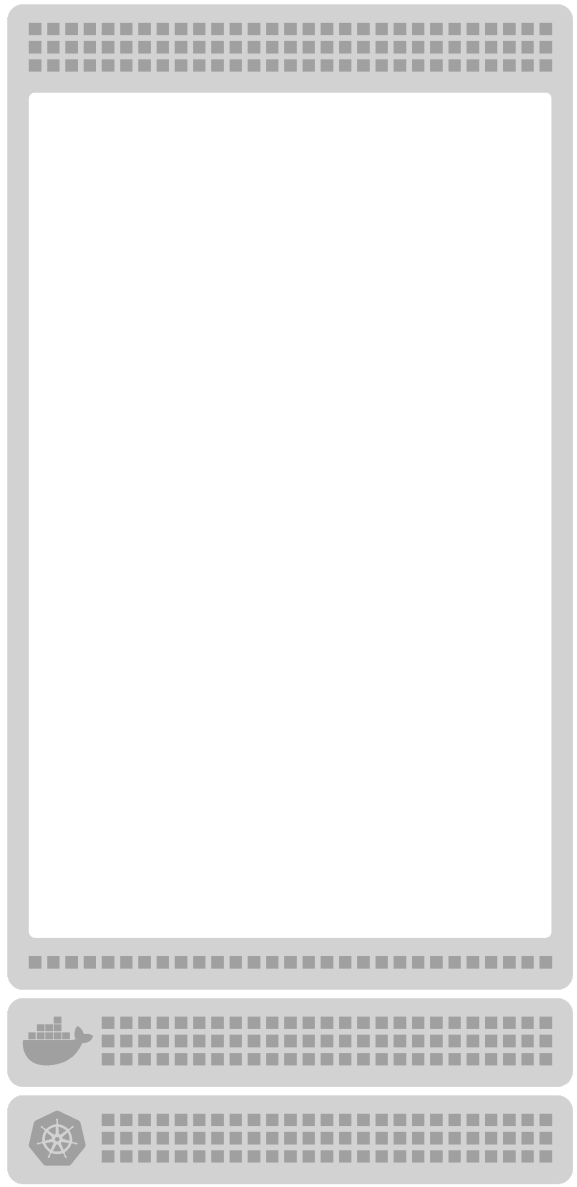
```
~$ cat pod.yaml
kind: Pod
apiVersion: v1
spec:
  containers:
    - name: myfrontend
      image: nginx
      volumeMounts:
      - mountPath: "/var/www/html"
        name: mypd
  volumes:
    - name: mypd
      persistentVolumeClaim:
        claimName: myclaim
~$ _
```

```
~$ cat pod.yaml
kind: Pod
apiVersion: v1
spec:
  containers:
    - name: myfrontend
      image: nginx
      volumeMounts:
      - mountPath: "/var/www/html"
        name: mypd
  volumes:
    - name: mypd
      persistentVolumeClaim:
        claimName: myclaim
~$ _
```


PERSISTENT VOLUME CLAIM
PERSISTENT VOLUME
VOLUME
20 GiB
RWM
10 GiB
ROX
90 GiB
RWO
Amazon EBS


PERSISTENT VOLUME CLA
PERSISTENT VOLUM
VOLUME
20 GiB
RWM
10 GiB
ROX
90 GiB
RWO
Amazon EBS

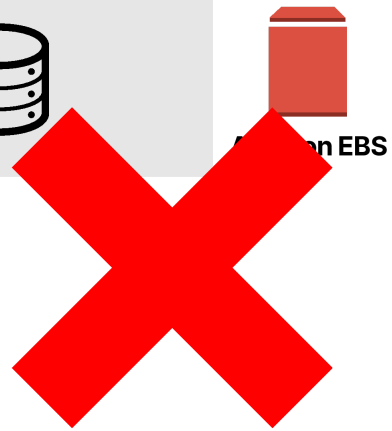
Persistent Volumes Claims in practice (2)

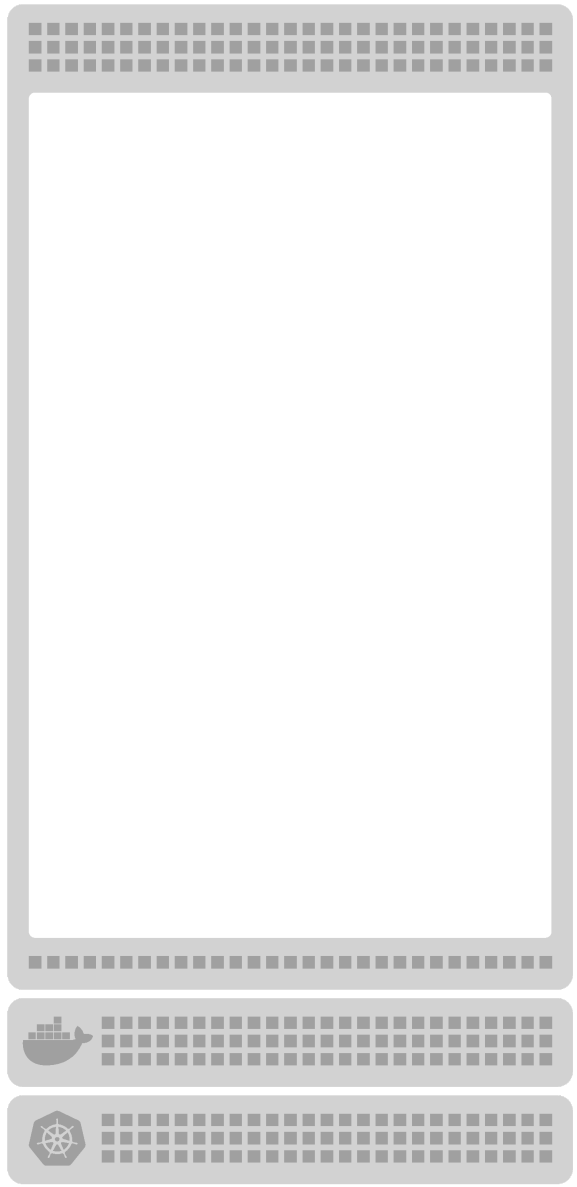


20 GiB
RWM

10 GiB
ROX

90 GiB
RWO



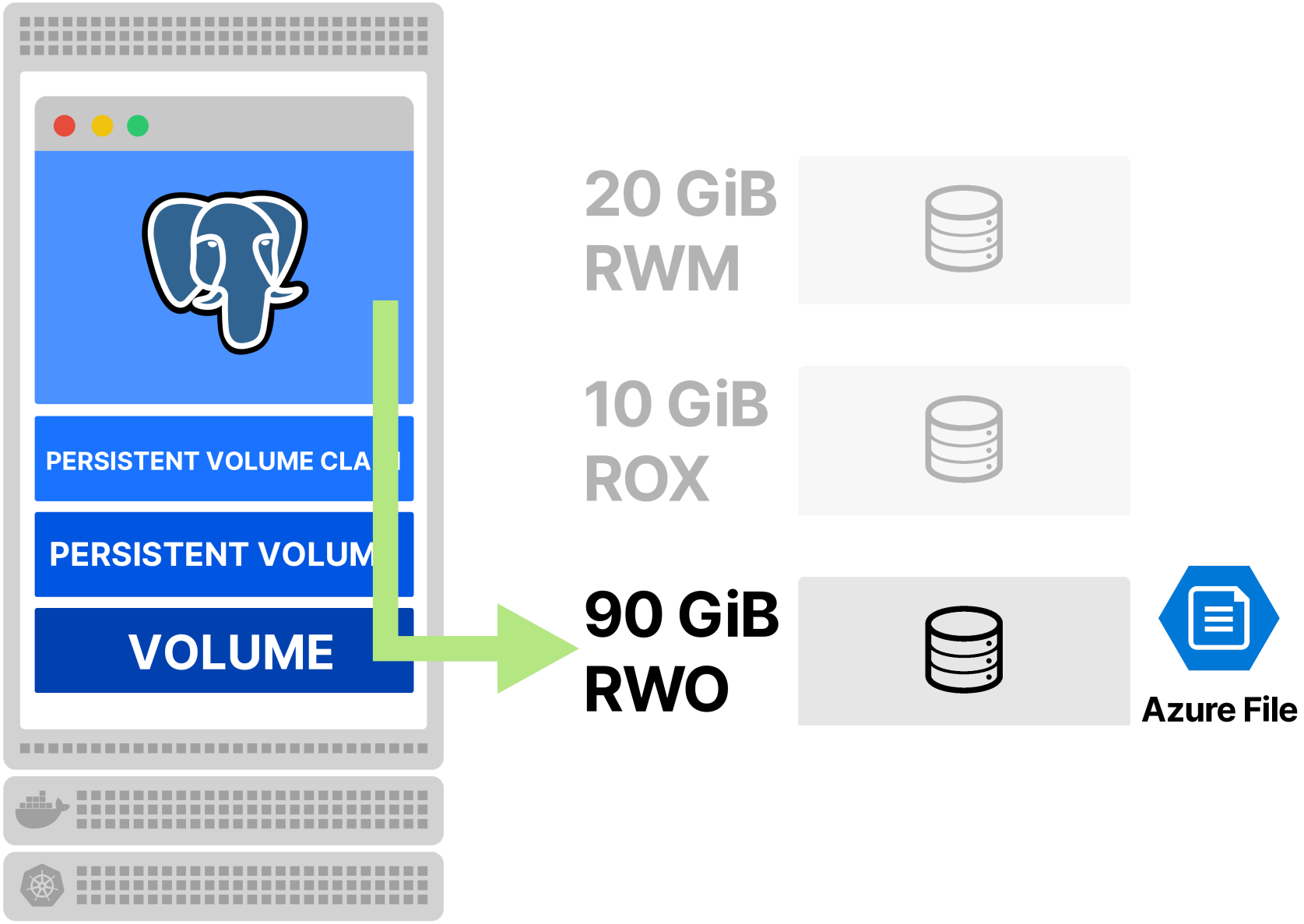


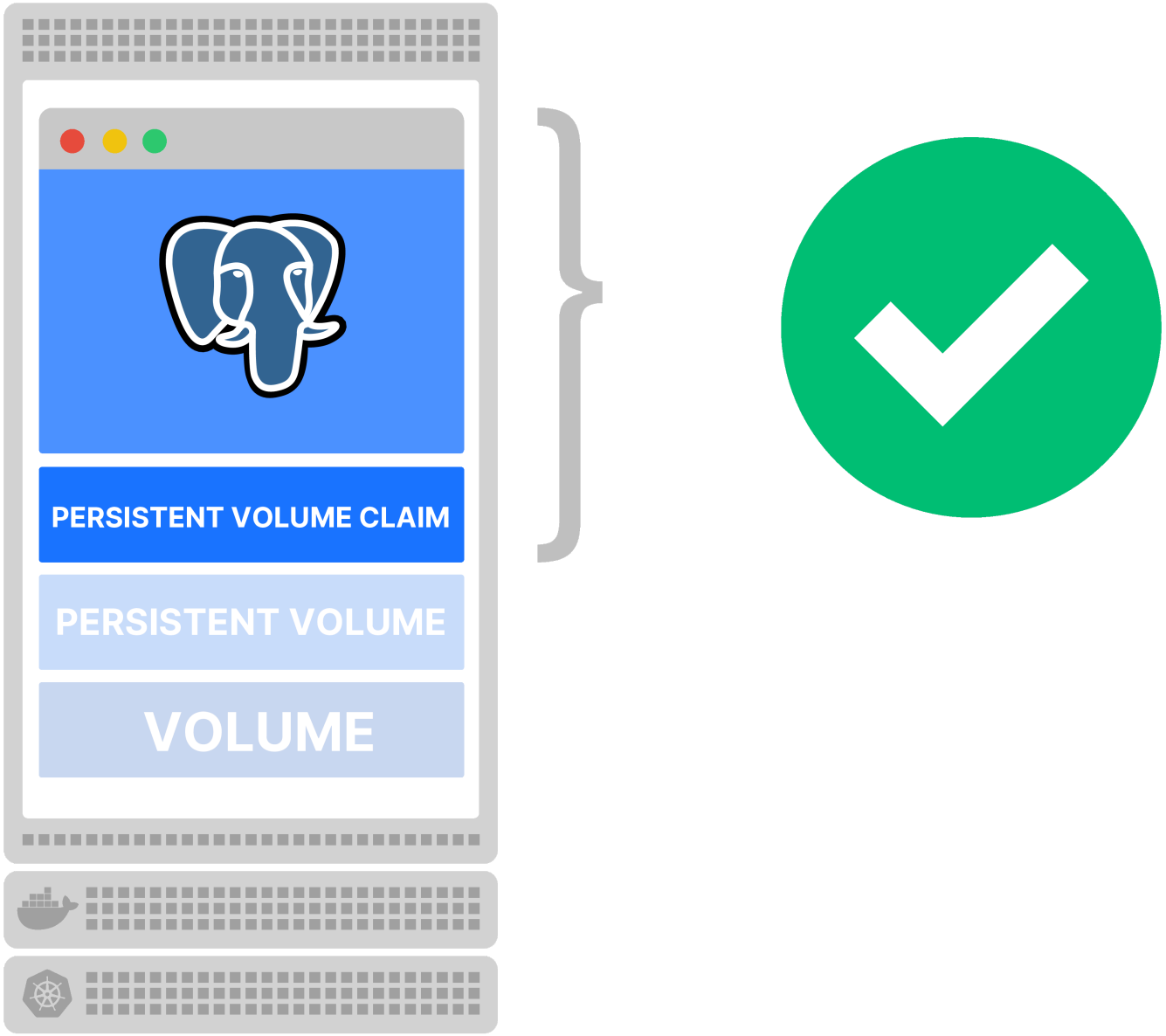
20 GiB
RWM

10 GiB
ROX

90 GiB
RWO

Azure File


PERSISTENT VOLUME CLA
PERSISTENT VOLUM
VOLUME
20 GiB
RWM
10 GiB
ROX
90 GiB
RWO
Azure File


PERSISTENT VOLUME CLAIM
PERSISTENT VOLUME
VOLUME

PERSISTENT VOLUME CLAIM
PERSISTENT VOLUME
VOLUME
amazon
web services
Microsoft Azure

Requests for volumes

```
~$ cat pvc.yaml
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: task-pv
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 90Gi

~$ _
```

Persistent Volumes access modes

Requests for volumes

Requests for volumes

read-write by a single node

read-only by many nodes

read-write by many nodes

Requests for volumes

read-write by a single node

read-only by many nodes

read-write by many nodes

Requests for volumes

read-write by a single node

read-only by many nodes

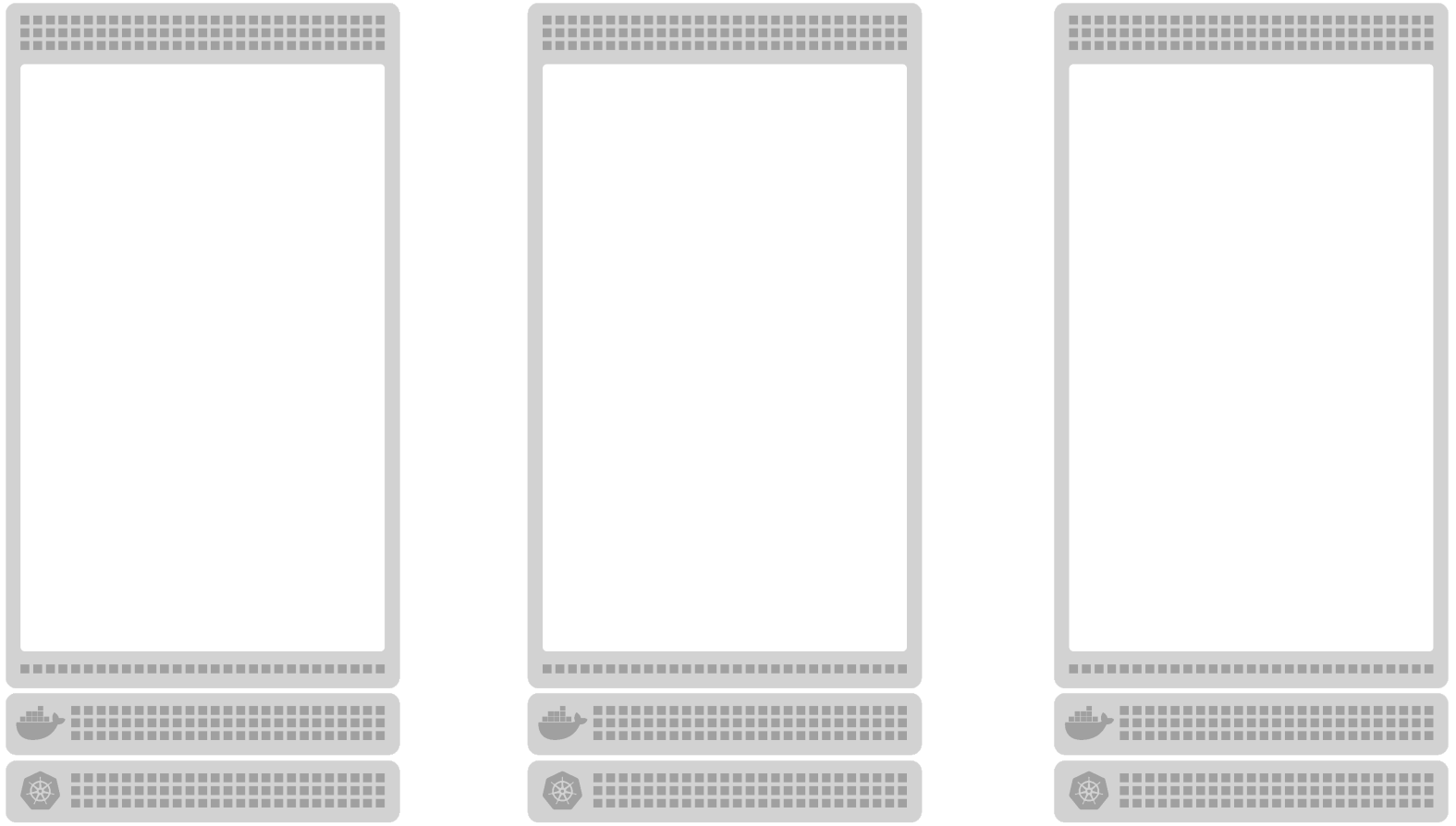
read-write by many nodes

Requests for volumes

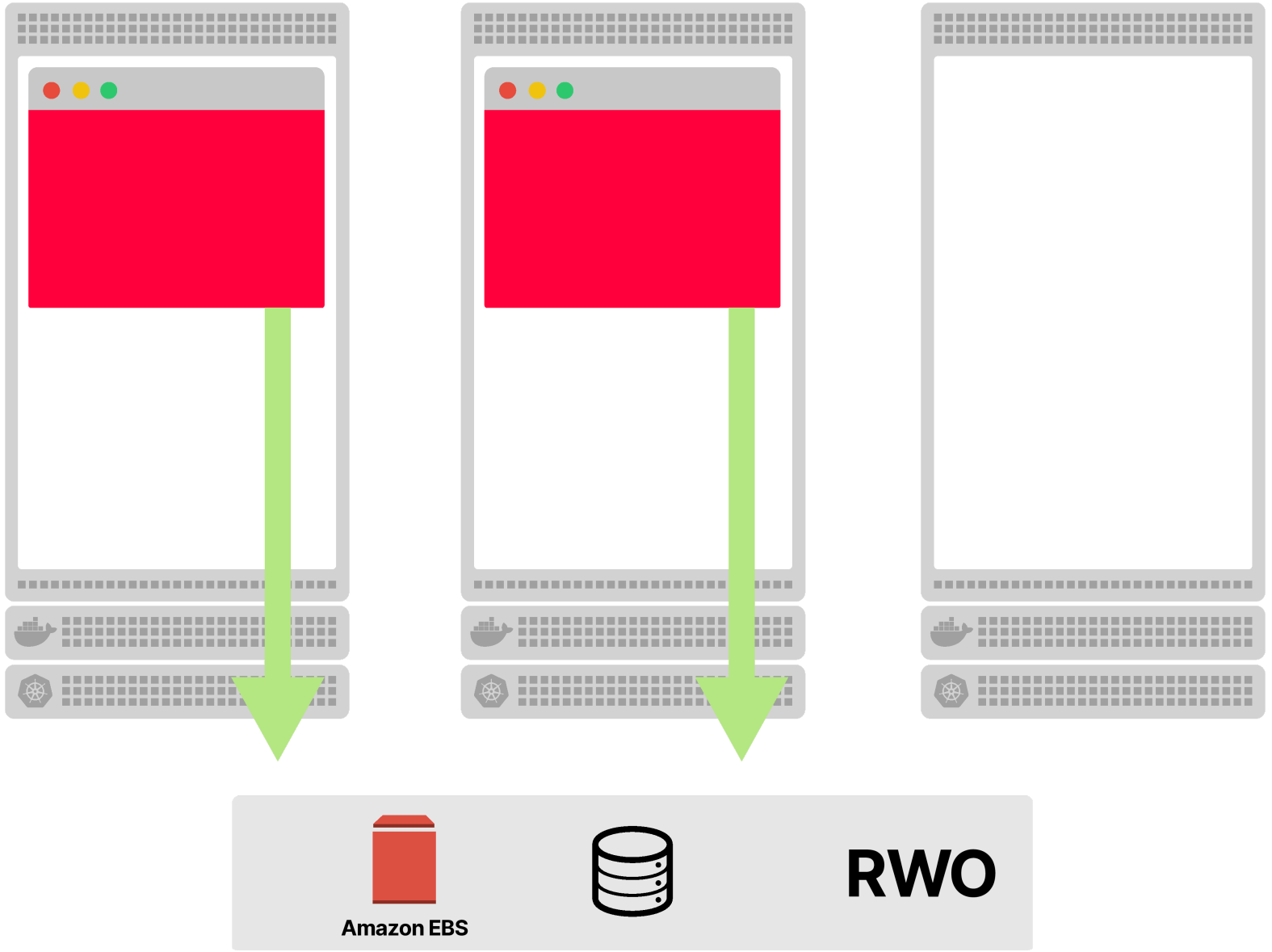
ReadWriteOnce
ReadOnlyMany
ReadWriteMany

Requests for volumes

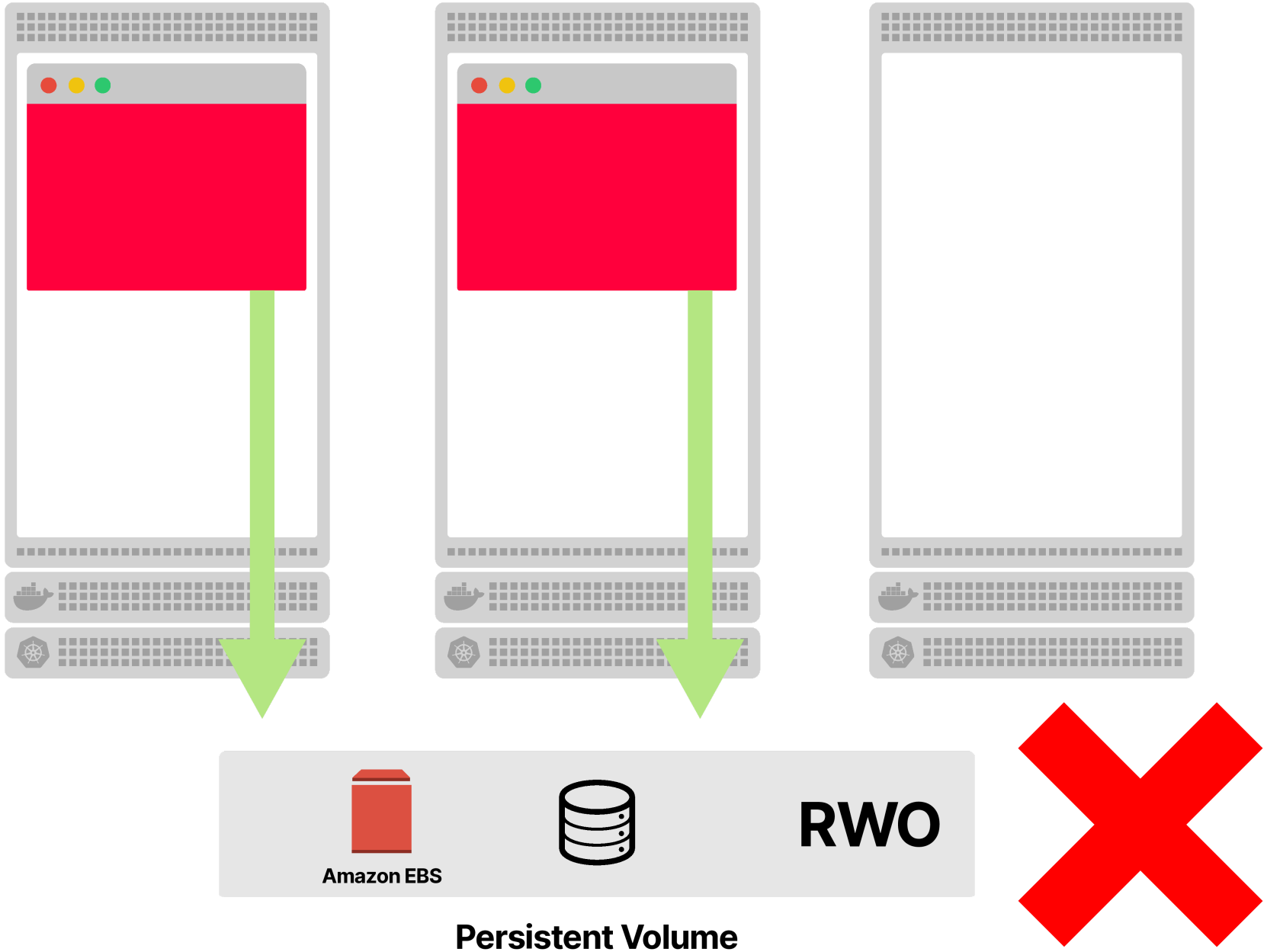
RWO
ROX
RWX

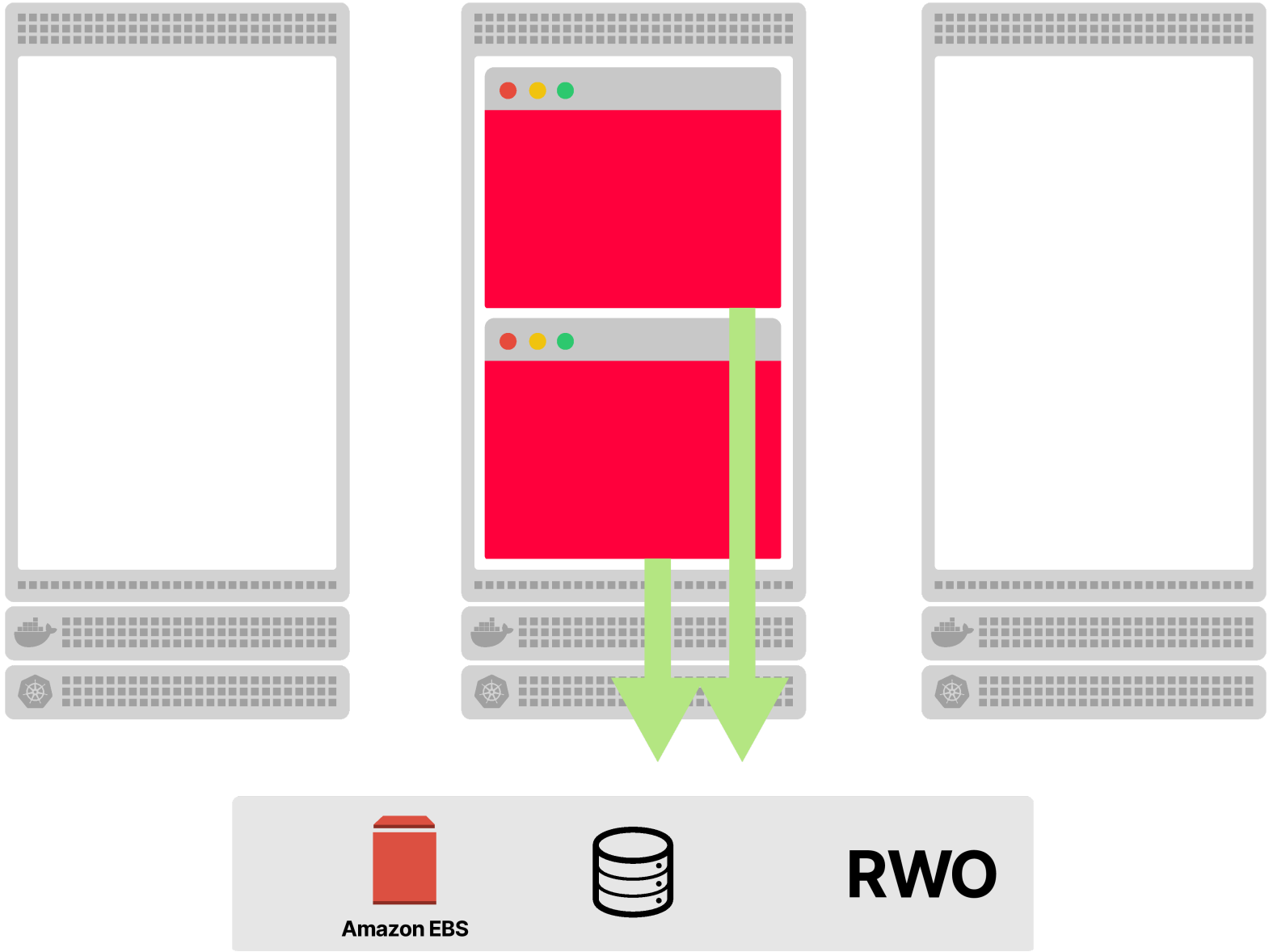


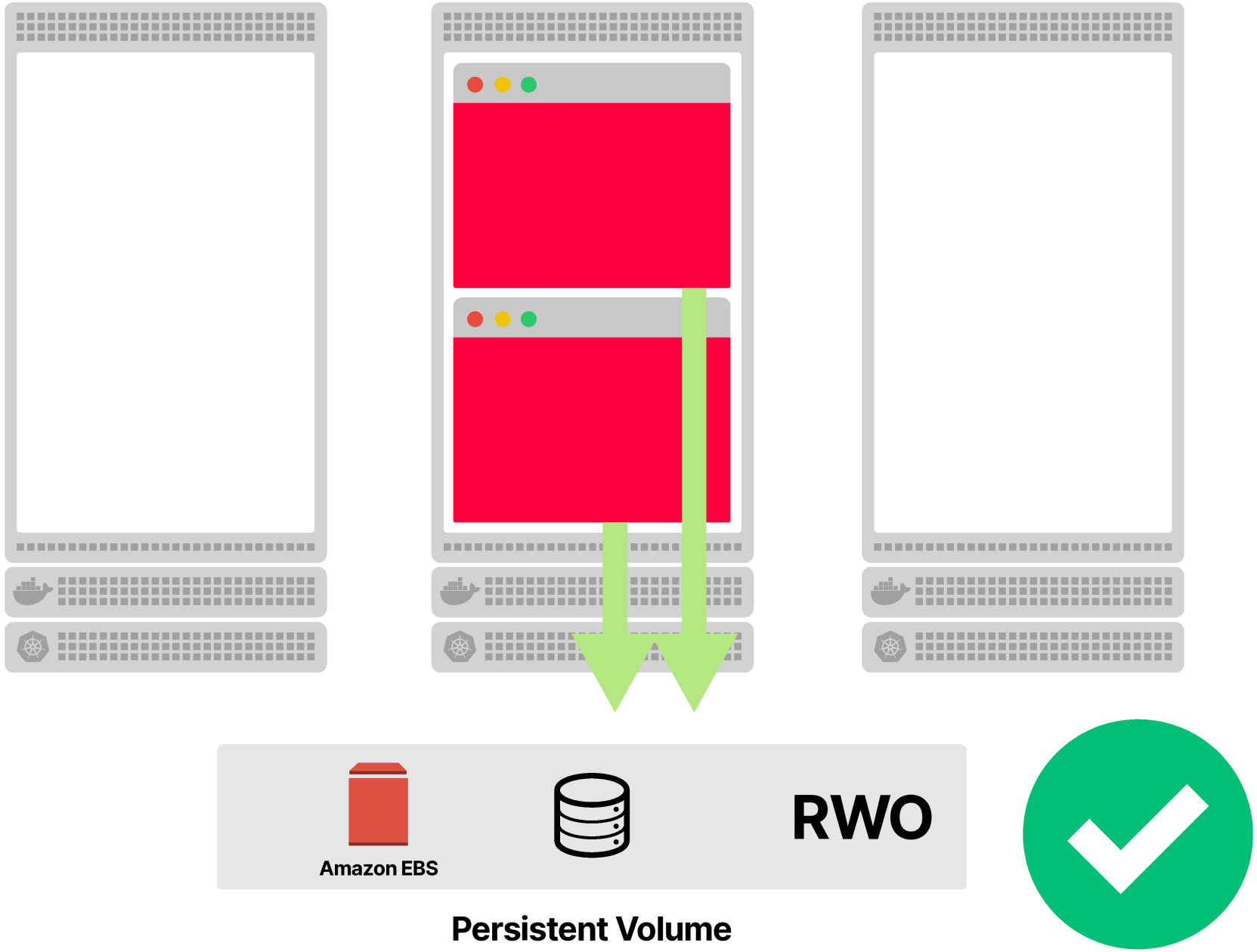
Persistent Volume



Persistent Volume


Amazon EBS
RWO
Persistent Volume


Amazon EBS
RWO
Persistent Volume


Amazon EBS
RWO
Persistent Volume

Requests for volumes — the scheduler

20 GiB RWM

20 GiB RWM

20 GiB ROX

20 GiB RWM

20 GiB ROX

20 GiB RWM

20 GiB RWO

20 GiB RWM

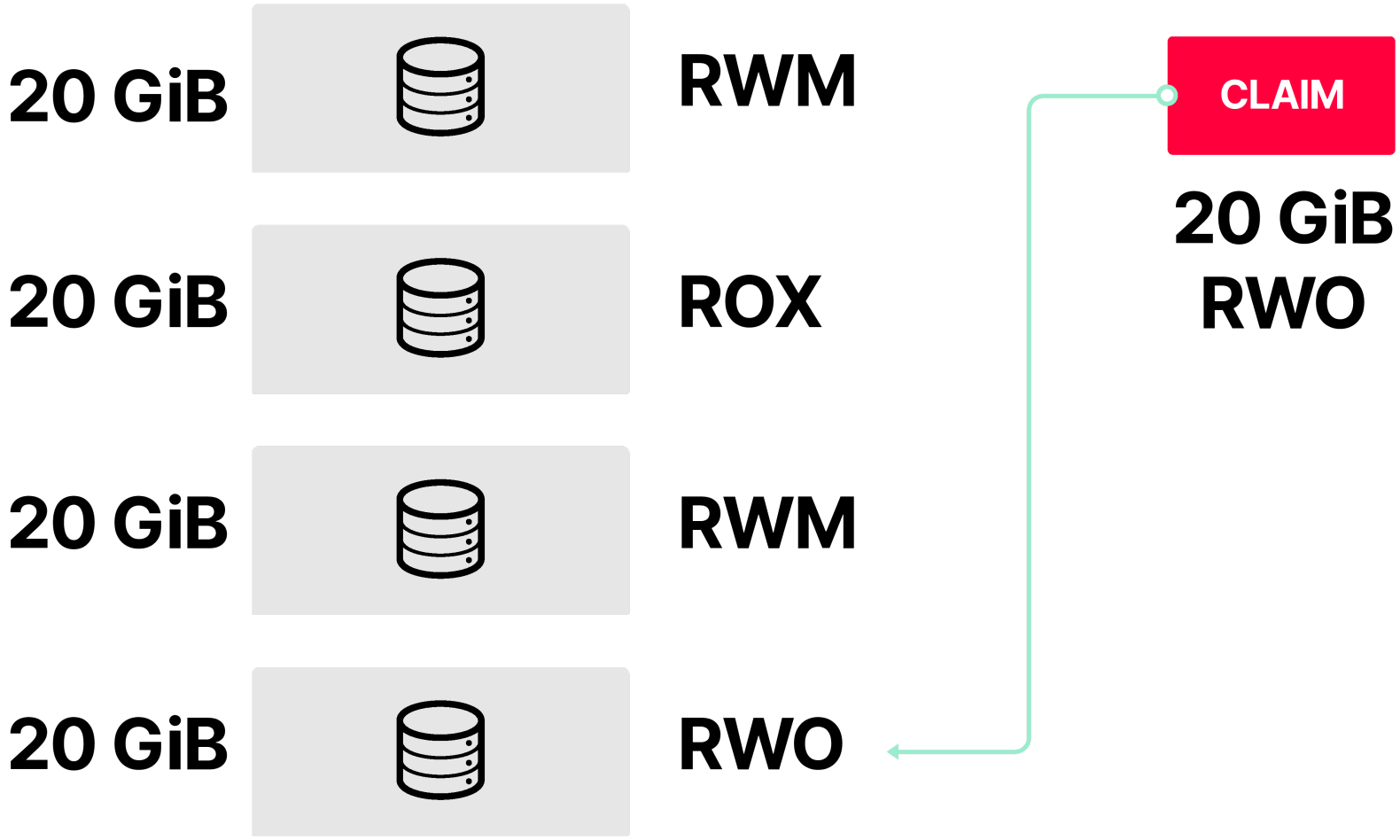
20 GiB ROX

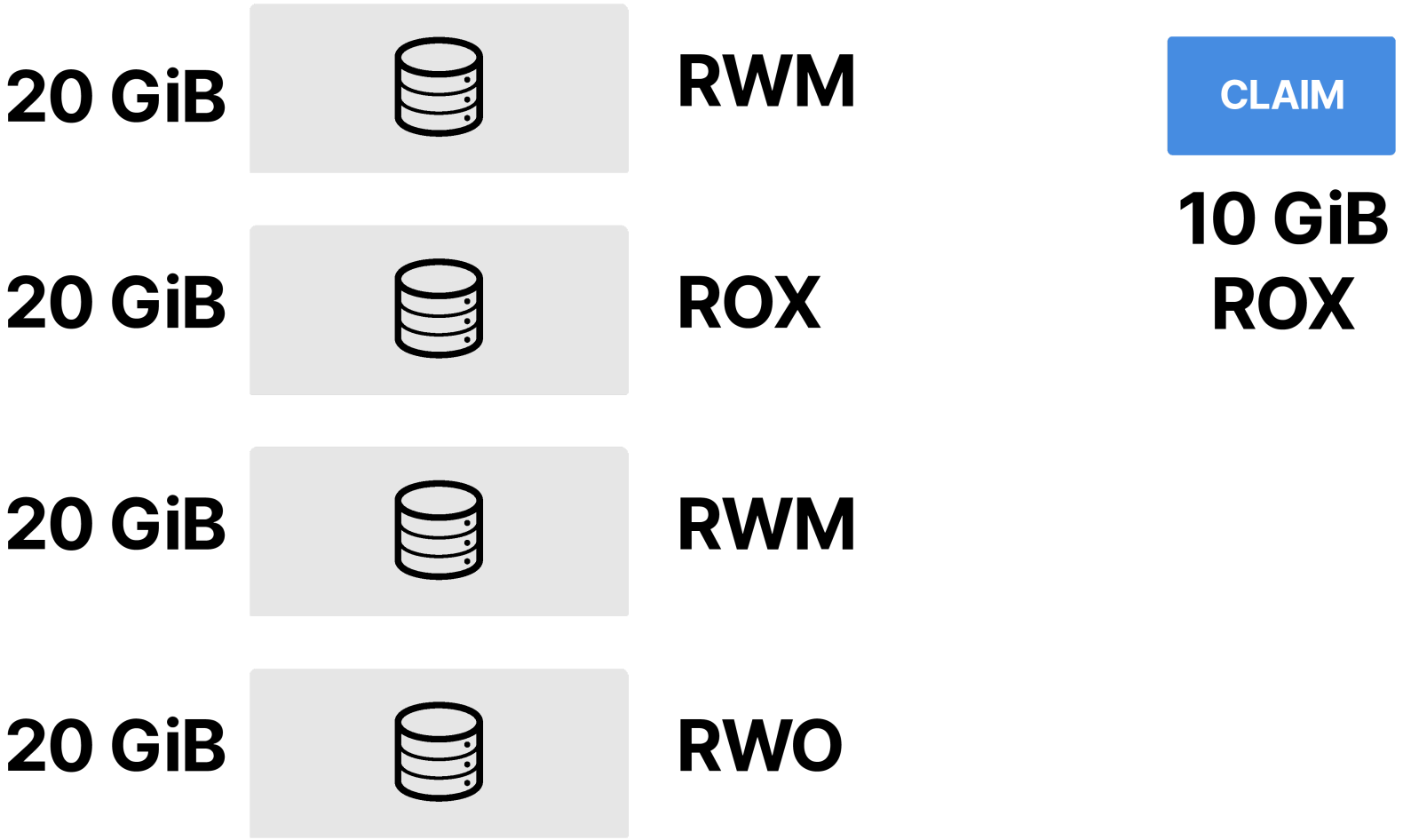
20 GiB RWM

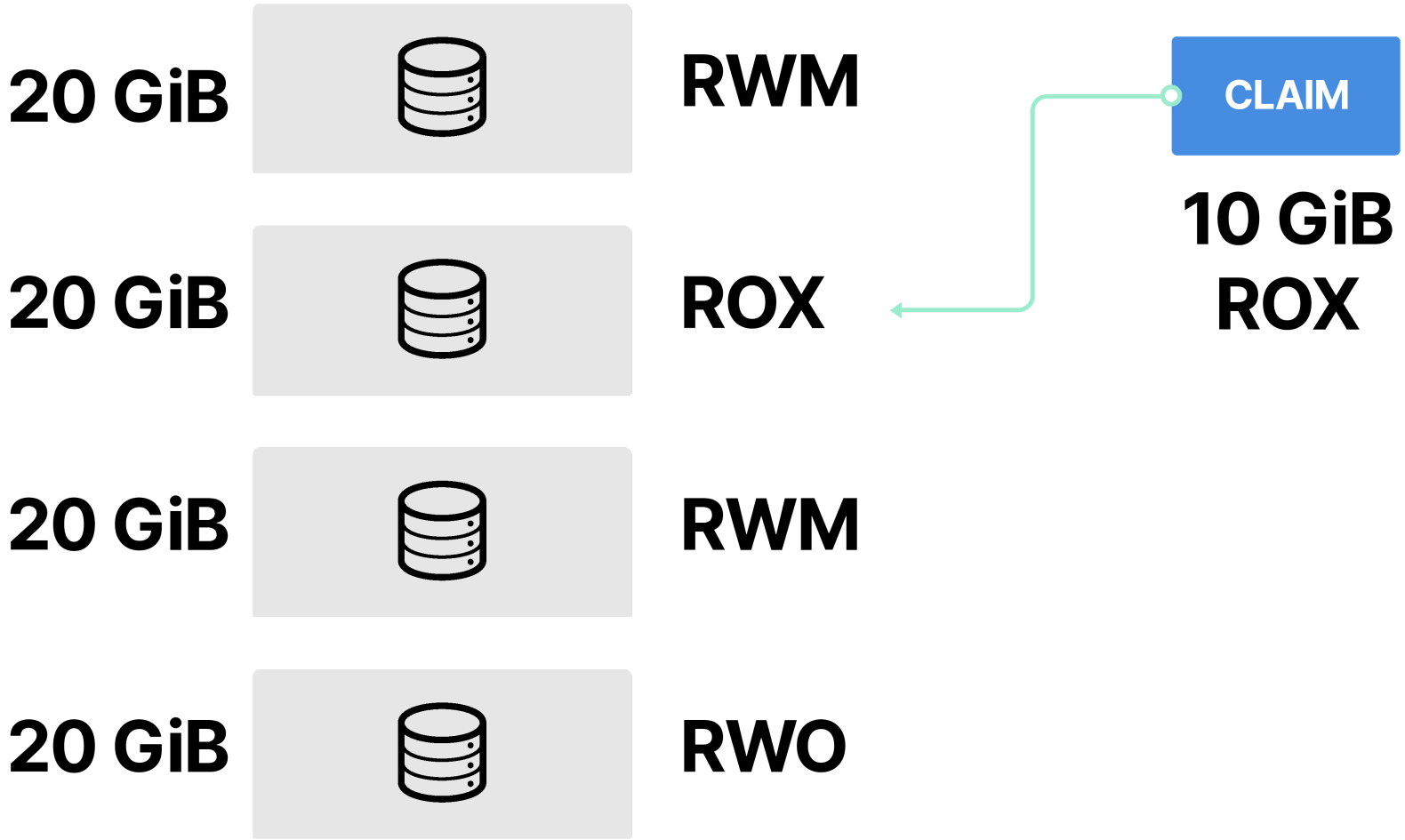
20 GiB RWO

CLAIM

20 GiB
RWO


20 GiB
RWM
20 GiB
ROX
20 GiB
RWM
20 GiB
RWO
CLAIM
20 GiB
RWO


20 GiB
RWM
20 GiB
ROX
20 GiB
RWM
20 GiB
RWO
CLAIM
10 GiB
ROX


20 GiB
RWM
20 GiB
ROX
20 GiB
RWM
20 GiB
RWO
CLAIM
10 GiB
ROX

20 GiB RWM

20 GiB ROX

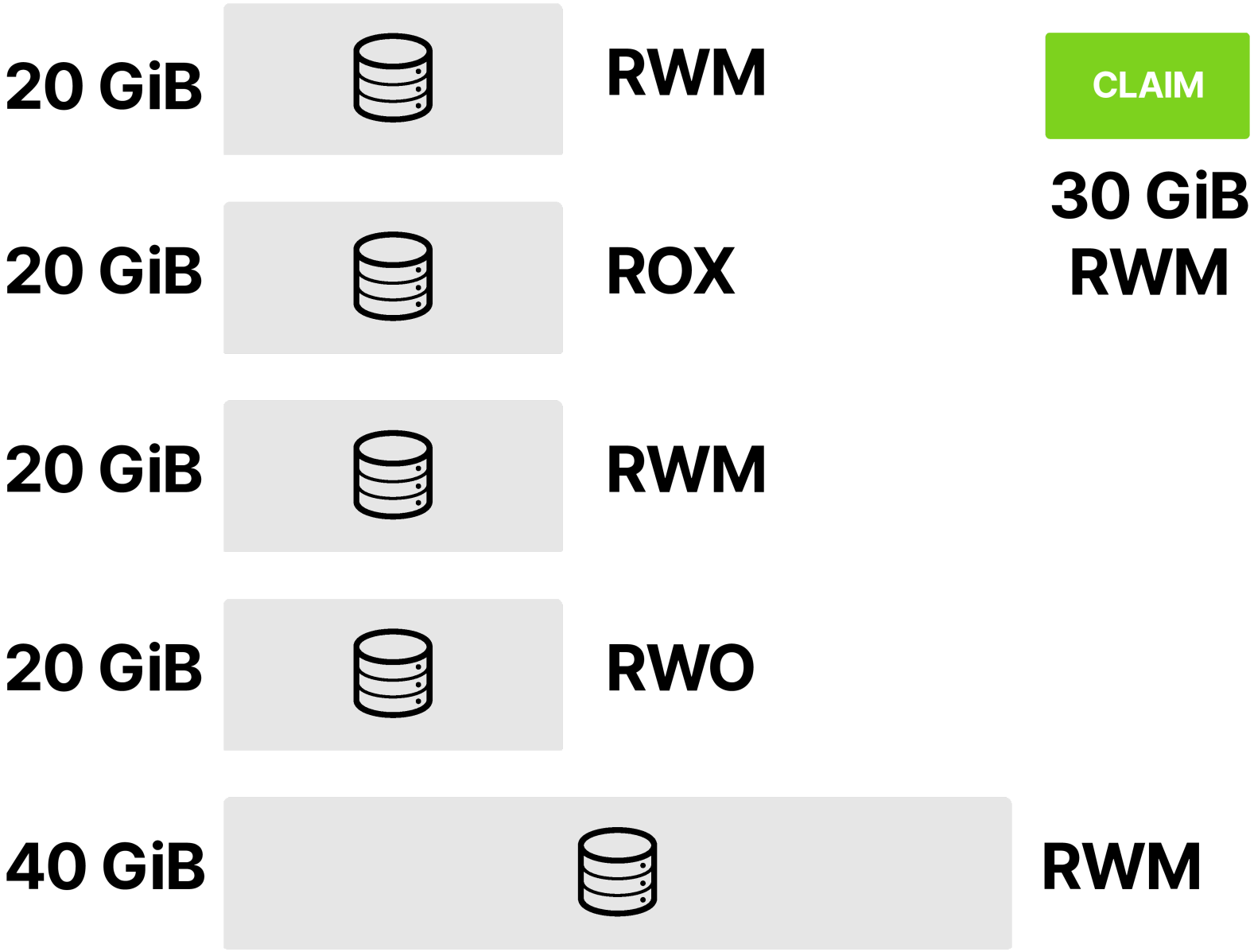
20 GiB RWM

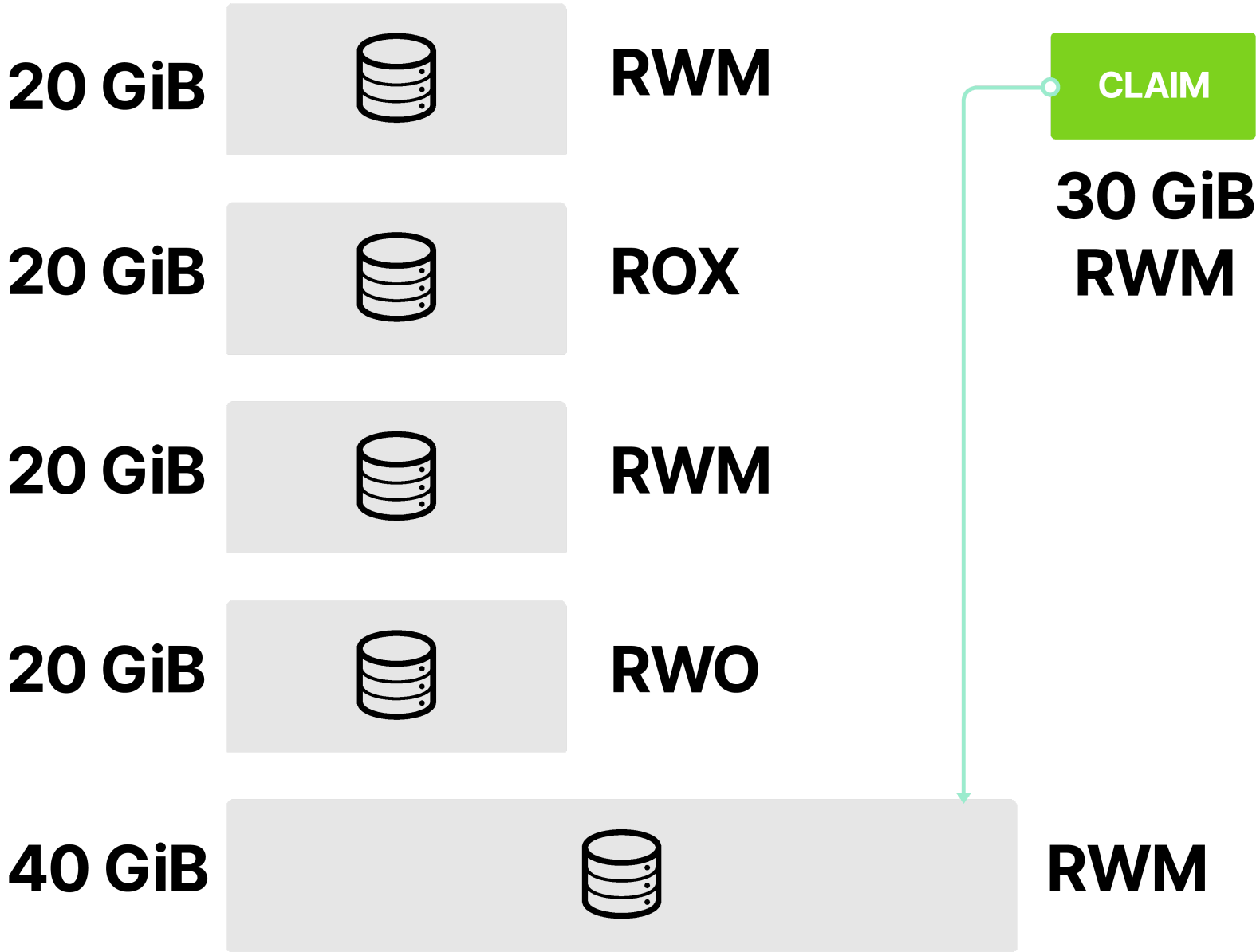
20 GiB RWO

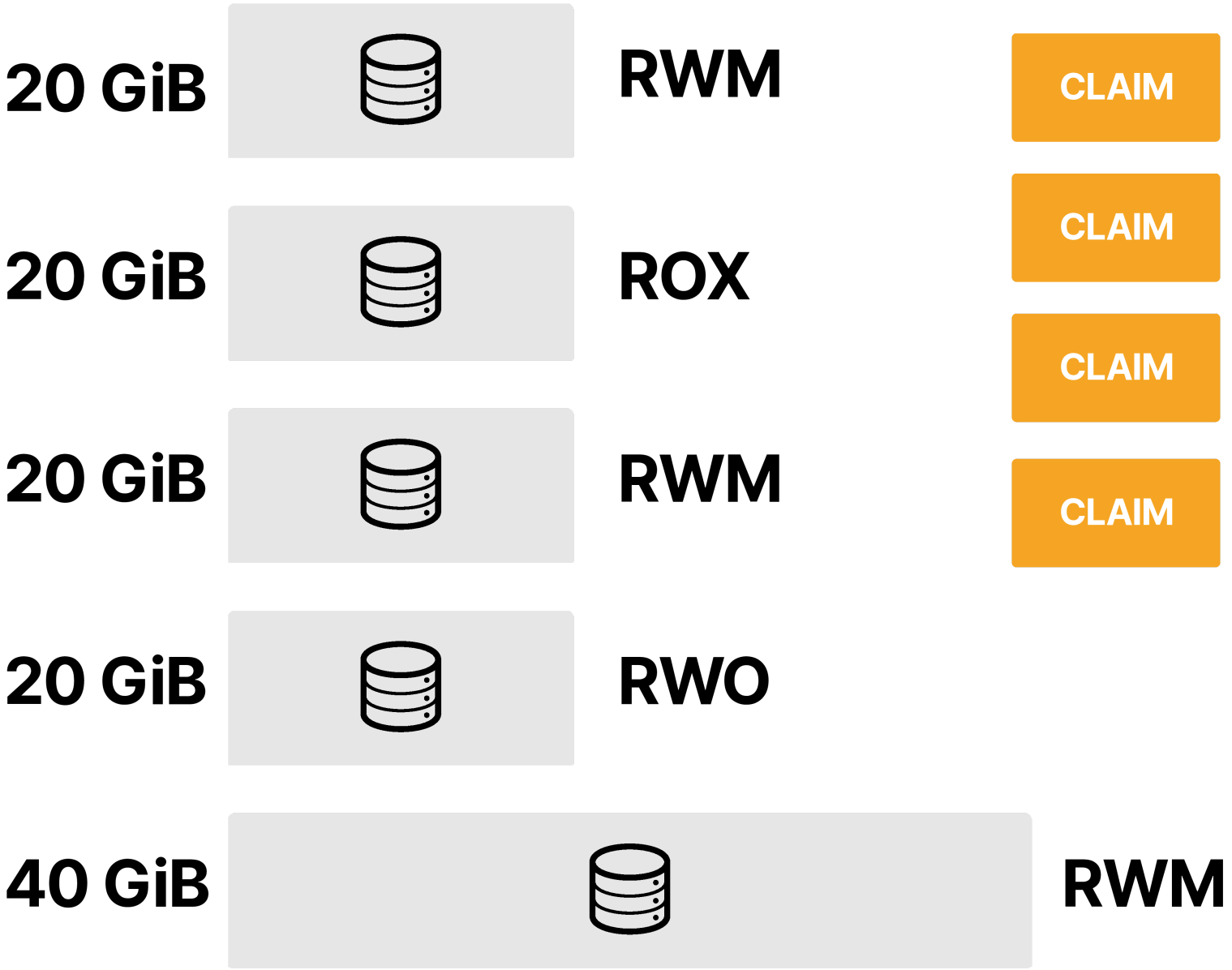
CLAIM

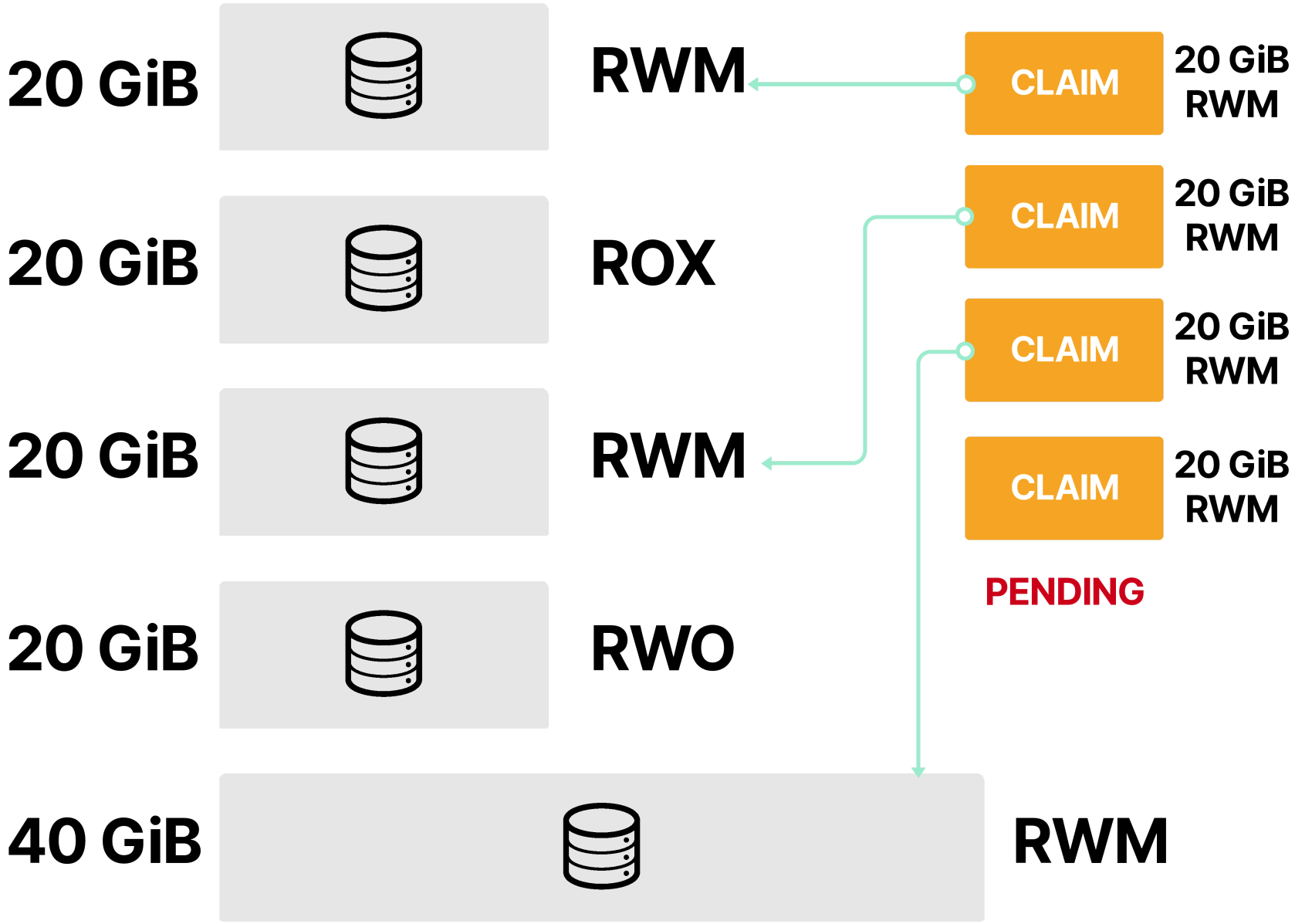
30 GiB
RWM

20 GiB
RWM
20 GiB
20 GiB
20 GiB
RWO
CLAIM
30 GiB
RWM
PENDING


20 GiB
RWM
20 GiB
ROX
20 GiB
RWM
20 GiB
RWO
40 GiB
RWM
CLAIM
30 GiB
RWM

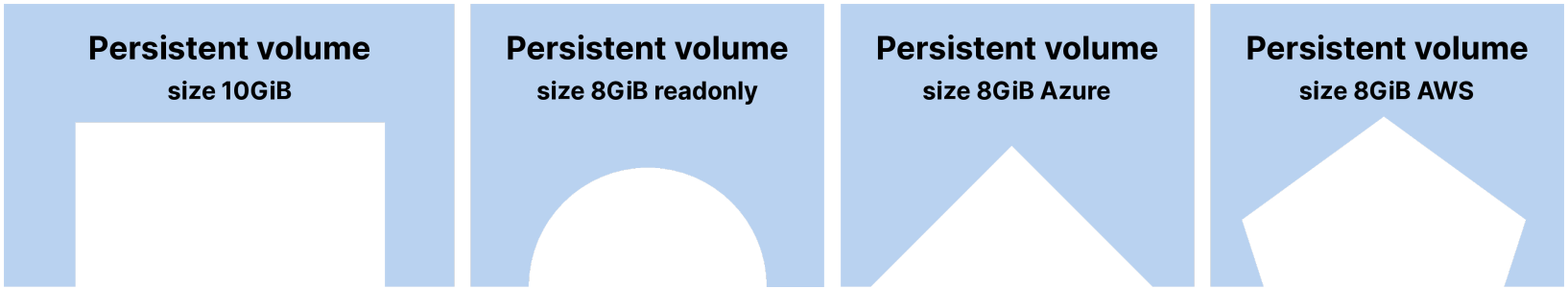

20 GiB
RWM
20 GiB
ROX
20 GiB
RWM
20 GiB
RWO
40 GiB
RWM
CLAIM
30 GiB
RWM


20 GiB
RWM
20 GiB
ROX
20 GiB
RWM
20 GiB
RWO
40 GiB
RWM
CLAIM
20 GiB
RWM
CLAIM
20 GiB
RWM
CLAIM
20 GiB
RWM


20 GiB
RWM
20 GiB
ROX
20 GiB
RWM
20 GiB
RWO
40 GiB
RWM
CLAIM
20 GiB
RWM
CLAIM
20 GiB
RWM
CLAIM
20 GiB
RWM
CLAIM
20 GiB
RWM
PENDING

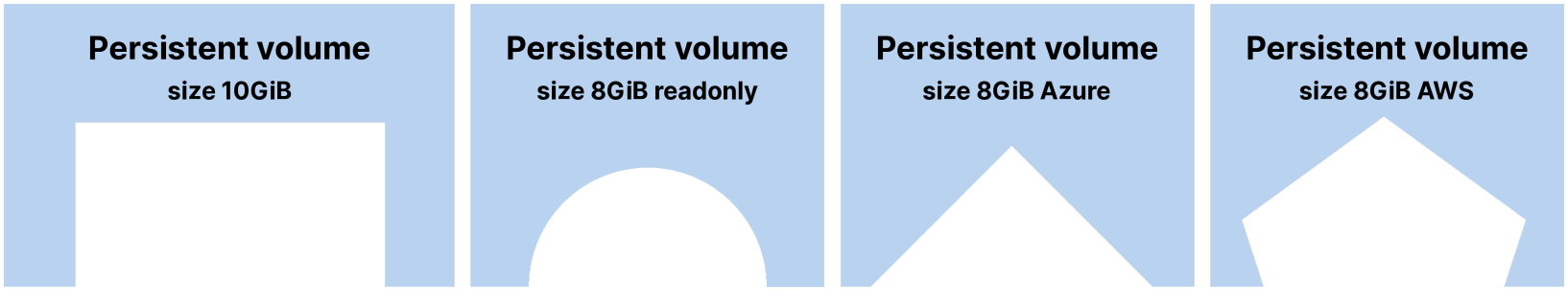
Dynamic provisioning

Templates



Standard **Safe** **Azure** **AWS**

Templates

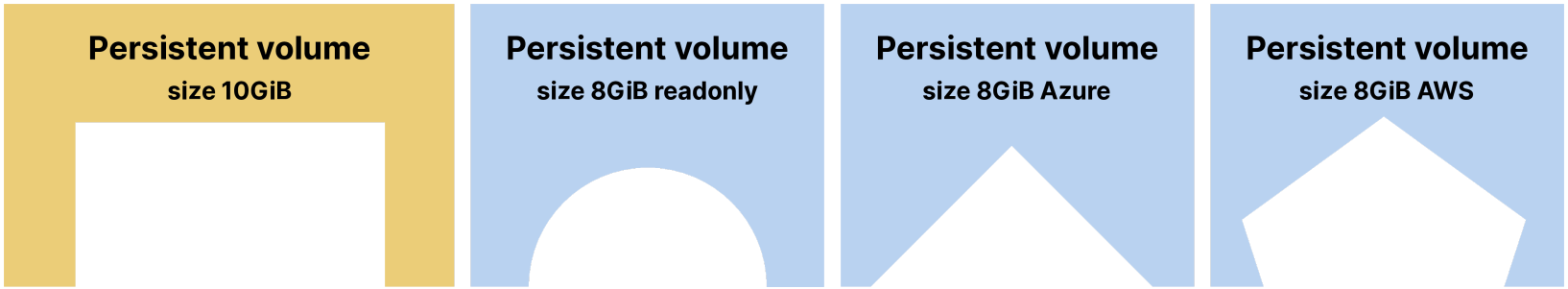


Standard **Safe** **Azure** **AWS**

Persistent Volume Claim

Standard

Templates

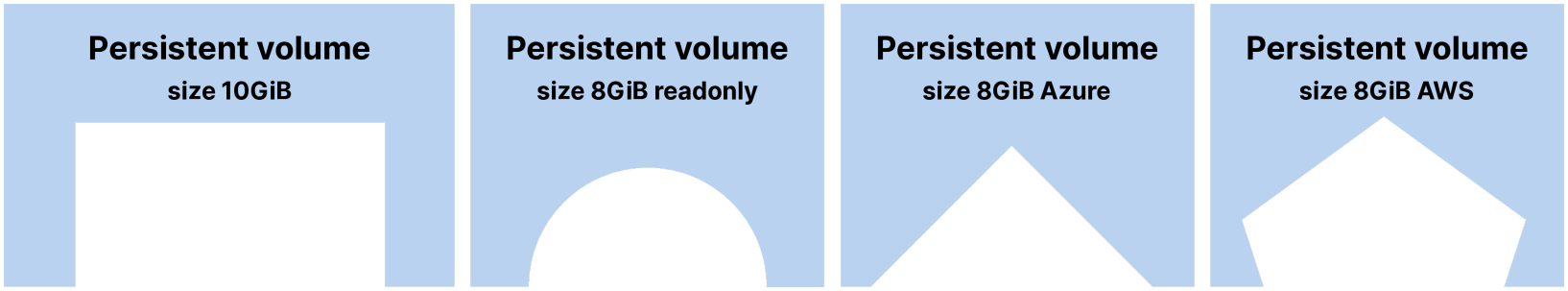


Standard **Safe** **Azure** **AWS**

Persistent Volume Claim

Standard

Templates


Persistent volume
size 10GiB
Persistent volume
size 8GiB readonly
Persistent volume
size 8GiB Azure
Persistent volume
size 8GiB AWS
Standard
Safe
Azure
AWS

Persistent volume
size 10GiB
Persistent Volume Claim
Standard

```
~$ cat pvc.yaml
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: myclaim
spec:
  accessModes:
    - ReadWriteOnce
  volumeMode: Filesystem
  resources:
    requests:
      storage: 8Gi
  storageClassName: slow
~$ _
```

```
~$ cat pvc.yaml
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: myclaim
spec:
  accessModes:
    - ReadWriteOnce
  volumeMode: Filesystem
  resources:
    requests:
      storage: 8Gi
  storageClassName: slow
~$ _
```

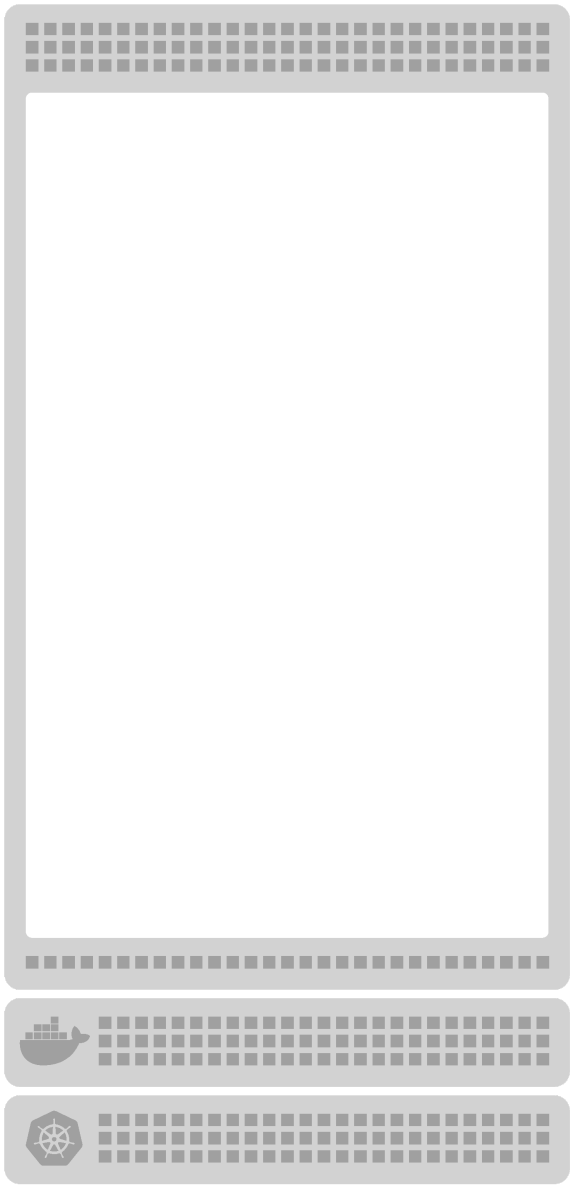
```
~$ cat pvc.yaml
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: myclaim
spec:
  accessModes:
    - ReadWriteOnce
  volumeMode: Filesystem
  resources:
    requests:
      storage: 8Gi
  storageClassName: slow
~$ _
```

Dynamic provisioning in practice

```
~$ cat storage-class.yaml
kind: StorageClass
apiVersion: storage.k8s.io/v1beta1
metadata:
  name: default
provisioner: kubernetes.io/aws-ebs
parameters:
  type: io1
  zone: us-east-1d
  zones: us-east-1d, us-east-1c
  iopsPerGB: "10"
  encrypted: "true"
  kmsKeyId: keyvalue
~$ _
```

```
~$ cat storage-class.yaml
kind: StorageClass
apiVersion: storage.k8s.io/v1beta1
metadata:
  name: default
provisioner: kubernetes.io/aws-ebs
parameters:
  type: io1
  zone: us-east-1d
  zones: us-east-1d, us-east-1c
  iopsPerGB: "10"
  encrypted: "true"
  kmsKeyId: keyvalue
~$ _
```

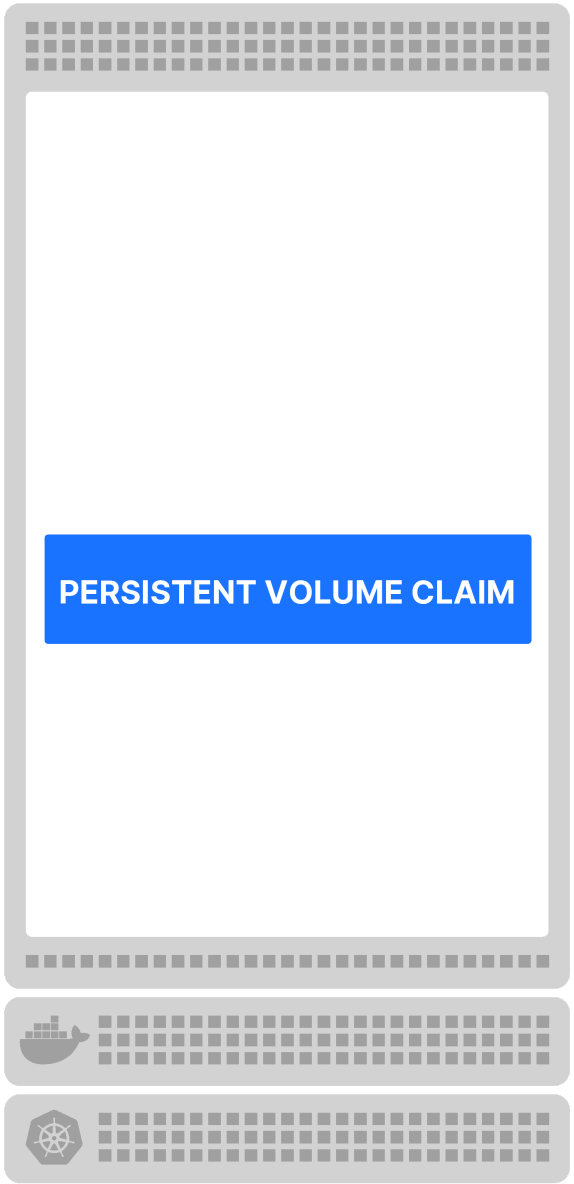
```
~$ cat storage-class.yaml
kind: StorageClass
apiVersion: storage.k8s.io/v1beta1
metadata:
  name: default
provisioner: kubernetes.io/aws-ebs
parameters:
  type: io1
  zone: us-east-1d
  zones: us-east-1d, us-east-1c
  iopsPerGB: "10"
  encrypted: "true"
  kmsKeyId: keyvalue
~$ _
```



StorageClass “default”

90 GiB
RWO

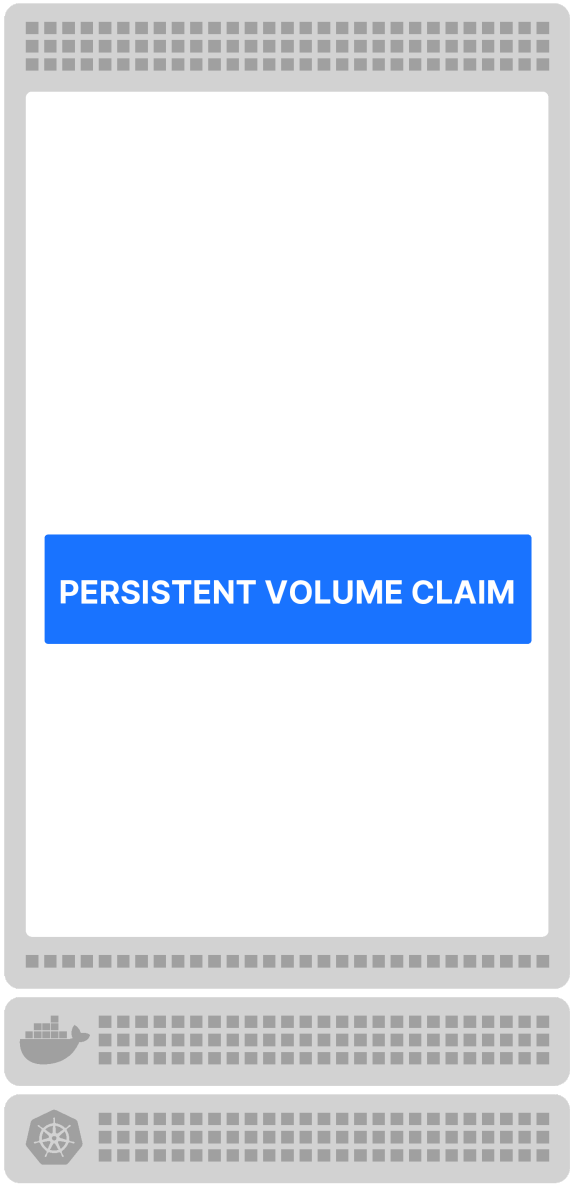
Amazon EBS



StorageClass “default”

90 GiB
RWO

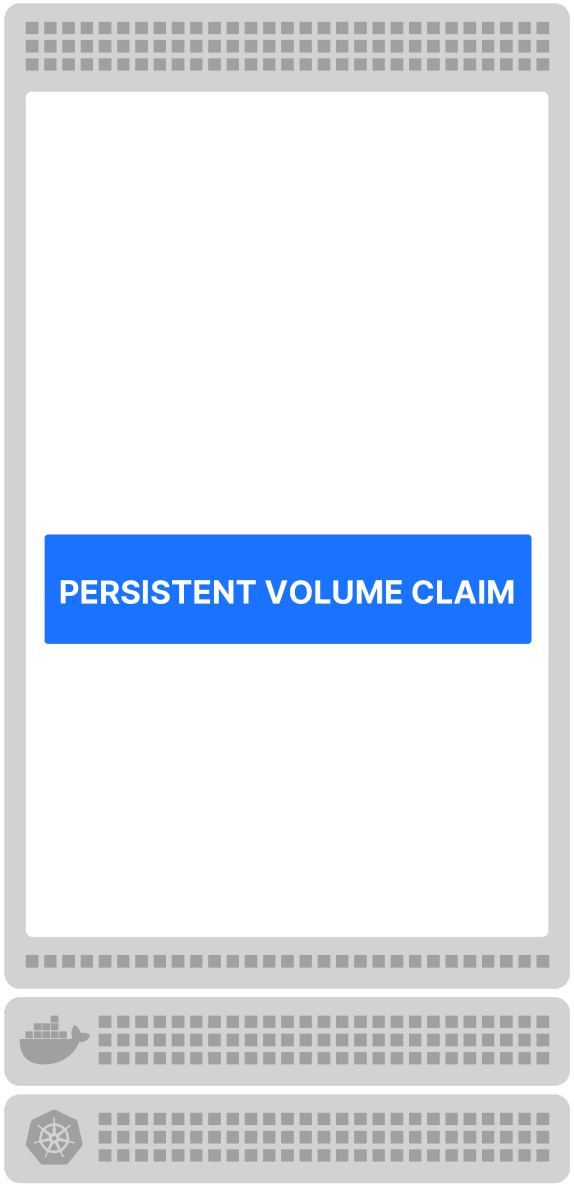
Amazon EBS



StorageClass “default”

90 GiB
RWO

Amazon EBS

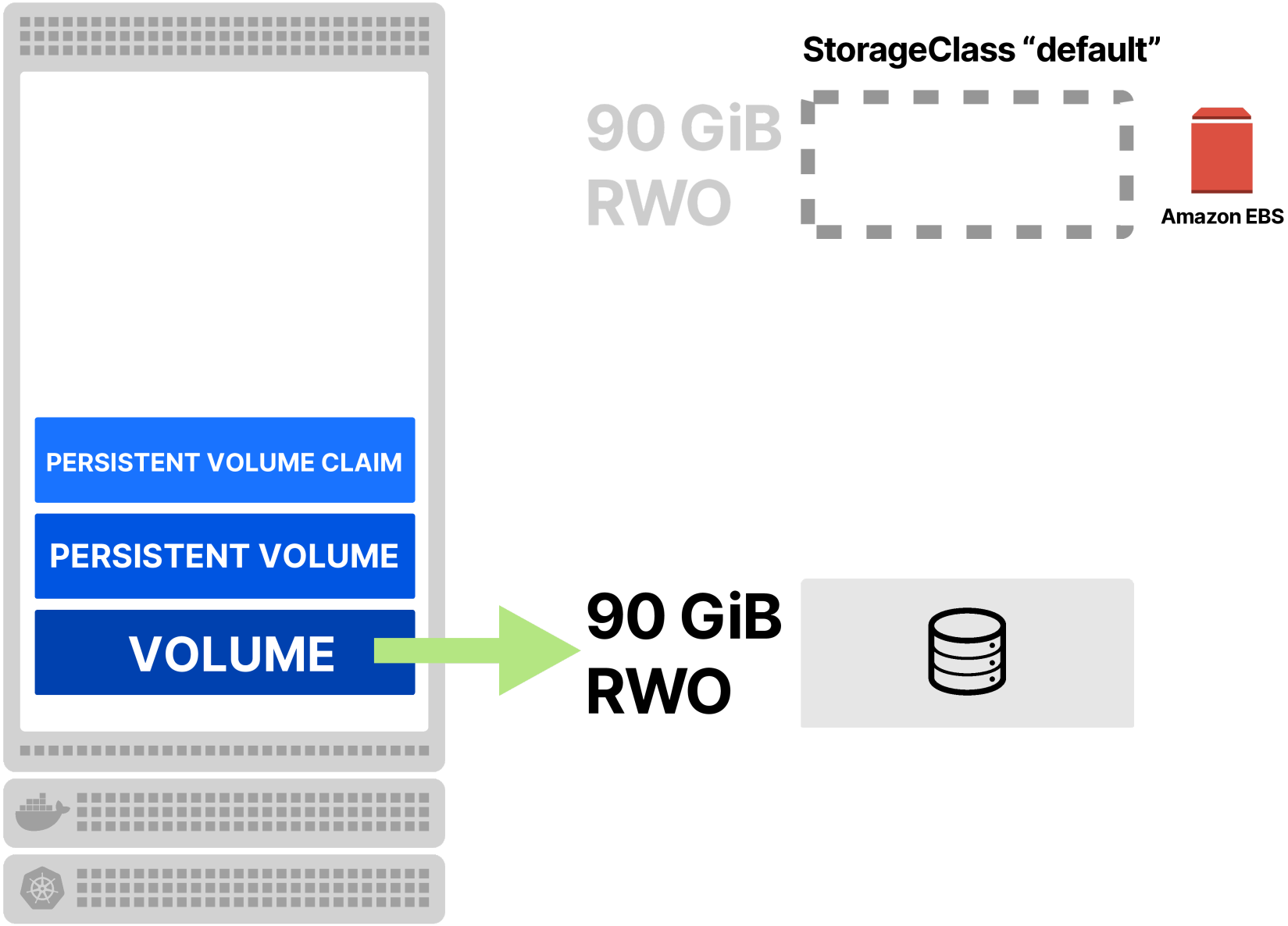


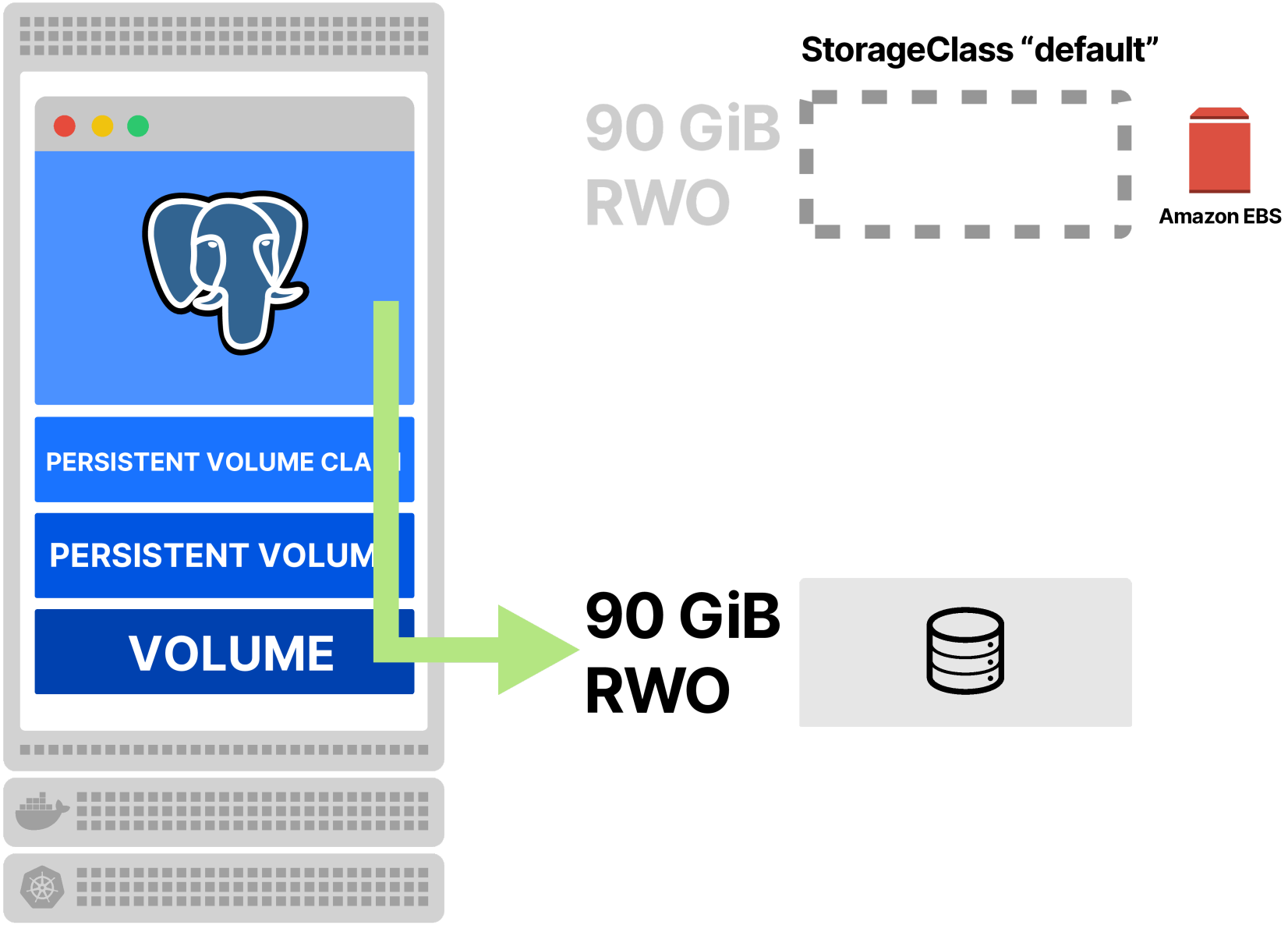
StorageClass “default”

90 GiB
RWO

Amazon EBS

90 GiB
RWO


StorageClass “default”
90 GiB
RWO
Amazon EBS
PERSISTENT VOLUME CLAIM
PERSISTENT VOLUME
VOLUME
90 GiB
RWO


StorageClass “default”
90 GiB
RWO
Amazon EBS
PERSISTENT VOLUME CLA
PERSISTENT VOLUM
VOLUME
90 GiB
RWO

Recap

Persistent Volumes, Persistent Volume Claims, StorageClass

Recap — PV, PVC, StorageClass

1. PVs map volumes
2. PVC: “I want to use a PV”
3. StorageClass is PVs on demand
4. Scheduling PVCs
5. Access modes

Recap — PV, PVC, StorageClass

1. PVs map volumes

2. PVC: “I want to use a PV”

3. StorageClass is PVs on demand

4. Scheduling PVCs

5. Access modes

Recap — PV, PVC, StorageClass

1. PVs map volumes

2. PVC: “I want to use a PV”

3. StorageClass is PVs on demand

4. Scheduling PVCs

5. Access modes

Recap — PV, PVC, StorageClass

1. PVs map volumes
2. PVC: “I want to use a PV”
3. StorageClass is PVs on demand
4. **Scheduling PVCs**
5. Access modes

Recap — PV, PVC, StorageClass

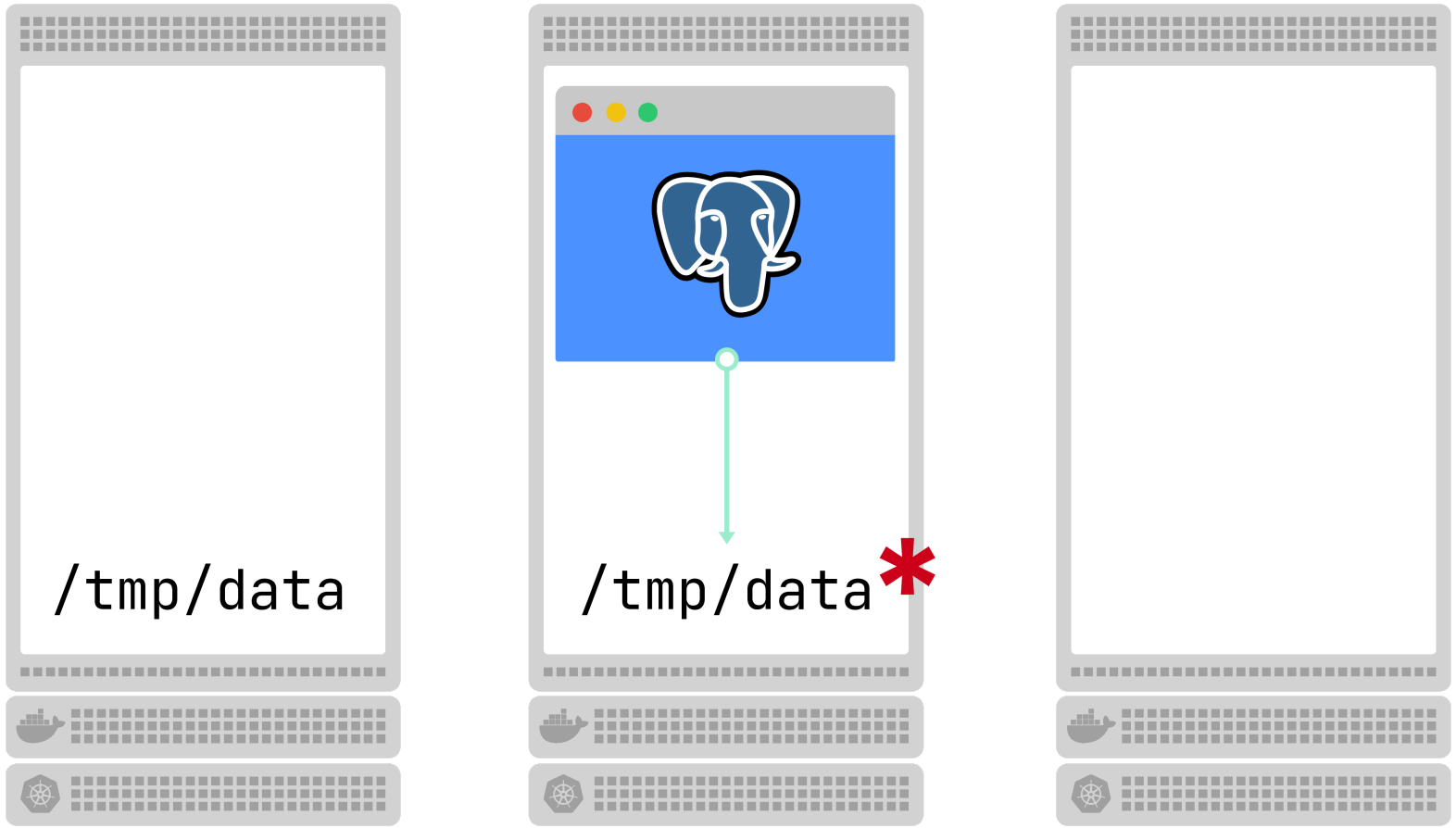
1. PVs map volumes
2. PVC: “I want to use a PV”
3. StorageClass is PVs on demand
4. Scheduling PVCs
5. Access modes

Popular volumes

03

Local volumes

01


/tmp/data
/tmp/data*

Local filesystem

Statically created PV

Scheduler extra hints

Local volumes

Local filesystem

Statically created PV

Scheduler extra hints

Local volumes

Local filesystem

Statically created PV

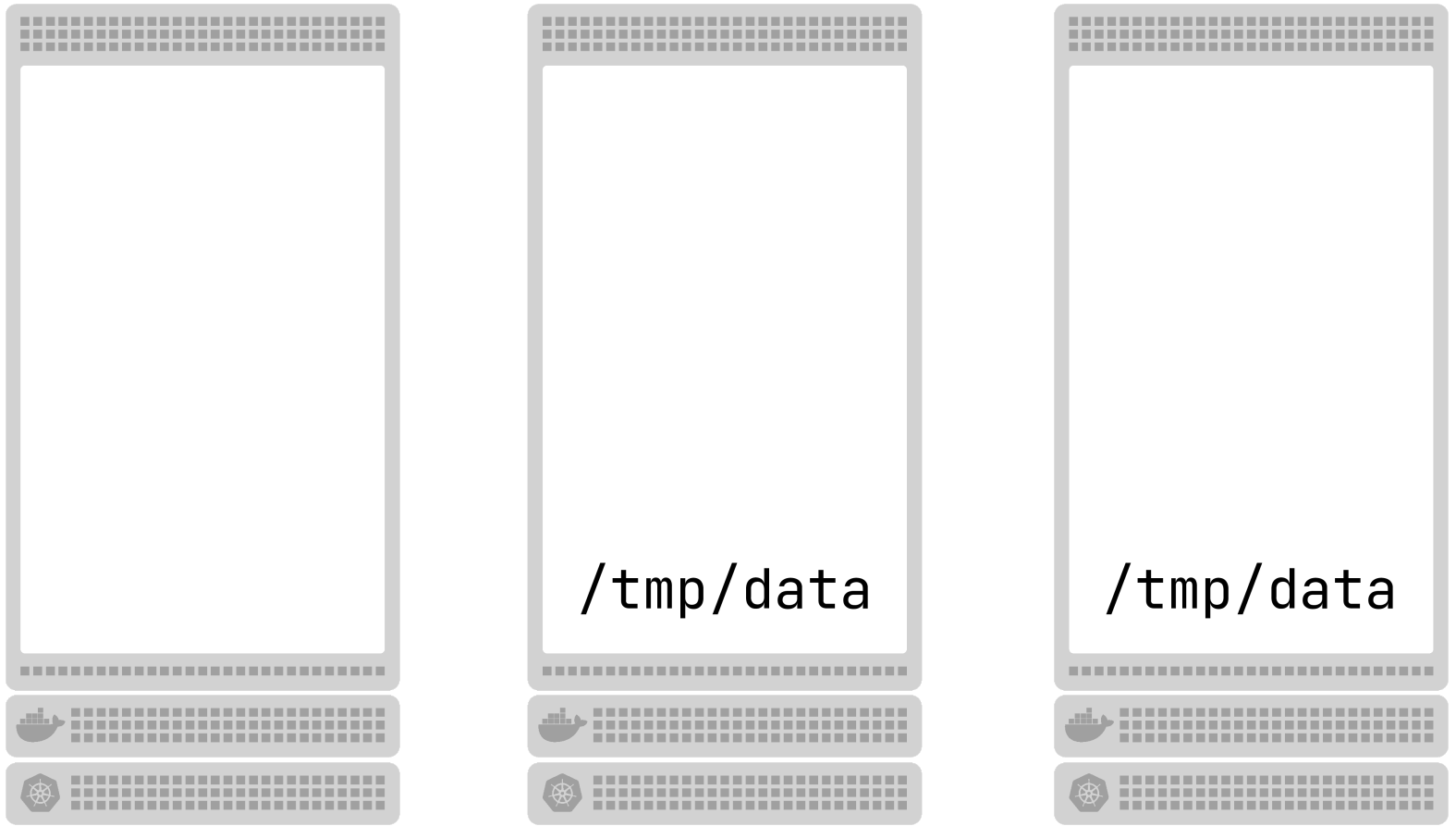
Scheduler extra hints

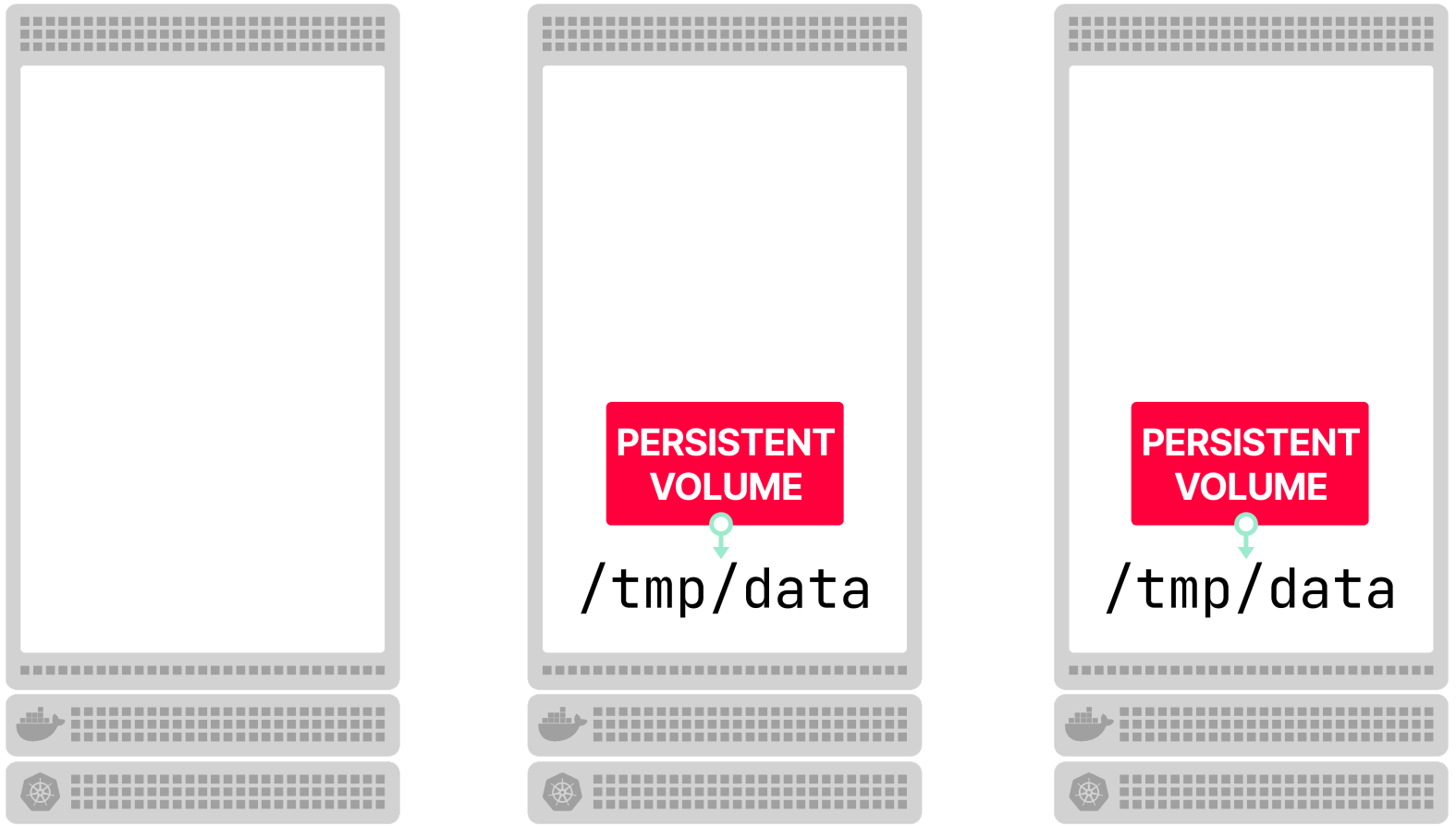
If a node becomes unhealthy, then the local volume will also become inaccessible

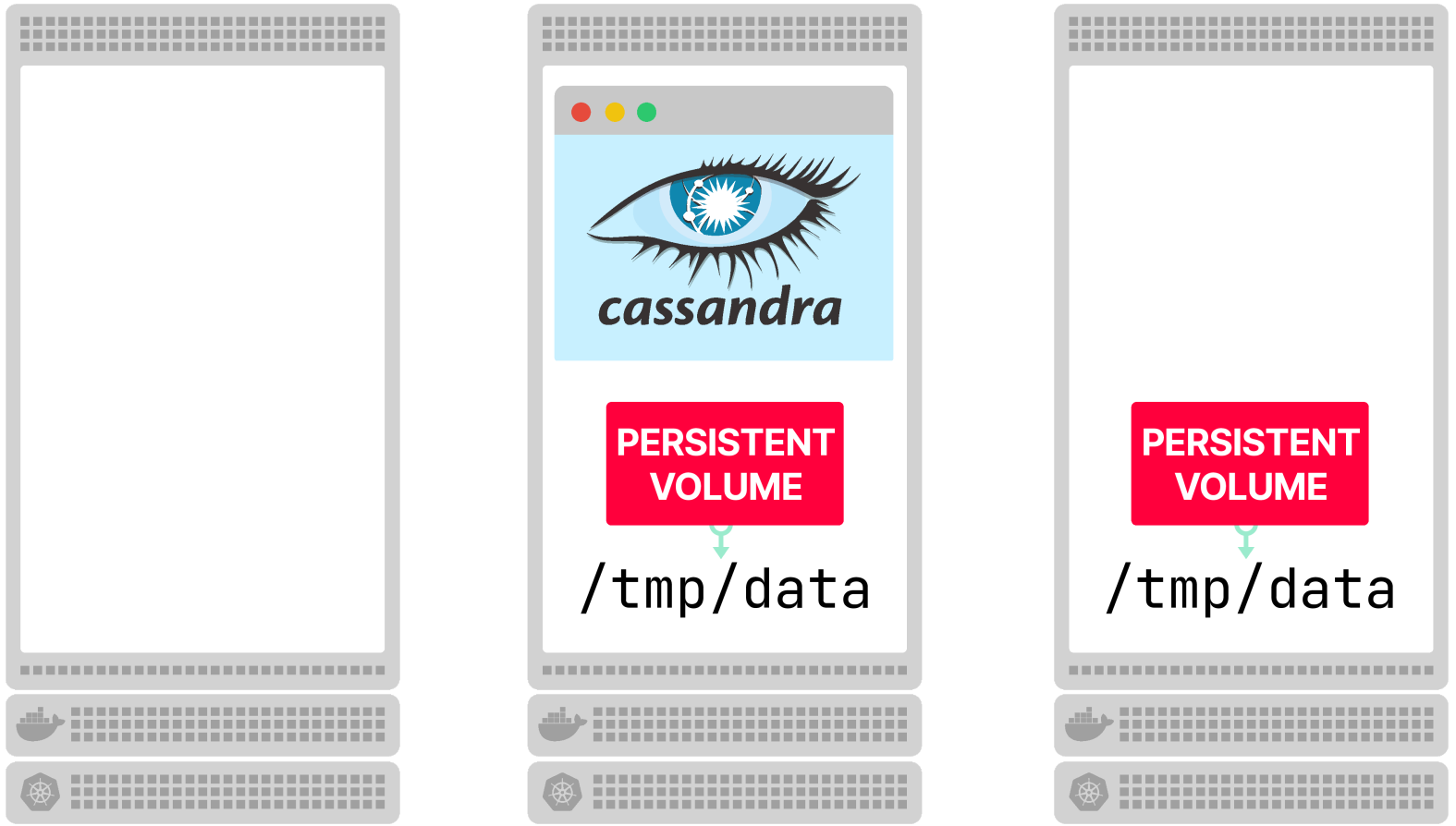
---

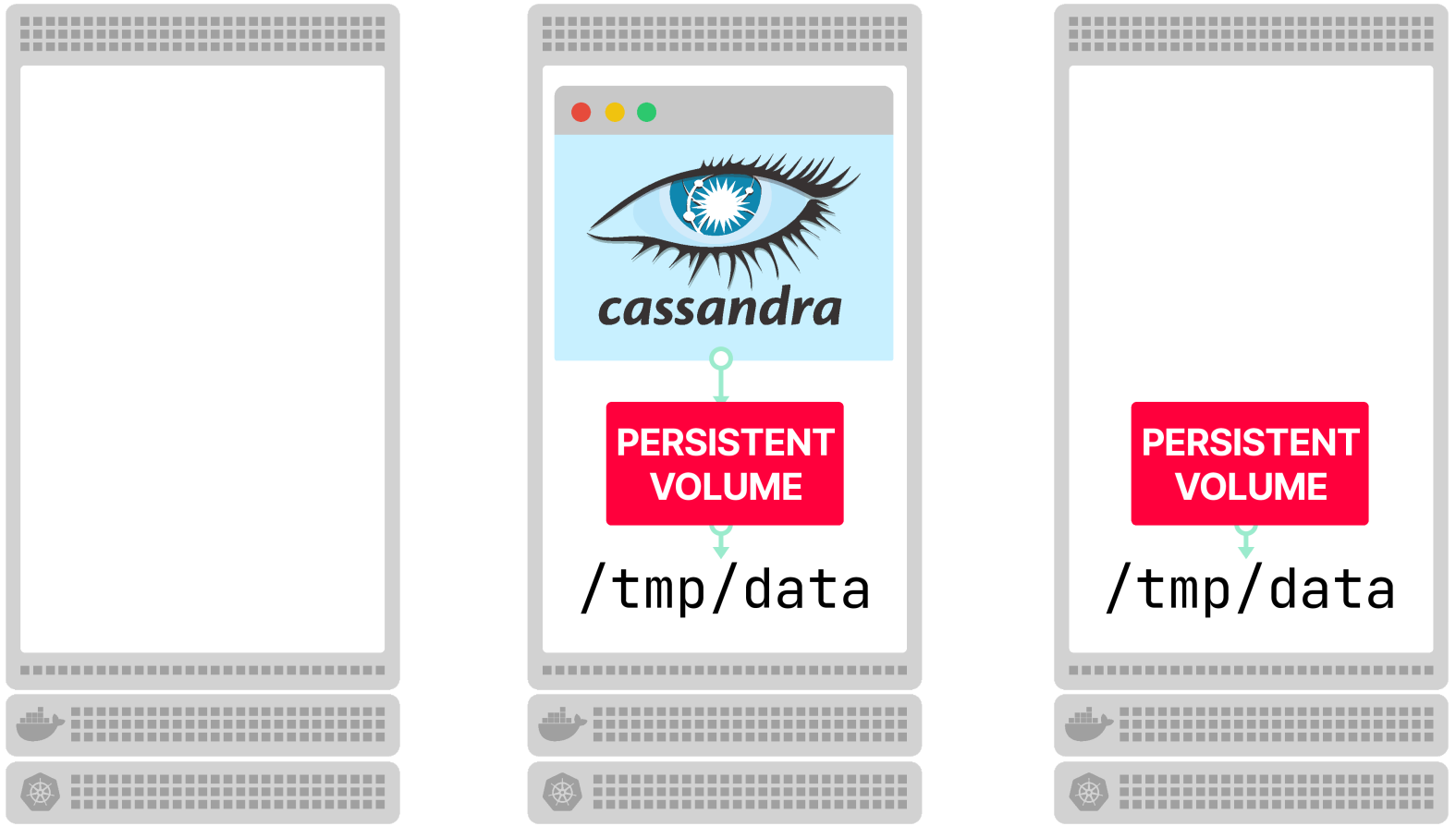
Local volumes

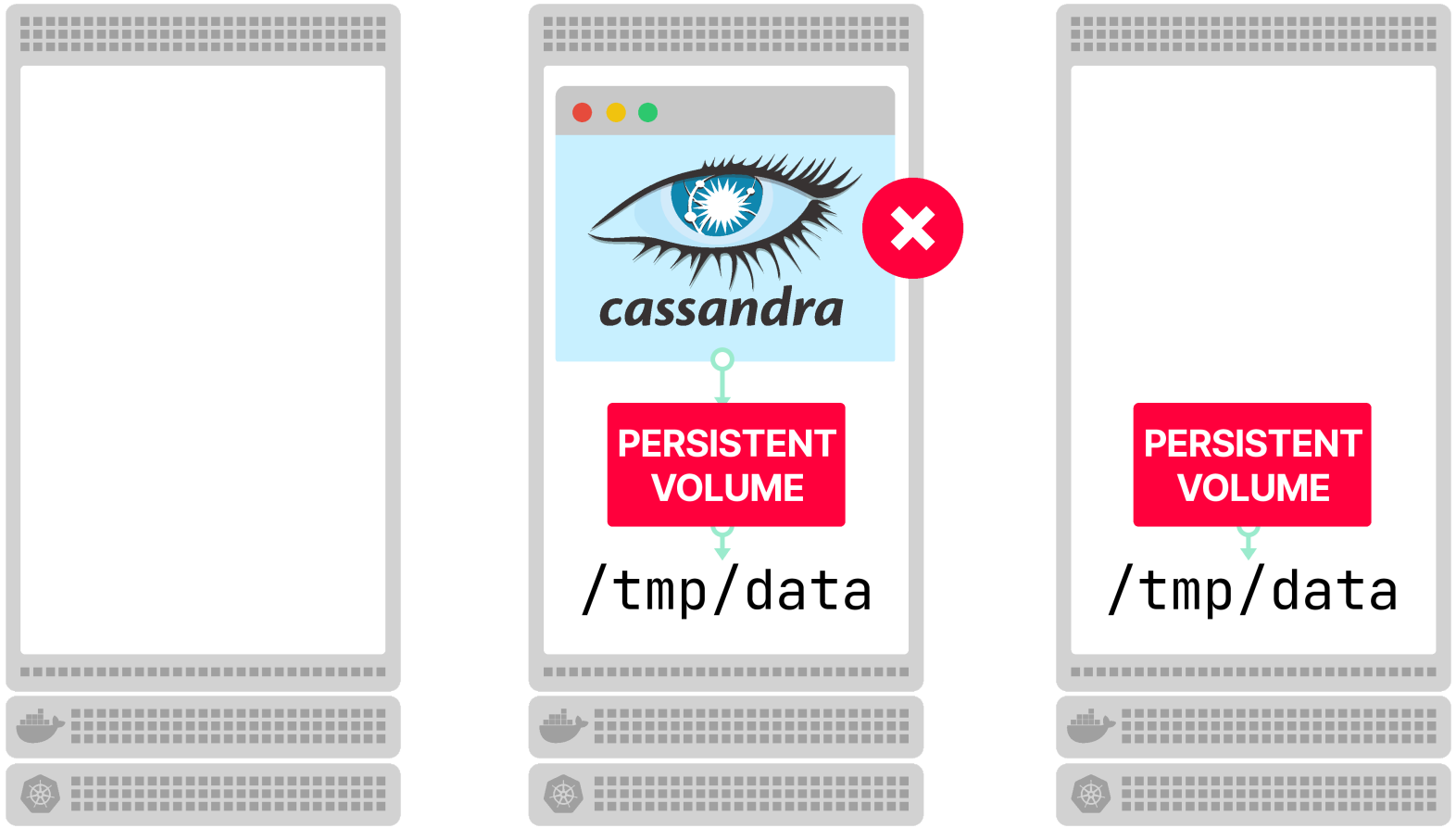
Local volumes in practice

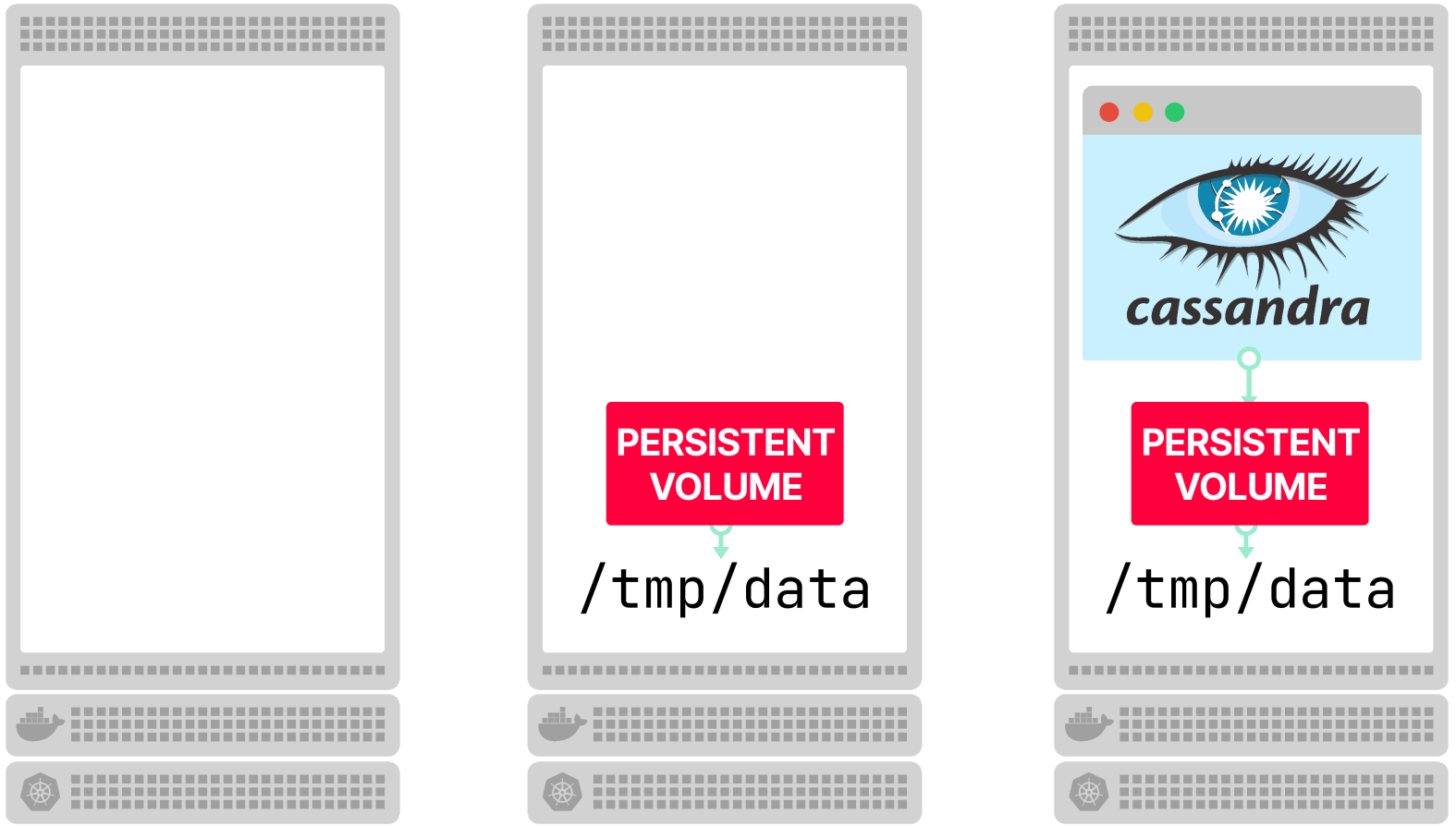

/tmp/data
/tmp/data


PERSISTENT VOLUME
/tmp/data
PERSISTENT VOLUME
/tmp/data


cassandra
PERSISTENT VOLUME
/tmp/data
PERSISTENT VOLUME
/tmp/data


cassandra
PERSISTENT VOLUME
/tmp/data
PERSISTENT VOLUME
/tmp/data


cassandra
PERSISTENT VOLUME
/tmp/data
PERSISTENT VOLUME
/tmp/data


PERSISTENT VOLUME
/tmp/data
cassandra
PERSISTENT VOLUME
/tmp/data

Useful for distributed storage systems that shard or replicate data across multiple nodes

---

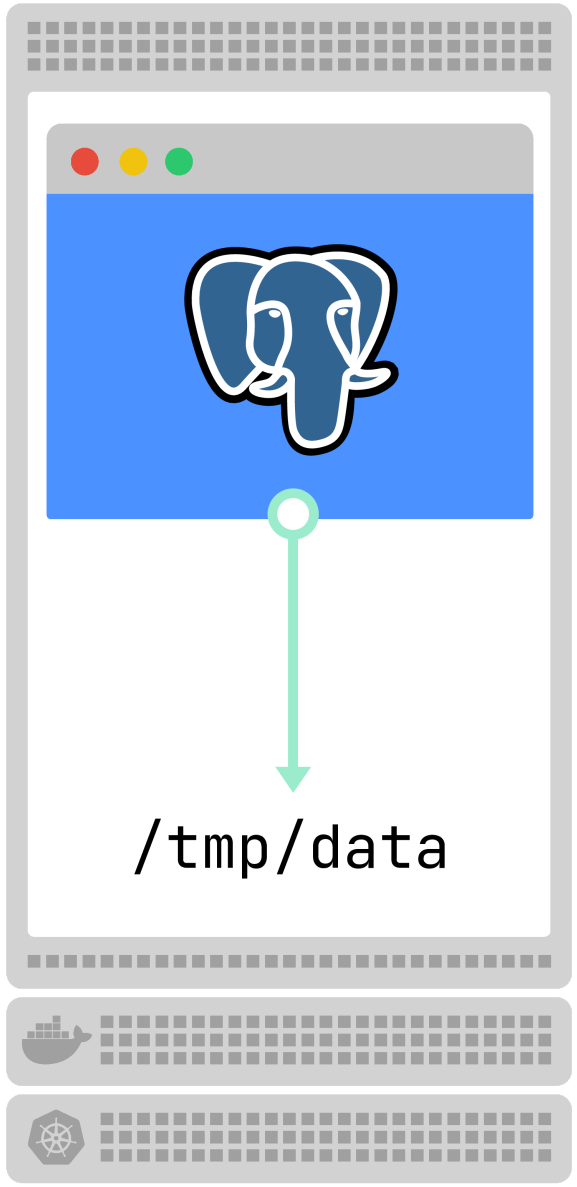
Local volumes in practice

Local volumes in practice

Cassandra
Gluster
Ceph

hostPath

02


/tmp/data

hostPath

Mount folder from node

GPUs, TUPs, /proc, etc.
Ignored by scheduler

hostPath

Mount folder from node

GPUs, TUPs, /proc, etc.

Ignored by scheduler

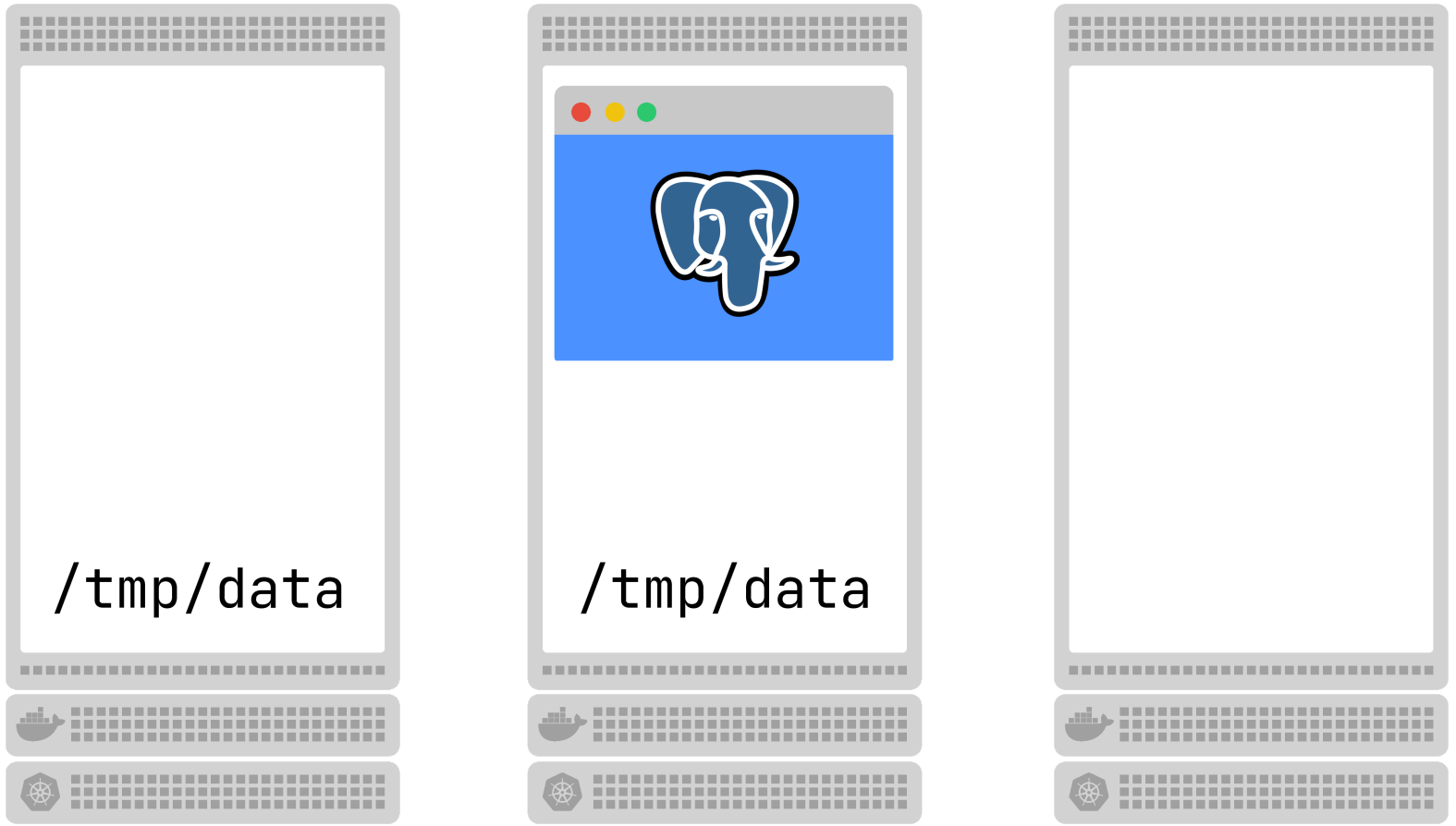
hostPath

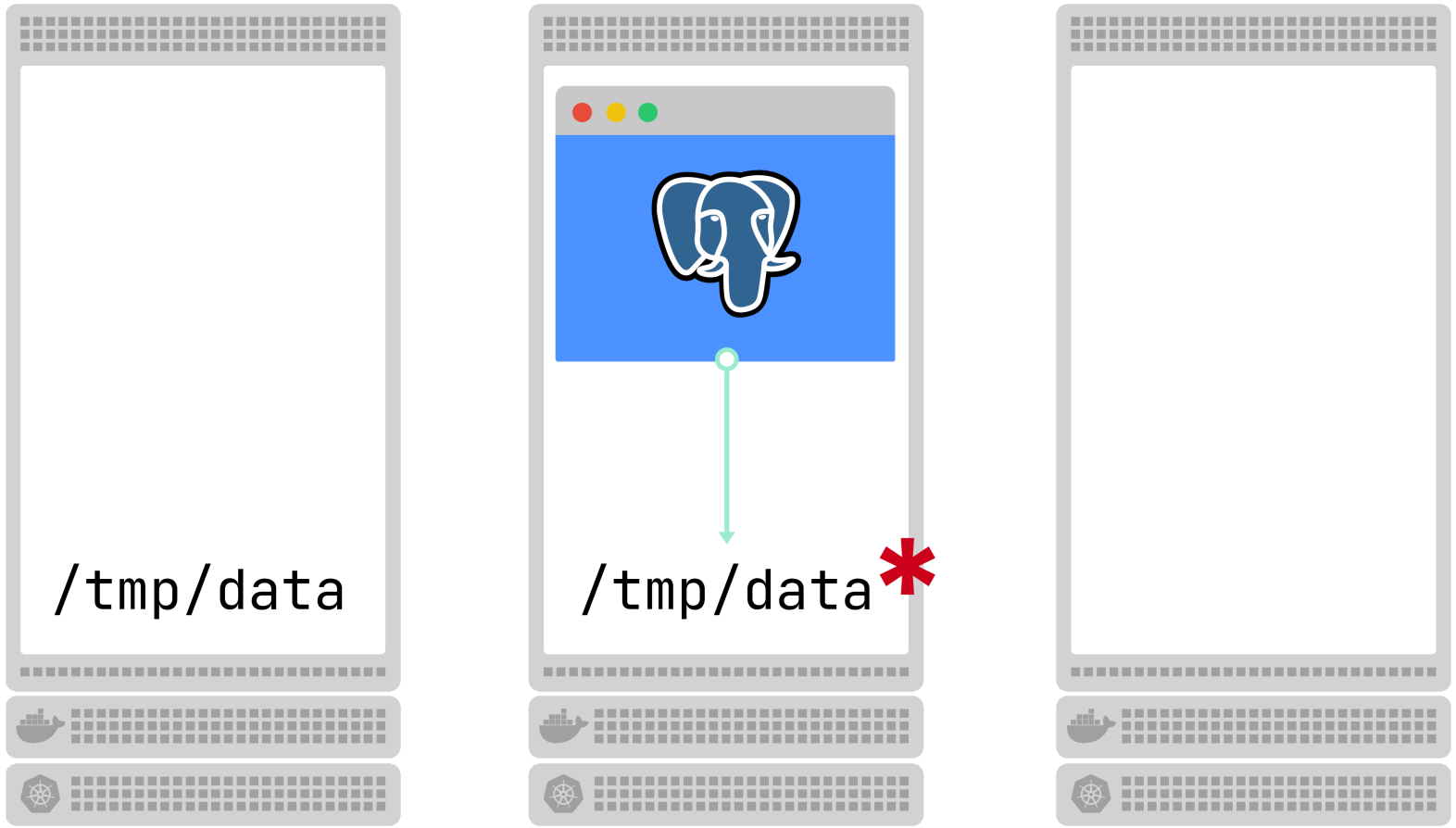
Mount folder from node

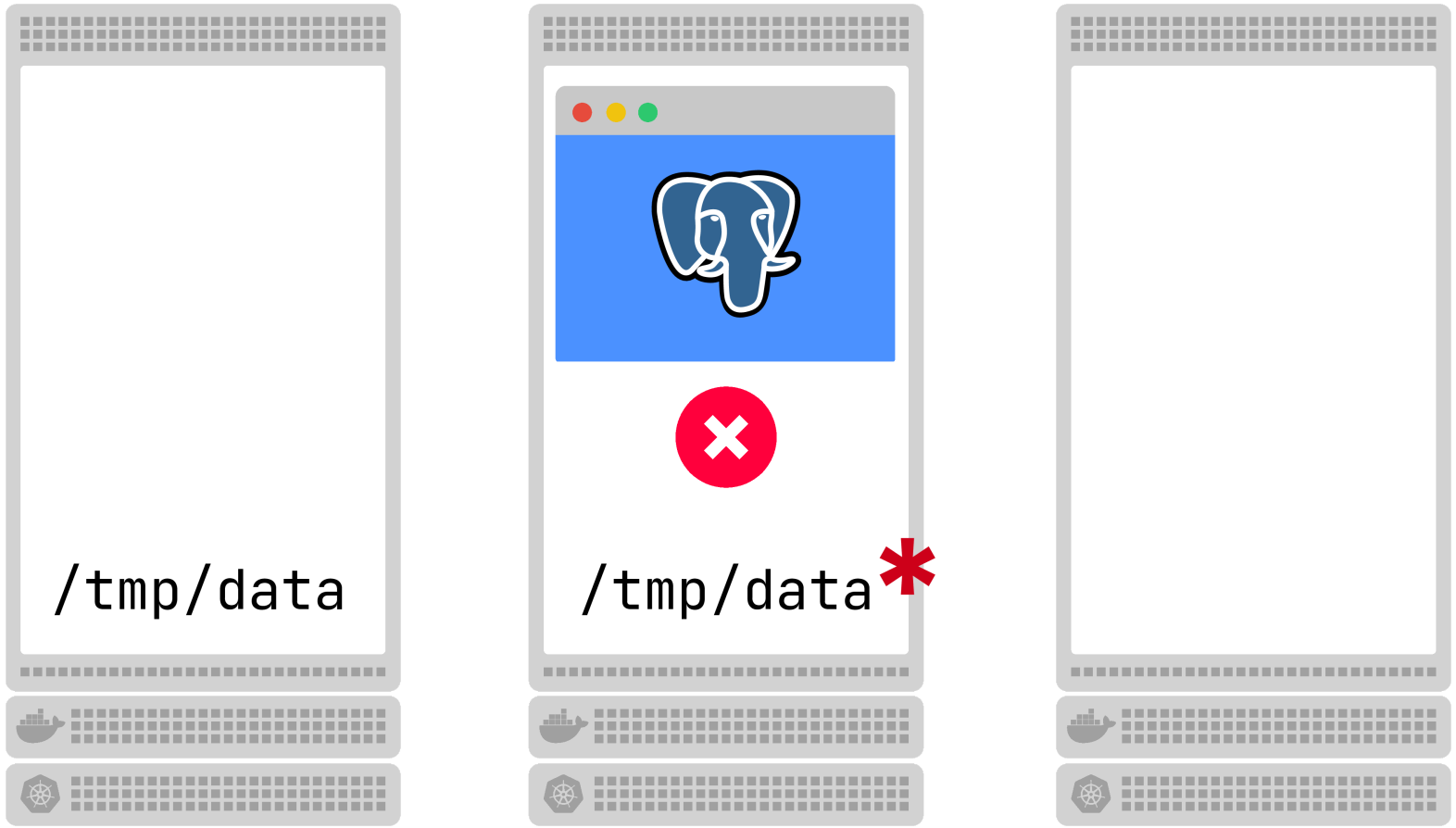
GPUs, TUPs, /proc, etc.

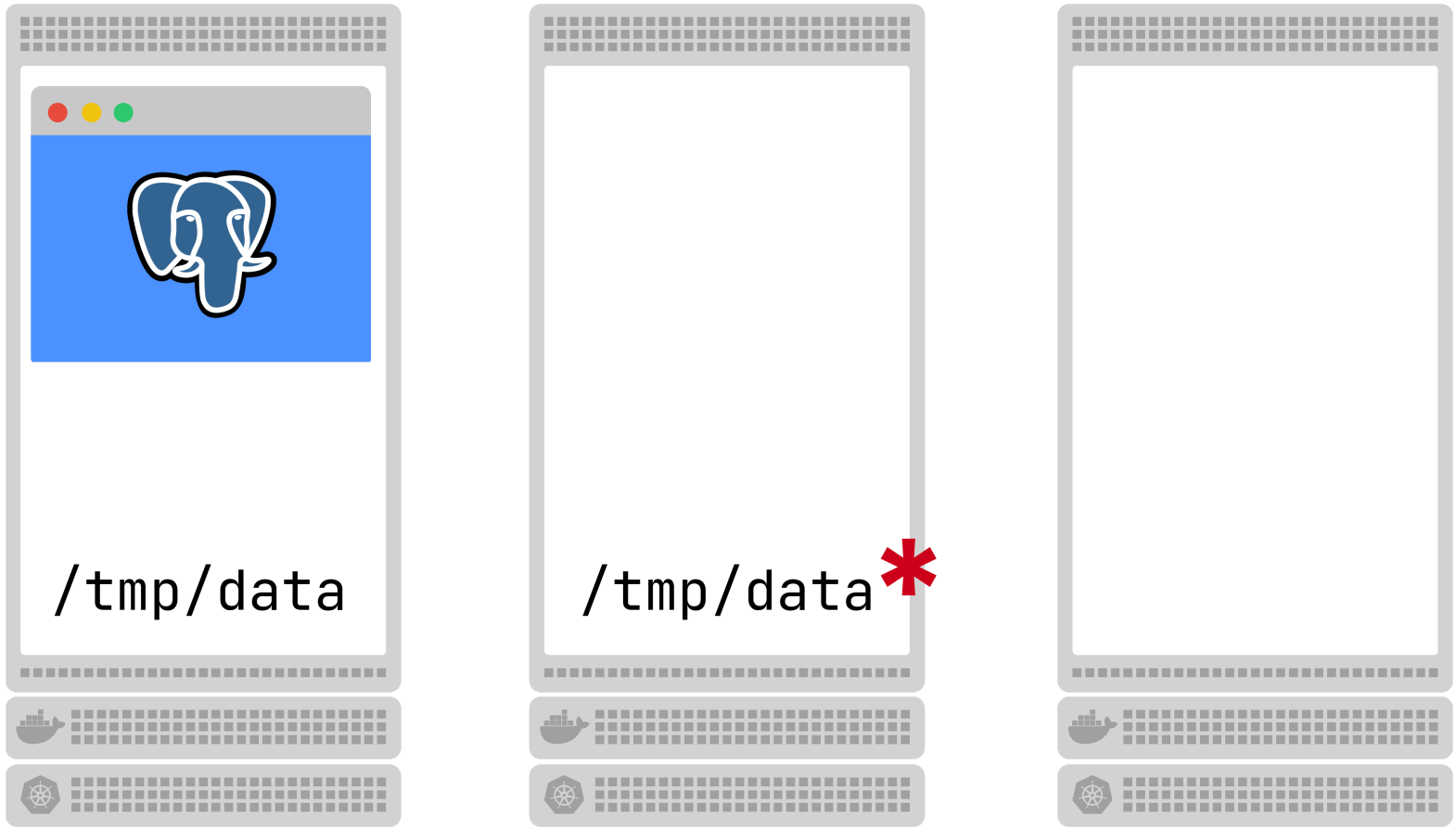
Ignored by scheduler

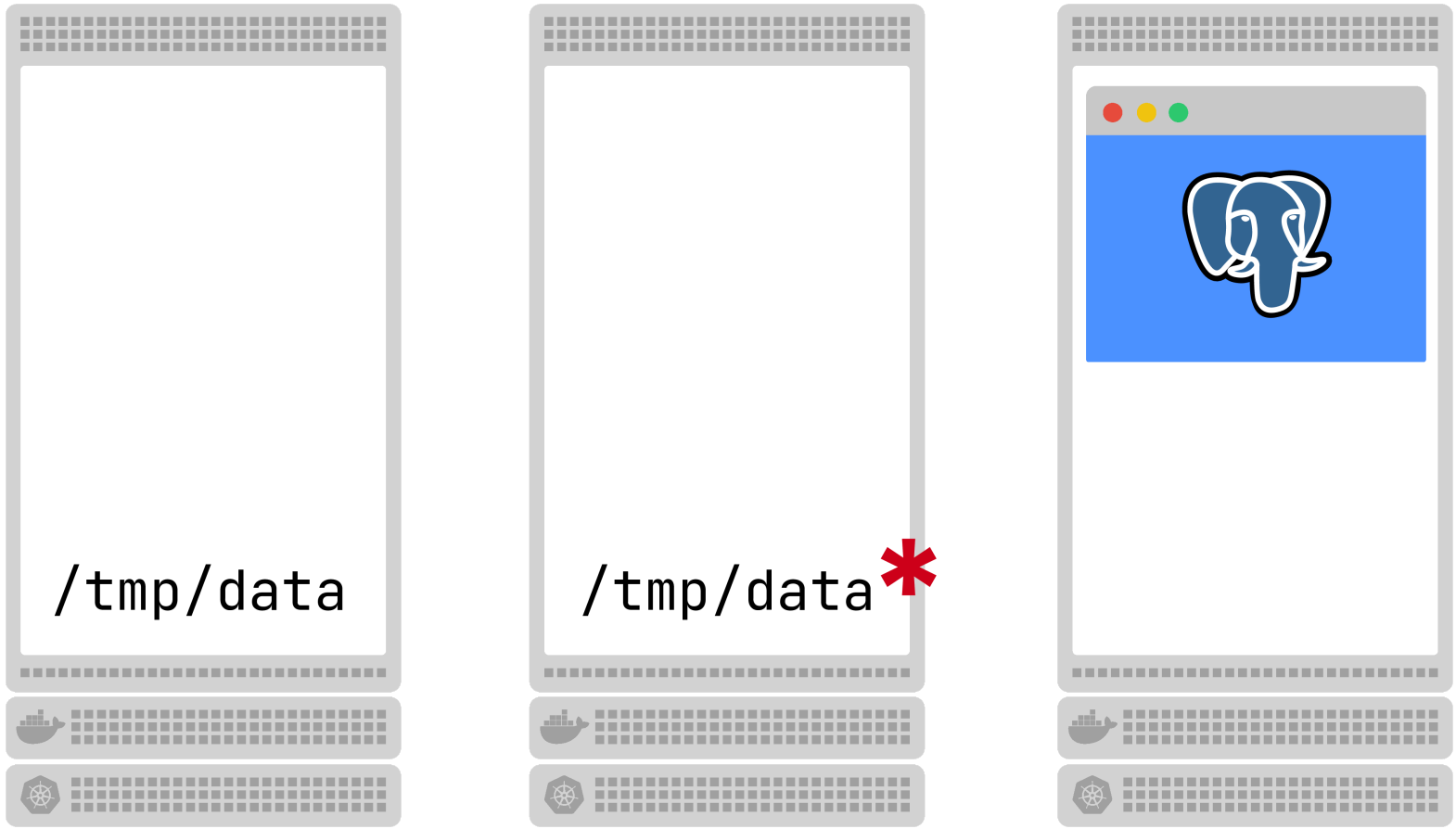
hostPath in practice


/tmp/data
/tmp/data


/tmp/data
/tmp/data*


/tmp/data
/tmp/data*


/tmp/data
/tmp/data*


/tmp/data
/tmp/data*

configMap and secret

03

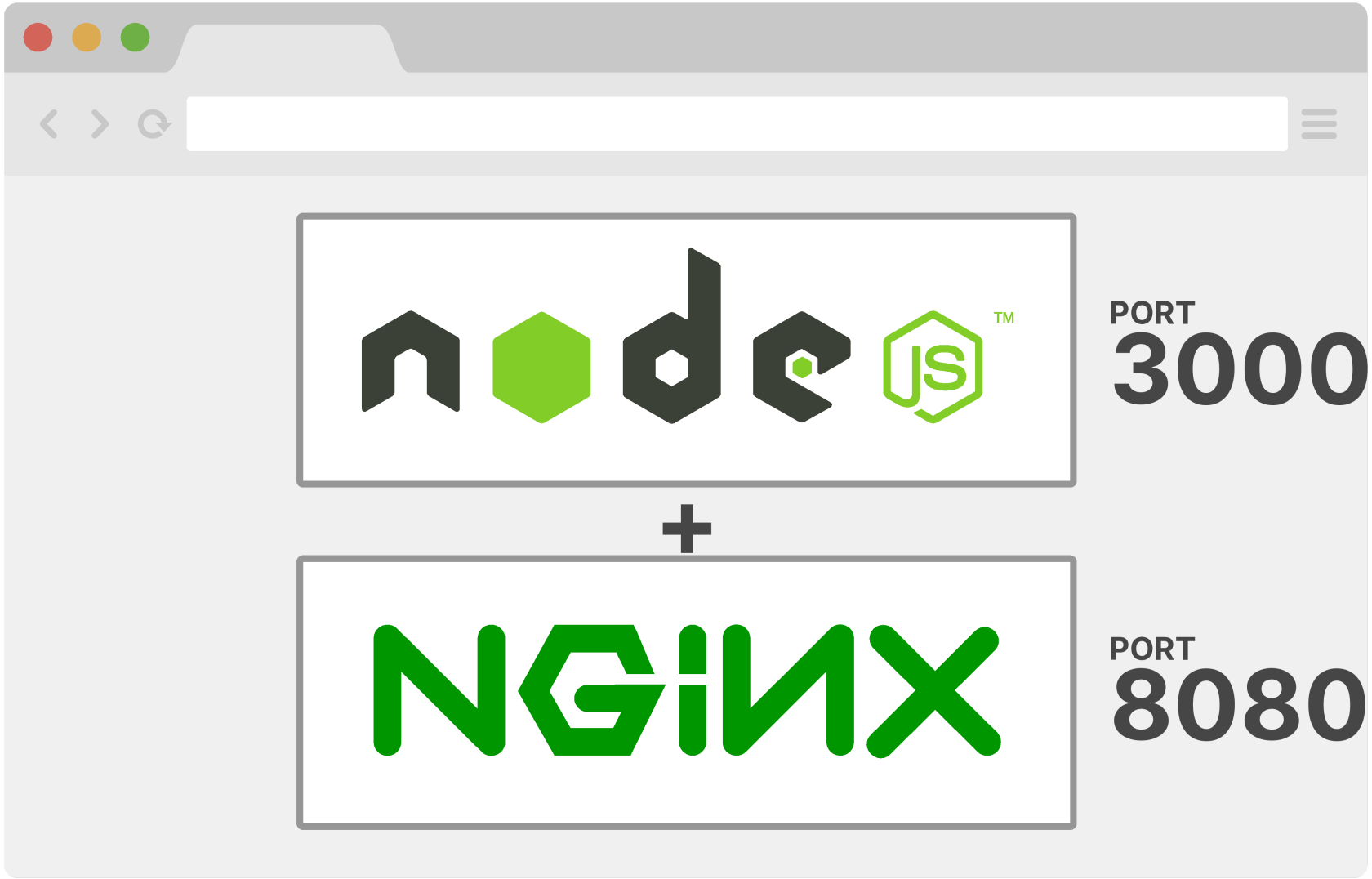
```
~$ cat pod.yaml
apiVersion: v1
kind: Pod
spec:
  containers:
  - name: mypod
    image: redis
    volumeMounts:
    - name: foo
      mountPath: "/etc/foo"
  volumes:
  - name: foo
    secret:
      secretName: mysecret
~$ _
```

```
~$ cat pod.yaml
apiVersion: v1
kind: Pod
spec:
  containers:
  - name: mypod
    image: redis
    volumeMounts:
    - name: foo
      mountPath: "/etc/foo"
  volumes:
  - name: foo
    secret:
      secretName: mysecret
~$ _
```

```
~$ cat pod.yaml
apiVersion: v1
kind: Pod
spec:
  containers:
  - name: mypod
    image: redis
    volumeMounts:
    - name: foo
      mountPath: "/etc/foo"
  volumes:
  - name: foo
    secret:
      secretName: mysecret
~$ _
```

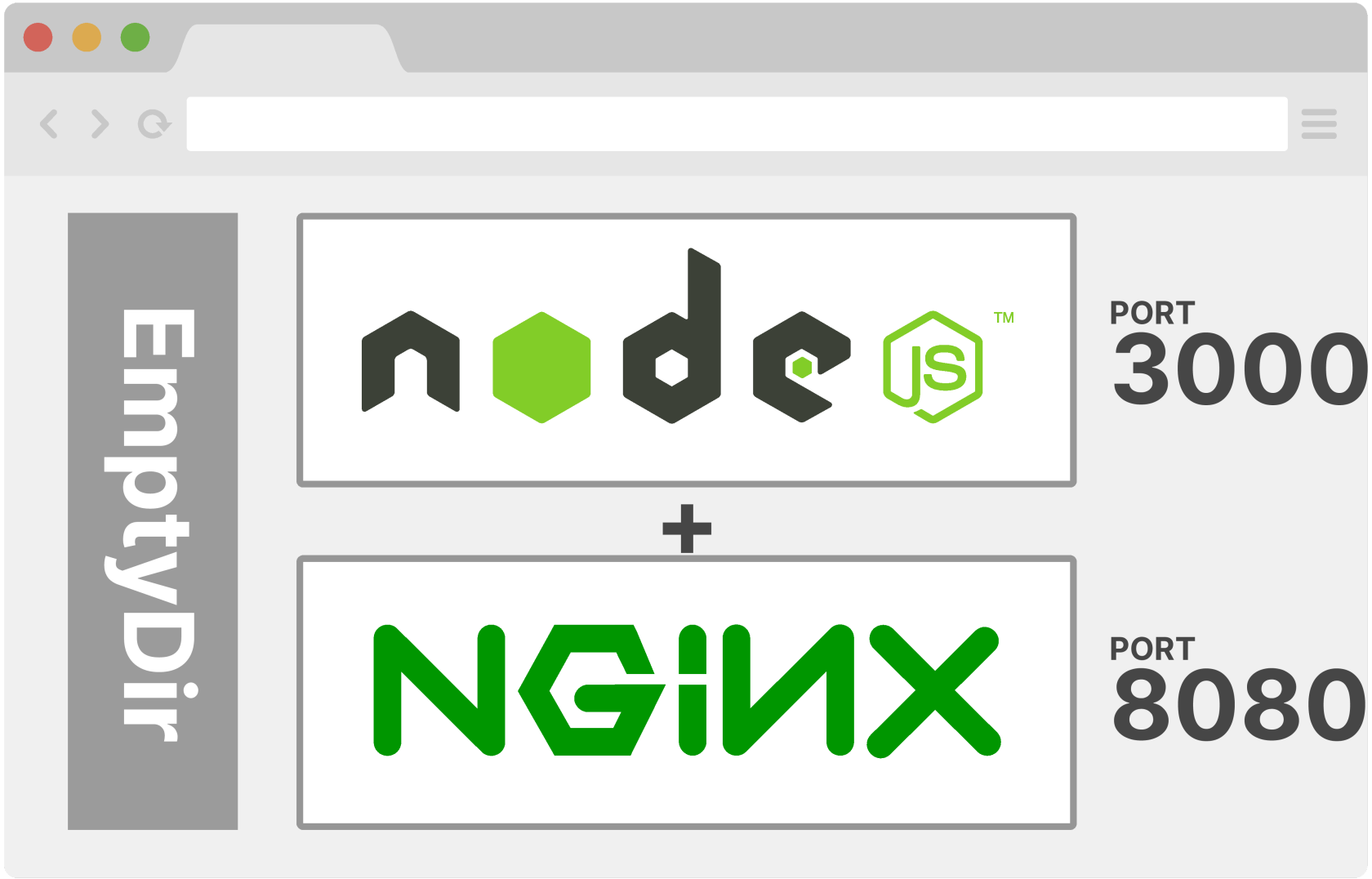
emptyDir

04



LOCALHOST

127.0.0.1



LOCALHOST

127.0.0.1

LOCALHOST

127.0.0.1

emptyDir

Data stored in the node

Containers can share FS

Data is lost when Pod dies

emptyDir

Data stored in the node

Containers can share FS

Data is lost when Pod dies

emptyDir

Data stored in the node

Containers can share FS

Data is lost when Pod dies

Flexvolumes and CSI

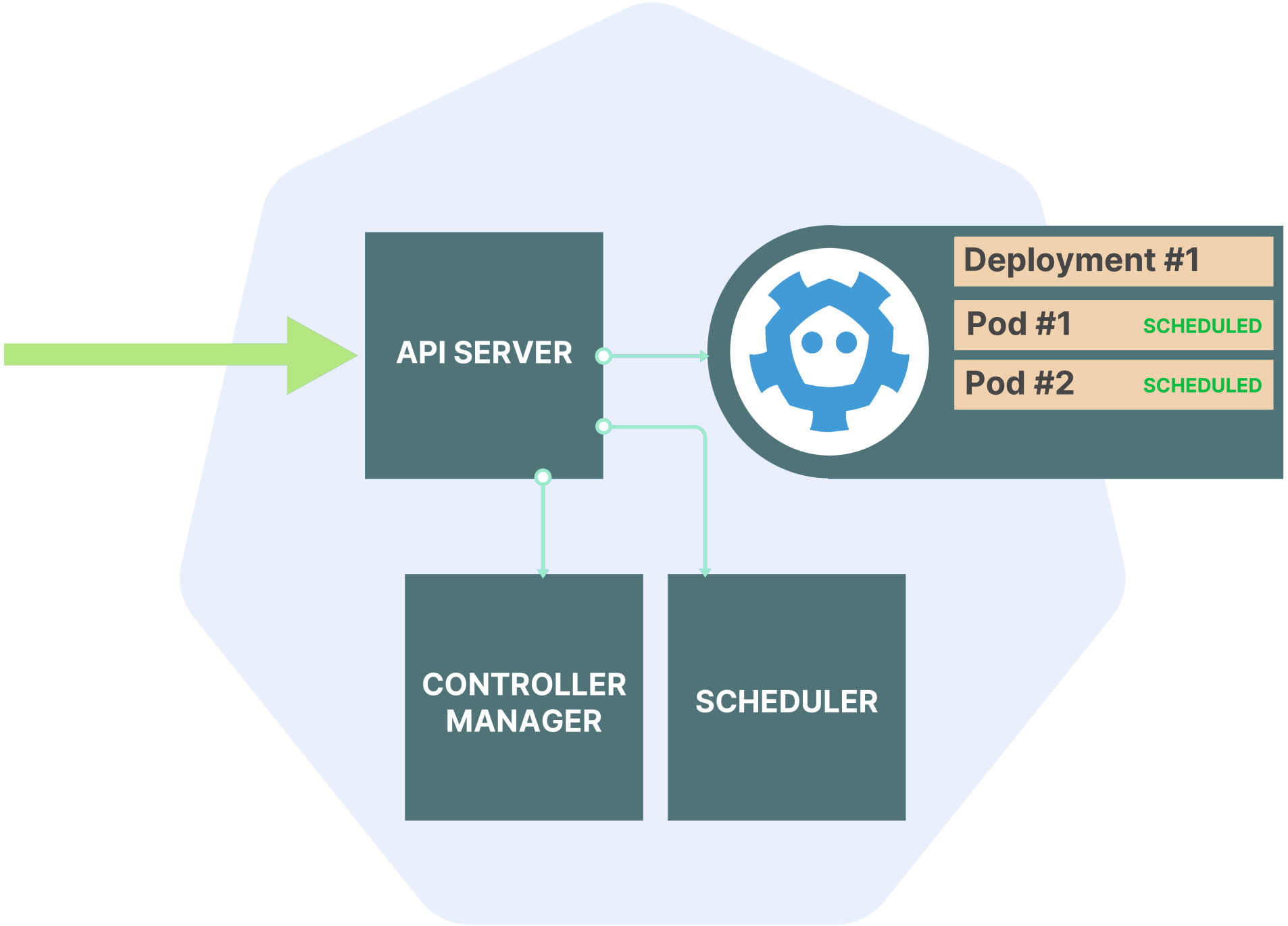
05

Before CSI and Flexvolume, all volume plugins were built, linked, compiled, and shipped with the core Kubernetes binaries

---

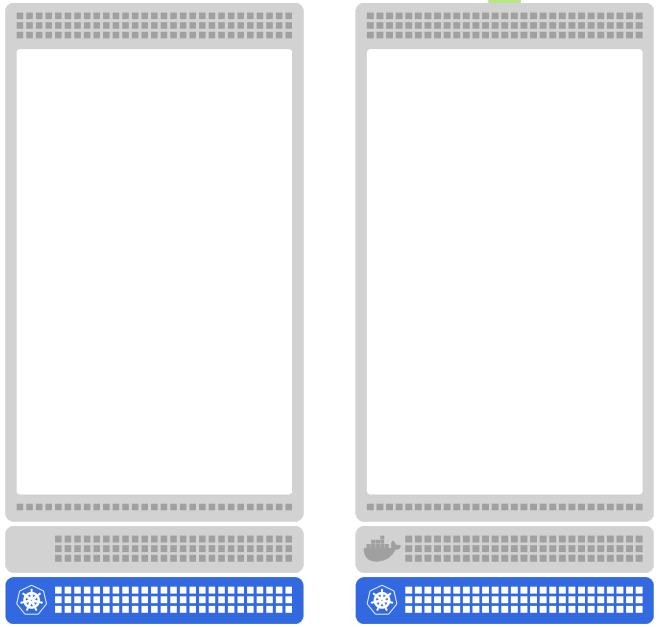
Flexvolumes and CSI

Create a 2 Pods deployment


API SERVER
Deployment #1
Pod #1
SCHEDULED
Pod #2
SCHEDULED
CONTROLLER MANAGER
SCHEDULER

Deployment with 2 replicas please

Is there anything for me?



Flexvolumes and CSI

1. CRI
2. CNI
3. CSI

Flexvolumes and CSI

1. CRI
2. CNI
3. CSI

CSI in practice

CSI driver

CSI volume

Pluggable

CSI in practice

CSI driver

CSI volume

Pluggable

CSI in practice

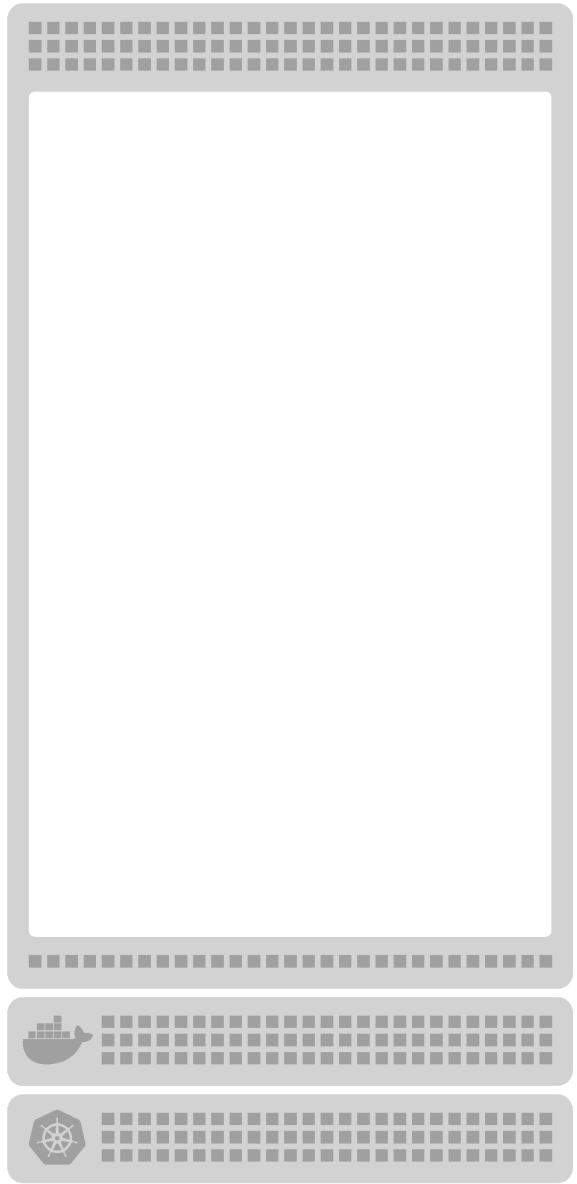
CSI driver

CSI volume

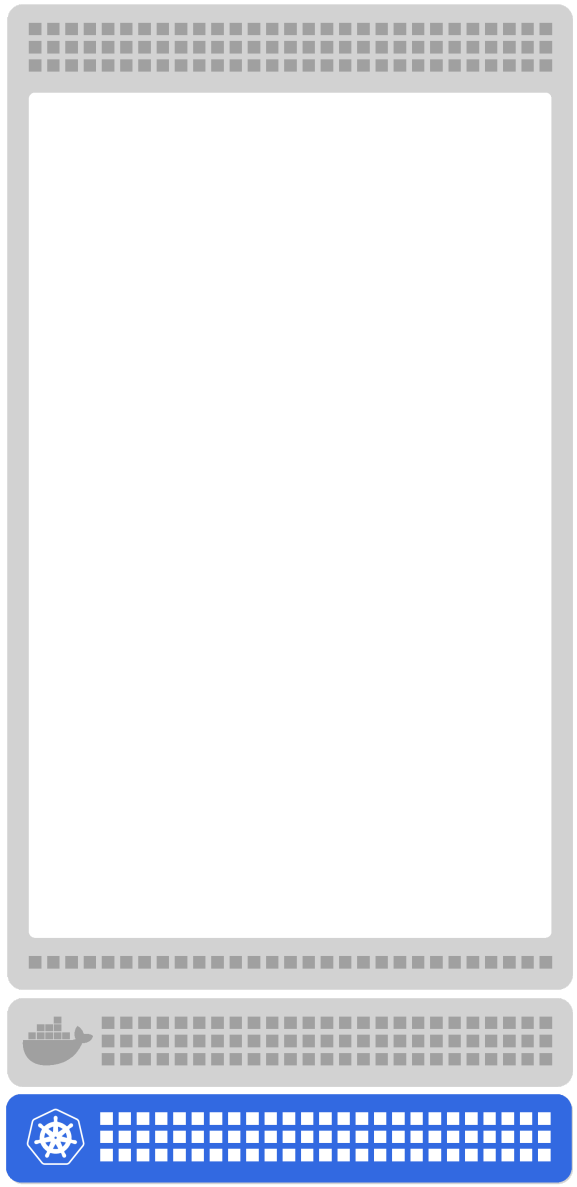
Pluggable

Azure/secrets-store-csi-driver-provider-azure

CSI in practice



Vault

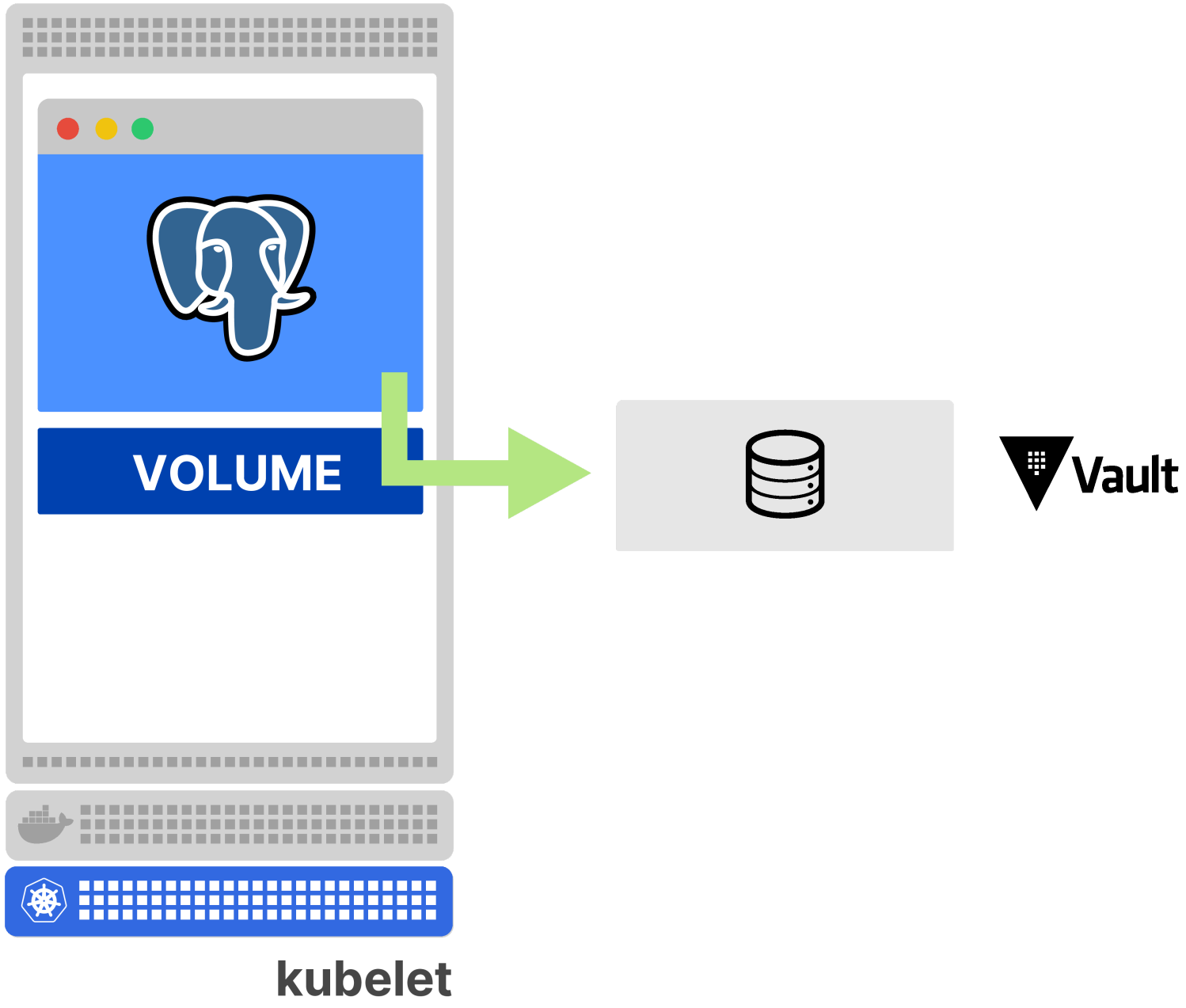

kubelet

Vault

```
~$ cat pod.yaml
kind: Pod
apiVersion: v1
metadata:
  name: nginx-secrets-store-inline
spec:
  containers:
  - image: nginx
    name: nginx
    volumeMounts:
    - name: secrets-store-inline
      mountPath: "/mnt/secrets-store"
      readOnly: true
  volumes:
    - name: secrets-store-inline
      csi:
        driver: secrets-store.csi.k8s.io
        readOnly: true
        volumeAttributes:
          secretProviderClass: "azure-kvname"
~$ _
```

```
~$ cat pod.yaml
kind: Pod
apiVersion: v1
metadata:
  name: nginx-secrets-store-inline
spec:
  containers:
  - image: nginx
    name: nginx
    volumeMounts:
    - name: secrets-store-inline
      mountPath: "/mnt/secrets-store"
      readOnly: true
  volumes:
    - name: secrets-store-inline
      csi:
        driver: secrets-store.csi.k8s.io
        readOnly: true
        volumeAttributes:
          secretProviderClass: "azure-kvname"
~$ _
```

```
~$ cat pod.yaml
kind: Pod
apiVersion: v1
metadata:
  name: nginx-secrets-store-inline
spec:
  containers:
  - image: nginx
    name: nginx
    volumeMounts:
    - name: secrets-store-inline
      mountPath: "/mnt/secrets-store"
      readOnly: true
  volumes:
    - name: secrets-store-inline
      csi:
        driver: secrets-store.csi.k8s.io
        readOnly: true
        volumeAttributes:
          secretProviderClass: "azure-kvname"
~$ _
```


VOLUME
Vault
kubelet

Recap

Popular volumes

Popular volumes

1. Local volumes
2. hostPath
3. ConfigMap and Secrets
4. emptyDir
5. CSI

Popular volumes

1. Local volumes

2. hostPath

3. ConfigMap and Secrets

4. emptyDir

5. CSI

Popular volumes

1. Local volumes
2. hostPath
3. ConfigMap and Secrets
4. emptyDir
5. CSI

Popular volumes

1. Local volumes
2. hostPath
3. ConfigMap and Secrets
4. **emptyDir**
5. CSI

Popular volumes

1. Local volumes
2. hostPath
3. ConfigMap and Secrets
4. emptyDir
5. CSI

Thanks.